



دانشکده مهندسی کامپیوتر
پروژه پایانی کارشناسی

عنوان پروژه:
طراحی و پیاده سازی سامانه هوشمند تشخیص دود و کنترل
تجهیزات خانگی مبتنی بر ESP32

نگارنده:
سامان صیاد فعال
۴۰۱۵۲۱۴۰۸

استاد راهنما:
دکتر امیر مهدی حسینی منزله

شهریور ماه ۱۴۰۵

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

چکیده

در این پروژه، یک سامانه هوشمند مبتنی بر برد ESP32 با هدف طراحی و پیاده سازی یک واحد یکپارچه برای پایش شرایط محیطی، تشخیص شرایط خطر و کنترل تجهیزات الکترونیکی توسعه داده شده است. هدف اصلی سامانه، فراهم کردن امکان دریافت و پردازش اطلاعات محیطی و انتقال آنها به یک سامانه نرم افزاری از طریق شبکه بی سیم، در کنار ایجاد قابلیت کنترل و ارسال فرمان به برخی تجهیزات الکترونیکی موجود در محیط است. به منظور پایش وضعیت محیط، دستگاه به مجموعه ای از حسگرها شامل حسگر گاز MQ-2، حسگر دما و رطوبت DHT11، حسگر شدت نور LDR و حسگر تشخیص حرکت PIR مجهز شده است. حسگر MQ-2 با استفاده از خروجی آنالوگ، امکان اندازه گیری سطح نسبی گاز و دود موجود در محیط را فراهم می کند و در صورت عبور مقدار اندازه گیری شده از آستانه تعیین شده، شرایط به عنوان وضعیت هشدار شناسایی شده و سامانه اقدام به فعال سازی هشدار صوتی می کند. همچنین اطلاعات مربوط به دما، رطوبت، شدت نور و وضعیت حرکت در بازه های زمانی مشخص از محیط دریافت شده و برای پایش و نمایش وضعیت سیستم در اختیار سامانه نرم افزاری قرار می گیرد. به منظور افزایش قابلیت کاربردی دستگاه در حوزه کنترل تجهیزات، دو بستر ارتباطی رادیویی RF در فرکانس های ۳۱۵ و ۴۳۳ مگاهرتز و همچنین ارتباط مادون قرمز IR در سامانه در نظر گرفته شده است. این بخش امکان دریافت و شناسایی فرمان های رادیویی و مادون قرمز، ذخیره سازی آنها و استفاده مجدد از فرمان های ذخیره شده برای کنترل تجهیزات سازگار را فراهم می کند. ارتباط دستگاه با شبکه از طریق قابلیت WiFi برد ESP32 برقرار شده و به منظور انتقال داده و فرمان میان دستگاه و سامانه نرم افزاری از پروتکل MQTT استفاده شده است. همچنین برای ساده سازی راه اندازی اولیه و جلوگیری از وابستگی به اطلاعات شبکه ثابت، یک سازوکار provisioning در دستگاه پیاده سازی شده است که از طریق یک نقطه دسترسی بی سیم و درگاه پیکربندی تحت وب، امکان تنظیم مشخصات شبکه WiFi، اطلاعات MQTT Broker را فراهم می کند. در بخش نرم افزاری کاربر نیز یک داشبورد تحت وب برای نمایش داده های دریافت شده از دستگاه و ارسال فرمان های کنترلی طراحی شده است. به این ترتیب، کاربر می تواند بدون نیاز به دسترسی مستقیم به سخت افزار، وضعیت محیط را پایش کرده، شرایط هشدار را مشاهده نموده و فرمان های مورد نیاز را برای تجهیزات ارسال کند. این سامانه می تواند به عنوان یک نمونه پایه برای توسعه راهکار های هوشمند سازی محیط های مسکونی و اداری، سامانه های پایش محیطی و سیستم های کنترل تجهیزات مورد استفاده قرار گیرد.

واژه های کلیدی: خانه هوشمند، اتوماسیون خانگی، ESP32، اینترنت اشیا (IoT)

فهرست مطالب

۱	فصل ۱: مقدمه
۲	۱-۱- بیان مسئله
۳	۱-۲- اهمیت و ضرورت پروژه
۴	۱-۳- اهداف پروژه
۵	۱-۴- محدوده پروژه
۵	۱-۵- روش انجام پروژه
۷	فصل ۲: مبانی نظری
۸	۲-۱- اینترنت اشیا
۸	۲-۱-۱- اجزای اصلی یک سامانه IoT
۹	۲-۱-۲- کاربرد اینترنت اشیا در خانه هوشمند
۹	۲-۲- سیستم های خانه هوشمند
۱۰	۲-۳- میکروکنترلر ESP32
۱۰	۲-۳-۲- GPIO
۱۱	۲-۳-۳- مبدل آنالوگ به دیجیتال ADC
۱۱	۲-۴- حسگر های محیطی
۱۱	۲-۴-۱- حسگر MQ-2
۱۲	۲-۴-۲- حسگر DHT11
۱۳	۲-۴-۳- حسگر شدت نور LDR
۱۴	۲-۴-۴- حسگر تشخیص حرکت PIR
۱۴	۲-۵- بازر
۱۵	۲-۶- ارتباط مادون قرمز
۱۶	۲-۷- ارتباط رادیویی RF
۱۷	۲-۸- ارتباط WiFi
۱۸	۲-۸-۱- Captive Portal
۱۸	۲-۹- پروتکل MQTT
۱۸	۲-۹-۱- MQTT Broker
۱۹	۲-۹-۲- کیفیت سرویس
۱۹	۲-۹-۳- ارتباط MQTT از طریق WebSocket
۲۰	۲-۱۰- ذخیره سازی اطلاعات در ESP32
۲۱	فصل ۳: تحلیل و طراحی سیستم
۲۲	۳-۱- تحلیل نیازمندی های سیستم
۲۴	۳-۲- معماری کلی سیستم
۲۴	۳-۲-۲- لایه سخت افزار و جمع آوری داده
۲۵	۳-۲-۳- لایه پردازش و کنترل
۲۵	۳-۲-۴- لایه ارتباطی و رابط کاربر
۲۶	۳-۳- طراحی سخت افزار
۲۶	۳-۳-۱- واحد پردازش مرکزی و اتصال حسگرها
۲۷	۳-۳-۲- سیستم ارتباط فرکانس رادیویی
۲۷	۳-۳-۳- سیستم ارتباط مادون قرمز
۲۸	۳-۳-۴- سیستم هشدار و نمایش وضعیت

فهرست مطالب

۲۹	۳-۴- طراحی نرم افزار و ساختار Firmware
۲۹	۳-۴-۱- ساختار ماژولار Firmware
۳۰	۳-۴-۲- راه اندازی و اجرای اصلی سیستم
۳۱	۳-۴-۳- مدیریت حسگرها و منطق هشدار
۳۲	۳-۵- طراحی ارتباط WiFi و MQTT
۳۳	۳-۵-۱- پیکربندی و ارتباط WiFi
۳۳	۳-۵-۲- ارتباط MQTT
۳۴	۳-۶- طراحی سیستم RF
۳۴	۳-۶-۱- دریافت و ذخیره سازی سیگنال RF
۳۵	۳-۶-۲- بازپخش و مدیریت سیگنال RF
۳۵	۳-۷- طراحی سیستم مادون قرمز
۳۶	۳-۷-۱- دریافت و ذخیره سازی سیگنال IR
۳۶	۳-۷-۲- بازپخش و مدیریت سیگنال IR
۳۶	۳-۸- طراحی سیستم ذخیره سازی
۳۷	۳-۸-۱- ذخیره تنظیمات و اطلاعات دستگاه
۳۷	۳-۸-۲- ذخیره سازی سیگنال RF و IR
۳۸	۳-۹- طراحی سیستم Provisioning و Captive Portal
۳۸	۳-۱۰- طراحی Web Dashboard

فصل ۴: پیاده سازی سامانه

۴۱	۴-۱- محیط و ابزار های توسعه
۴۲	۴-۲- پیاده سازی سخت افزار
۴۲	۴-۲-۱- پیاده سازی حسگر های محیطی
۴۵	۴-۲-۲- پیاده سازی سیستم هشدار صوتی
۴۶	۴-۳- پیاده سازی سیستم ارتباط رادیویی RF
۴۷	۴-۳-۱- دریافت و ذخیره سازی سیگنال RF
۴۷	۴-۳-۲- بازپخش و مدیریت سیگنال RF
۴۹	۴-۳-۳- منطق کلی دریافت و ارسال RF
۴۹	۴-۴- پیاده سازی سیستم مادون قرمز IR
۵۰	۴-۴-۱- دریافت و ذخیره سازی سیگنال IR
۵۱	۴-۴-۲- بازپخش و مدیریت سیگنال IR
۵۳	۴-۴-۳- منطق کلی دریافت و ارسال IR
۵۳	۴-۵- پیاده سازی ارتباط WiFi و MQTT
۵۳	۴-۵-۱- ایجاد و راه اندازی MQTT Client
۵۴	۴-۵-۲- برقراری و مدیریت اتصال WiFi و MQTT
۵۶	۴-۵-۳- دریافت و انتشار فرمان های MQTT
۵۷	۴-۶- پیاده سازی پیکربندی اولیه و مدیریت اتصال به شبکه
۵۷	۴-۶-۱- ساختار اطلاعات و راه اندازی
۵۹	۴-۶-۲- پیاده سازی Captive Portal و مدیریت درخواست های HTTP
۶۰	۴-۶-۳- دریافت، اعتبارسنجی و ذخیره تنظیمات
۶۰	۴-۷- پیاده سازی مدیریت حافظه و ذخیره سازی اطلاعات
۶۱	۴-۷-۱- ذخیره و بازیابی وضعیت دستگاه
۶۱	۴-۷-۲- ذخیره و بازیابی سیگنال های RF و IR

فهرست مطالب

۶۱	۴-۸- راه اندازی و هماهنگی ماژول ها.....
۶۳	۴-۸-۱- مدیریت فرمان های سیستم.....
۶۴	۴-۸-۲- مدیریت حسگرها و وظایف دوره ای.....
۶۵	۴-۹- پیاده سازی داشبورد وب و رابط کاربری.....
۶۵	۴-۹-۲- اتصال به MQTT Broker و دریافت اطلاعات.....
۶۶	۴-۹-۳- ارسال فرمان و مدیریت وضعیت سامانه.....
۶۷	فصل ۵: آزمون و ارزیابی سیستم
۶۸	۵-۱- روش آزمون و ارزیابی سامانه.....
۶۸	۵-۲- آزمون عملکرد حسگرها.....
۶۹	۵-۲-۱- آزمون حسگر دما و رطوبت.....
۶۹	۵-۲-۲- آزمون حسگر نور.....
۷۰	۵-۲-۳- آزمون حسگر دود و آتش.....
۷۱	۵-۲-۴- آزمون حسگر تشخیص حرکت.....
۷۱	۵-۳- آزمون سامانه هشدار.....
۷۲	۵-۴- آزمون اتصال به WiFi و MQTT Broker.....
۷۲	۵-۵- آزمون داشبورد سامانه.....
۷۴	۵-۶- آزمون سیستم RF.....
۷۴	۵-۶-۱- آزمون دریافت سیگنال RF.....
۷۴	۵-۶-۲- آزمون ارسال سیگنال RF.....
۷۵	۵-۶-۳- آزمون مدیریت سیگنال RF.....
۷۵	۵-۷- آزمون سیستم IR.....
۷۶	۵-۷-۱- آزمون دریافت سیگنال IR.....
۷۶	۵-۷-۲- آزمون ارسال سیگنال IR.....
۷۷	۵-۷-۳- آزمون مدیریت سیگنال IR.....
۷۷	۵-۸- آزمون زیرسامانه Provisioning.....
۷۹	فصل ۶: نتیجه گیری
۸۰	۶-۱- جمع بندی مراحل انجام پروژه.....
۸۰	۶-۲- نتایج حاصل از پیاده سازی.....
۸۱	۶-۳- محدودیت های پروژه.....
۸۱	۶-۴- پیشنهاد های توسعه در آینده.....
۸۲	۶-۵- جمع بندی نهایی.....
۸۳	فصل ۷: مراجع
۸۵	فصل ۸: پیوست ها

فهرست اشکال

۱۰	 شکل (۱-۲) میکروکنترلر ESP32
۱۲	 شکل (۲-۲) حسگر MQ-2
۱۲	 شکل (۳-۲) حسگر DHT11
۱۳	 شکل (۴-۲) حسگر شدت نور
۱۴	 شکل (۵-۲) حسگر تشخیص حرکت PIR
۱۴	 شکل (۶-۲) بازو
۱۵	 شکل (۷-۲) گیرنده و فرستنده مادون قرمز
۱۶	 شکل (۸-۲) گیرنده رادیویی
۱۶	 شکل (۹-۲) فرستنده رادیویی
۲۴	 شکل (۱-۳) معماری کلی سیستم
۳۰	 شکل (۲-۳) روند راه اندازی سیستم
۳۱	 شکل (۳-۳) روند اجرای حلقه اصلی سیستم
۳۱	 شکل (۴-۳) روند مدیریت حسگر ها
۳۱	 شکل (۵-۳) روند مدیریت حسگر MQ-2
۴۲	 شکل (۱-۴) روند ساخت و بارگذاری Firmware
۴۹	 شکل (۲-۴) فرایند دریافت سیگنال RF
۴۹	 شکل (۳-۴) فرایند ارسال سیگنال RF
۵۳	 شکل (۴-۴) فرایند دریافت سیگنال IR
۵۳	 شکل (۵-۴) فرایند ارسال سیگنال IR
۵۹	 شکل (۶-۴) صفحه تنظیمات
۶۵	 شکل (۷-۴) داشبورد وب
۶۹	 شکل (۱-۵) نتایج حسگر دما و رطوبت
۶۹	 شکل (۲-۵) نتیجه حسگر نور
۷۰	 شکل (۳-۵) نتایج حسگر دود و آتش
۷۱	 شکل (۴-۵) نتایج حسگر تشخیص حرکت
۷۲	 شکل (۵-۵) نتیجه آزمون اتصال به شبکه و MQTT Broker
۷۲	 شکل (۶-۵) نمایش اطلاعات حسگرها در داشبورد
۷۳	 شکل (۷-۵) نمایش هشدار در داشبورد
۷۳	 شکل (۸-۵) ارسال فرمان از داشبورد
۷۴	 شکل (۹-۵) دریافت سیگنال RF
۷۴	 شکل (۱۰-۵) ارسال سیگنال RF
۷۵	 شکل (۱۱-۵) مدیریت سیگنال RF
۷۶	 شکل (۱۲-۵) دریافت سیگنال IR
۷۶	 شکل (۱۳-۵) ارسال سیگنال IR
۷۷	 شکل (۱۴-۵) مدیریت سیگنال IR
۷۸	 شکل (۱۵-۵) صفحه تنظیمات Provisioning

فهرست جداول

۲۸	جدول (۱-۳) پایه های ESP32
۲۹	جدول (۲-۳) وظیفه ماژول های Firmware
۳۳	جدول (۳-۳) Topic های مربوط به حسگرها
۳۸	جدول (۴-۳) مشخصات AP
۵۸	جدول (۱-۴) ساختار داده تنظیمات Provisioning

فصل ۱:

مقدمه

در سال‌های اخیر، توسعه فناوری‌های ارتباطی و سامانه‌های نهفته موجب گسترش استفاده از سیستم‌های هوشمند در محیط‌های مسکونی، اداری و صنعتی شده است. یکی از مهم‌ترین حوزه‌های حاصل از این توسعه، اینترنت اشیا (Internet of Things یا IoT) است. اینترنت اشیا به شبکه‌ای از تجهیزات فیزیکی اطلاق می‌شود که با بهره‌گیری از حسگرها، پردازنده‌ها، نرم‌افزارها و واسط‌های ارتباطی، قابلیت جمع‌آوری، پردازش، تبادل اطلاعات و در برخی موارد اعمال کنترل بر محیط پیرامون خود را دارند. [1] در چنین سامانه‌هایی، حسگرها وظیفه دریافت اطلاعات از محیط، واحد پردازشی وظیفه تحلیل و مدیریت داده‌ها و بخش ارتباطی وظیفه انتقال اطلاعات و دریافت فرمان‌های کنترلی را بر عهده دارد [1] [2].

یکی از کاربردهای مهم اینترنت اشیا، توسعه سامانه‌های هوشمند خانگی است. در یک محیط هوشمند، تجهیزات مختلف می‌توانند از طریق یک شبکه ارتباطی با یکدیگر و با کاربر تعامل داشته باشند. این تعامل می‌تواند شامل پایش شرایط محیطی، تشخیص رخداد‌های غیرعادی، ارسال هشدار، کنترل تجهیزات الکتریکی و ثبت اطلاعات باشد. استفاده از چنین سامانه‌هایی می‌تواند امکان نظارت از راه دور و کاهش وابستگی به کنترل دستی تجهیزات را فراهم کند.

در این پروژه، یک هاب هوشمند مبتنی بر برد ESP32 طراحی و پیاده سازی شده است. ESP32 یک سامانه روی تراشه با قابلیت WiFi در باند ۲.۴ گیگاهرتز، پردازنده ۳۲ بیتی و مجموعه‌ای از واسط‌های ورودی/خروجی دیجیتال و آنالوگ است که توسط شرکت Espressif ارائه شده و برای کاربردهایی از جمله تجهیزات هوشمند خانگی و اینترنت اشیا مناسب است [3]. این ویژگی‌ها باعث شده است ESP32 بتواند علاوه بر پردازش داده‌های حسگرها، ارتباط شبکه‌ای و کنترل تجهیزات جانبی را نیز در یک واحد سخت‌افزاری انجام دهد.

سیستم توسعه یافته در این پروژه علاوه بر تشخیص دود و گاز، امکان اندازه‌گیری دما، رطوبت و شدت نور و همچنین تشخیص حرکت را فراهم می‌کند. همچنین برای افزایش قابلیت کنترل تجهیزات مختلف، دو مجموعه ارتباط رادیویی در فرکانس‌های ۳۱۵ و ۴۳۳ مگاهرتز و یک مجموعه ارتباط مادون قرمز در نظر گرفته شده است. به این ترتیب، سیستم قادر است برخی سیگنال‌های رادیویی و مادون قرمز را دریافت، ذخیره و در صورت نیاز بازپخش کند.

ارتباط میان هاب هوشمند و زیرساخت نرم‌افزاری سیستم از طریق WiFi و پروتکل MQTT انجام می‌شود. MQTT یک پروتکل ارتباطی سبک مبتنی بر الگوی انتشار/اشتراک (Publish/Subscribe) است که به طور ویژه برای محیط‌های دارای محدودیت منابع و کاربردهای ماشین به ماشین و اینترنت اشیا مناسب طراحی شده است [4]. در این پروژه، داده‌های حسگرها از طریق MQTT منتشر شده و فرمان‌های کنترلی نیز از طریق همین بستر به ESP32 ارسال می‌شوند.

از سوی دیگر، برای تعامل کاربر با سیستم، یک داشبورد تحت وب طراحی شده است که از طریق WebSocket به MQTT Broker متصل می‌شود. این داشبورد امکان مشاهده اطلاعات حسگرها، مشاهده هشدارها و ارسال فرمان‌های کنترلی را فراهم می‌کند. در نتیجه، پروژه حاضر را می‌توان یک هاب هوشمند چند منظوره دانست که وظایف پایش محیط، تشخیص رخداد، ایجاد هشدار، ارتباط شبکه‌ای و کنترل تجهیزات را در یک سامانه یکپارچه انجام می‌دهد.

۱-۱- بیان مسئله

در بسیاری از محیط‌های مسکونی و اداری، تجهیزات الکتریکی و سامانه‌های مختلف به صورت مستقل مورد استفاده قرار می‌گیرند و اطلاعات مربوط به شرایط محیطی نیز معمولاً به صورت متمرکز و مستمر جمع

آوری نمی شود. در روش های سنتی، تشخیص رخدادهایی مانند افزایش میزان دود یا گاز، تغییرات دما، حضور افراد یا تغییر شدید شرایط نوری ممکن است به صورت دستی یا توسط تجهیزات مستقل انجام شود. چنین رویکردی علاوه بر افزایش پیچیدگی سیستم، امکان پایش یکپارچه و کنترل از راه دور را محدود می کند.

از طرف دیگر، بسیاری از تجهیزات موجود در محیط های خانگی از روش های ارتباطی متفاوتی برای کنترل استفاده می کنند. برخی تجهیزات از ریموت های مادون قرمز و برخی دیگر از ریموت های رادیویی در فرکانس هایی مانند ۳۱۵ یا ۴۳۳ مگاهرتز استفاده می کنند. در نتیجه، برای کنترل یکپارچه این تجهیزات معمولاً نیاز به چندین کنترل کننده و واسط مجزا وجود دارد.

مسئله اصلی این پروژه، طراحی و پیاده سازی یک سامانه یکپارچه است که بتواند اطلاعات مربوط به شرایط محیط را از طریق مجموعه ای از حسگرها دریافت کرده، آنها را در یک واحد پردازشی مدیریت کند، اطلاعات را از طریق شبکه WiFi و MQTT در اختیار نرم افزار قرار دهد و امکان کنترل تجهیزات الکتریکی را نیز فراهم کند.

برای دستیابی به این هدف، از ESP32 به عنوان هسته پردازشی و ارتباطی سیستم استفاده شده است. در بخش پایش محیط، سنسور MQ-2 برای تشخیص دود و گازهای قابل اشتعال، سنسور DHT11 برای اندازه گیری دما و رطوبت، حسگر نوری LDR برای اندازه گیری نسبی شدت نور و حسگر حرکتی PIR برای تشخیص حرکت مورد استفاده قرار گرفته اند. اطلاعات به دست آمده از این حسگرها به صورت دوره ای پردازش و از طریق MQTT منتشر می شوند.

در بخش کنترل نیز سیستم از گیرنده و فرستنده های RF در دو باند ۳۱۵ و ۴۳۳ مگاهرتز و همچنین گیرنده و فرستنده مادون قرمز استفاده می کند. کتابخانه RC Switch امکان دریافت و ارسال کدهای برخی تجهیزات رادیویی رایج در فرکانس های ۳۱۵ و ۴۳۳ مگاهرتز را فراهم می کند [5]. همچنین کتابخانه IRremoteESP8266 برای دریافت و ارسال سیگنال های مادون قرمز مورد استفاده قرار گرفته است [6]. بنابراین، مسئله این پروژه را می توان به صورت طراحی یک هاب هوشمند تعریف کرد که سه وظیفه اصلی پایش شرایط محیطی، تشخیص رخداد و ایجاد هشدار و کنترل تجهیزات الکتریکی را در یک بستر واحد ترکیب می کند.

۲-۱- اهمیت و ضرورت پروژه

افزایش استفاده از تجهیزات الکتریکی و گسترش سیستم های هوشمند، نیاز به سامانه هایی را افزایش داده است که بتواند علاوه بر کنترل تجهیزات، وضعیت محیط را نیز پایش کنند. یکی از مزیت های اصلی استفاده از اینترنت اشیا در چنین سامانه هایی، ایجاد ارتباط میان دنیای فیزیکی و سامانه های نرم افزاری است؛ به گونه ای که اطلاعات حسگرها می توانند دریافت، منتقل و در بخش نرم افزاری پردازش شوند.

وجود چندین حسگر در یک سامانه امکان جمع آوری همزمان اطلاعات مختلف محیطی را فراهم می کند. در پروژه حاضر، اطلاعات مربوط به دما، رطوبت، شدت نور، میزان دود و وضعیت حرکت در اختیار سیستم قرار می گیرد. این اطلاعات می توانند برای بررسی وضعیت لحظه ای محیط و تشخیص برخی شرایط غیرعادی مورد استفاده قرار گیرند.

استفاده از سنسور های محیطی امکان واکنش سریع تر سیستم نسبت به رخدادهایی مانند افزایش مقدار گاز یا دود و تشخیص حرکت را فراهم می کند. در پروژه حاضر، در صورت عبور مقدار خوانده شده از سنسور MQ-2 از آستانه تعیین شده، سیستم وضعیت هشدار را فعال کرده و پیام مربوطه را از طریق MQTT منتشر

می‌کند. همچنین در صورت تشخیص حرکت، امکان ایجاد هشدار و فعال سازی بازر وجود دارد. ارتباط WiFi و MQTT باعث می‌شود اطلاعات جمع آوری شده تنها در سطح خود دستگاه باقی نماند و بتوان آنها را به یک Broker و نرم‌افزارهای دیگر منتقل کرد. MQTT به دلیل استفاده از مدل Publish/Subscribe، امکان جداسازی تولیدکننده و مصرف کننده داده را فراهم می‌کند و برای ارتباط میان تجهیزات IoT مناسب است [4].

یکی از ویژگی‌های مهم پروژه، ترکیب پایش محیط با کنترل تجهیزات الکتریکی است. سیستم می‌تواند سیگنال‌های RF در دو باند ۳۱۵ و ۴۳۳ مگاهرتز را دریافت و ذخیره کرده و در زمان مناسب بازپخش کند. کتابخانه RCTransmit نیز برای کنترل تجهیزات رادیویی رایج با فرستنده‌ها و گیرنده‌های ۳۱۵ و ۴۳۳ مگاهرتز طراحی شده است [5]. همچنین استفاده از ارتباط مادون قرمز باعث می‌شود سیستم بتواند با برخی تجهیزات مجهز به ریموت IR تعامل داشته باشد. کتابخانه IRremoteESP8266 برای ESP32 امکان دریافت و ارسال سیگنال‌های مادون قرمز و پشتیبانی از پروتکل‌های متعدد را فراهم می‌کند [6].

ترکیب چندین قابلیت در یک برد ESP32 باعث می‌شود به جای استفاده از چند سیستم مستقل برای پایش محیط، دریافت فرمان‌های بی‌سیم و کنترل تجهیزات، بتوان بخش قابل توجهی از این وظایف را در یک هاب واحد پیاده‌سازی کرد. همچنین معماری ماژولار نرم‌افزار، استفاده از MQTT و قابلیت‌های ارتباطی ESP32 امکان توسعه سیستم در آینده را فراهم می‌کند. برای مثال، می‌توان حسگرهای جدید، تجهیزات کنترلی بیشتر یا سرویس‌های نرم‌افزاری دیگر را بدون تغییر اساسی در ساختار کلی سیستم به آن اضافه کرد.

۳-۱- اهداف پروژه

هدف اصلی این پروژه، طراحی و پیاده سازی یک سامانه هوشمند مبتنی بر ESP32 برای تشخیص دود و آتش و پایش محیط و کنترل تجهیزات الکتریکی از طریق شبکه WiFi است و همچنین اهداف فرعی پروژه عبارت‌اند از:

1. طراحی و پیاده سازی بخش سخت افزاری مبتنی بر ESP32
2. تشخیص وجود دود و شرایط مرتبط با آتش با استفاده از MQ-2
3. اندازه گیری دما و رطوبت محیط با استفاده از DHT11
4. اندازه گیری شدت نور محیط با استفاده از LDR
5. تشخیص حضور یا حرکت در محیط با استفاده از حسگر PIR
6. ایجاد سیستم هشدار صوتی با استفاده از بازر
7. دریافت سیگنال‌های مادون قرمز از کنترلر تجهیزات مختلف
8. ذخیره فرمان‌های مادون قرمز دریافت شده
9. ارسال سیگنال‌های مادون قرمز برای کنترل تجهیزات مختلف
10. دریافت و ارسال سیگنال‌های رادیویی برای کنترل تجهیزات رادیویی
11. برقراری ارتباط دستگاه با شبکه WiFi
12. استفاده از MQTT برای تبادل اطلاعات میان دستگاه و سامانه نرم افزاری

13. ایجاد قابلیت پایش سیستم از راه دور
14. فراهم کردن امکان کنترل تجهیزات الکتریکی از طریق سامانه نرم افزاری
15. ایجاد ساختاری قابل توسعه برای اضافه کردن حسگرها و قابلیت های جدید در آینده

۴-۱- محدوده پروژه

محدوده این پروژه شامل طراحی و پیاده سازی یک نمونه عملیاتی از سامانه هوشمند پایش و کنترل محیط مبتنی بر ESP32 است. در این پروژه، ESP32 به عنوان واحد پردازش مرکزی وظیفه دریافت اطلاعات از حسگرها، پردازش داده ها، مدیریت ارتباطات و ارسال یا دریافت سیگنال های مادون قرمز و رادیویی را بر عهده دارد.

- تشخیص دود و شرایط مرتبط با آتش
 - اندازه گیری دما و رطوبت و شدت نور
 - تشخیص حرکت
 - ارسال و دریافت سیگنال های مادون قرمز
 - ارسال و دریافت سیگنال های رادیویی
 - اتصال دستگاه به WiFi
 - ارتباط با سامانه نرم افزاری
 - ارسال اطلاعات حسگرها به سامانه نرم افزاری
 - دریافت فرمان از سامانه نرم افزاری برای کنترل تجهیزات الکتریکی
- با وجود قابلیت های فوق، پروژه دارای محدوده مشخصی است. برای مثال، عملکرد تشخیص آتش در این پروژه در سطح یک سامانه نمونه و آموزشی/مهندسی در نظر گرفته شده و نباید آن را به عنوان جایگزین تجهیزات ایمنی و اعلام حریق دارای تاییدیه استاندارد برای کاربرد های حیاتی در نظر گرفت. همچنین کنترل تجهیزات از طریق IR و RF به نوع پروتکل و سازگاری تجهیزات مقصد وابسته است.

۵-۱- روش انجام پروژه

فرایند انجام پروژه به صورت مرحله ای و بر اساس نیازمندی های سخت افزاری و نرم افزاری سیستم انجام شده است. در مرحله نخست، مسئله و نیازمندی های اصلی سیستم شامل تشخیص شرایط خطر، پایش محیط و کنترل تجهیزات الکتریکی بررسی شد. سپس معماری کلی سیستم و نحوه ارتباط میان حسگرها، ESP32 و سامانه نرم افزاری مشخص گردید. در مرحله بعد، قطعات سخت افزاری مورد نیاز انتخاب و مدار اولیه طراحی شد. ESP32 به عنوان کنترل کننده مرکزی انتخاب شد؛ زیرا علاوه بر قابلیت پردازشی، امکانات ارتباطی مورد نیاز پروژه از جمله WiFi را در اختیار قرار می دهد. پس از طراحی سخت افزار، Firmware دستگاه توسعه داده شد. این بخش وظیفه خواندن اطلاعات حسگرها، تشخیص وضعیت های مختلف، مدیریت ورودی و خروجی ها، دریافت و ارسال سیگنال های IR و RF و همچنین مدیریت ارتباط شبکه را بر عهده دارد. در بخش ارتباطی، دستگاه از طریق WiFi به شبکه متصل شده و ارتباط میان ESP32 و

سامانه نرم افزاری با استفاده از MQTT برقرار شده است. در این معماری، دستگاه می تواند اطلاعات مربوط به حسگرها و وضعیت خود را در Topic های مشخص منتشر کند و در مقابل، پیام های کنترلی را از Topic های مربوطه دریافت نماید. در ادامه بخش نرم افزاری توسعه داده شده و قابلیت مشاهده اطلاعات و ارسال فرمان های کنترلی در آن فراهم شده است. پس از اتصال اجزای مختلف، آزمایش های مربوط به حسگرها، ارتباط WiFi، تبادل پیام MQTT و عملکرد فرستنده و گیرنده های IR و RF انجام شده است. در نهایت، عملکرد کلی سامانه با بررسی سناریو های مختلف مورد ارزیابی قرار گرفته است.

فصل ۲:

مبانی نظری

گسترش فناوری های ارتباطی، میکروکنترلر های کم مصرف و شبکه های بی سیم باعث شده است که تجهیزات الکترونیکی بتوانند علاوه بر انجام وظایف کنترلی، اطلاعات محیط پیرامون خود را نیز دریافت و از طریق شبکه منتقل کنند. این رویکرد یکی از پایه های اصلی شکل گیری اینترنت اشیا (Internet of Things یا IoT) است.

در سامانه های هوشمند، ترکیب حسگر ها، واحد پردازش، ارتباط شبکه و نرم افزار کاربردی امکان پایش شرایط محیطی و کنترل تجهیزات را فراهم می کند. در یک خانه هوشمند، چنین سامانه ای می تواند اطلاعاتی مانند دما، رطوبت، شدت نور، وجود دود یا آتش را دریافت کرده و در مقابل، تجهیزات مختلف را از طریق ارتباطات بی سیم کنترل کند.

پروژه حاضر یک سیستم هوشمند مبتنی بر ESP32 است که علاوه بر پایش شرایط محیطی، قابلیت تشخیص دود و آتش، تشخیص حرکت، ایجاد هشدار صوتی و دریافت و ارسال فرمان های مادون قرمز و رادیویی را فراهم می کند. ارتباط دستگاه با زیرساخت شبکه از طریق WiFi و تبادل داده با استفاده از پروتکل MQTT انجام می شود.

۲-۱- اینترنت اشیا

اینترنت اشیا به مجموعه ای از تجهیزات فیزیکی گفته می شود که دارای حسگر، قابلیت پردازش و ارتباط شبکه بوده و می توانند اطلاعات مربوط به محیط یا وضعیت خود را جمع آوری و با سایر سامانه ها مبادله کنند [1]. در یک سامانه IoT معمولاً سه وظیفه اصلی دریافت اطلاعات از محیط، پردازش و تفسیر اطلاعات، ارسال اطلاعات یا اجرای فرمان وجود دارد. در پروژه حاضر، حسگرهای MQ-2، DHT11، LDR و PIR وظیفه دریافت اطلاعات محیطی را بر عهده دارند. ESP32 وظیفه پردازش اولیه را انجام داده و WiFi و MQTT برای انتقال اطلاعات به کار می روند.

۲-۱-۱ اجزای اصلی یک سامانه IoT

یک سامانه اینترنت اشیا از مجموعه ای از اجزای سخت افزاری و نرم افزاری تشکیل می شود که وظیفه دریافت اطلاعات از محیط، پردازش داده ها، برقراری ارتباط و ارائه خدمات به کاربر را بر عهده دارند. معمولاً می توان عملکرد کلی آنها را در چهار بخش اصلی شامل لایه حسگر، لایه پردازش، لایه ارتباطی و لایه کاربرد بررسی کرد. در این ساختار، داده ابتدا از محیط دریافت می شود، سپس توسط واحد پردازشی مورد پردازش قرار گرفته و از طریق بستر ارتباطی به سایر اجزای سامانه منتقل می شود. در نهایت، داده های پردازش شده در اختیار کاربر یا سایر سرویس های کاربردی قرار گرفته و امکان انجام عملیات کنترلی نیز فراهم می شود.

۲-۱-۱-۱ لایه حسگر

لایه حسگر نخستین بخش سامانه IoT است که مستقیماً با محیط فیزیکی ارتباط دارد. وظیفه اصلی این لایه، دریافت کمیت ها و رخداد های فیزیکی موجود در محیط و تبدیل آنها به داده ای است که واحد پردازشی بتواند آن را دریافت و تحلیل کند. کمیت هایی مانند دما، رطوبت، شدت نور، وجود دود یا گاز و حرکت افراد از جمله اطلاعاتی هستند که می توانند توسط حسگر های مختلف اندازه گیری شوند. نوع

حسگر های مورد استفاده در این لایه به کاربرد سامانه بستگی دارد و معمولاً برای پوشش داده چندین جنبه از وضعیت محیط، از مجموعه ای از حسگر ها به صورت هم زمان استفاده می شود.

۲-۱-۱-۲- لایه پردازش

لایه پردازش وظیفه دریافت داده های حاصل از لایه حسگر، مدیریت آنها و اجرای منطق کنترلی سامانه را بر عهده دارد. این لایه در واقع هسته تصمیم گیری سامانه IoT محسوب می شود و می تواند عملیاتی مانند خواندن دوره ای حسگر ها، بررسی شرایط خاص، تشخیص رخداد ها، مدیریت هشدارها و آماده سازی داده ها برای انتقال به شبکه را انجام دهد. در سامانه های مبتنی بر میکروکنترلر، این وظایف معمولاً توسط یک برد پردازشی انجام می شود که علاوه بر قدرت پردازشی، رابط های ورودی و خروجی و امکانات ارتباطی مورد نیاز را در اختیار سیستم قرار می دهد.

۲-۱-۱-۳- لایه ارتباطی

لایه ارتباطی مسئول ایجاد ارتباط میان تجهیزات، واحد پردازشی و سرویس های نرم افزاری است. در یک سامانه IoT، داده های تولید شده توسط حسگرها زمانی ارزش کاربردی بیشتری پیدا می کنند که بتوان آنها را از طریق یک شبکه به سایر بخش های سیستم منتقل کرد. همچنین این لایه باید امکان انتقال اطلاعات در جهت معکوس، یعنی از سایر بخش های سیستم به دستگاه را نیز فراهم کند. انتخاب فناوری و پروتکل ارتباطی مناسب تاثیر مستقیمی بر نحوه عملکرد و توسعه پذیری سامانه IoT دارد.

۲-۱-۱-۴- لایه کاربرد

لایه کاربرد بالاترین سطح معماری سامانه IoT است و وظیفه آن تبدیل داده ها و قابلیت های لایه های پایین تر به خدمات قابل استفاده برای کاربر است. در این لایه، اطلاعات دریافت شده از تجهیزات و حسگر ها می توانند به شکل مناسب به کاربر نمایش داده شوند و در مقابل، فرمان های مورد نظر کاربر نیز از طریق رابط کاربری به سامانه ارسال شوند. در واقع این لایه نقطه تعامل مستقیم کاربر با سامانه محسوب می شود و باید بتواند وضعیت سیستم را به شکلی قابل فهم ارائه کند.

۲-۱-۲- کاربرد اینترنت اشیا در خانه هوشمند

خانه هوشمند یکی از مهم ترین حوزه های کاربرد اینترنت اشیا است. در یک سامانه خانه هوشمند، تجهیزات و حسگر های مختلف از طریق یک شبکه ارتباطی به یکدیگر متصل می شوند و امکان جمع آوری اطلاعات، پایش وضعیت و کنترل تجهیزات فراهم می شود. این تجهیزات می تواند شامل حسگر های محیطی، سیستم های هشدار، تجهیزات روشنایی، لوازم خانگی و سایر تجهیزات الکترونیکی باشند. اتصال این اجزا به یک سامانه مرکزی یا بستر نرم افزاری باعث می شود اطلاعات محیطی در یک محل قابل مشاهده بوده و بسیاری از عملیات کنترلی بدون نیاز به حضور فیزیکی کاربر انجام شوند.

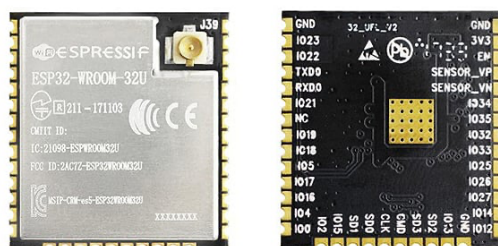
۲-۲- سیستم های خانه هوشمند

خانه هوشمند به محیطی اطلاق می شود که در آن تجهیزات و سامانه های مختلف با استفاده از فناوری های ارتباطی و پردازشی به یکدیگر متصل شده و امکان پایش، مدیریت و کنترل تجهیزات و شرایط محیطی

به صورت خودکار یا از راه دود فراهم می شود. در چنین سامانه هایی، اطلاعات محیط توسط حسگر های مختلف جمع آوری شده و پس از پردازش، از طریق یک بستر ارتباطی به بخش نرم افزاری منتقل می شوند. در مقابل، کاربر نیز می تواند با ارسال فرمان های کنترلی، عملکرد تجهیزات متصل به سامانه را تغییر دهد. فناوری هایی مانند WiFi، Bluetooth، Zigbee و Z-Wave از جمله فناوری های مورد استفاده در پیاده سازی سامانه های خانه هوشمند هستند.

۲-۳- میکروکنترلر ESP32

ESP32 خانواده ای از تراشه های سیستم روی تراشه (SoC¹) شرکت Espressif Systems است که برای کاربرد های ارتباطی و سیستم های نهفته طراحی شده است. این خانواده دارای پردازنده ۳۲ بیتی، حافظه، WiFi، ADC، Bluetooth و مجموعه ای از رابط های جانبی است. طبق مستندات رسمی، ESP32 از WiFi با استاندارد های 802.11b/g/n پشتیبانی کرده و دارای ADC دوازده بیتی، GPIO های متعدد، PWM، SPI، I2C و سایر واحد های جانبی است [3]. وجود WiFi داخلی یکی از دلایل اصلی انتخاب ESP32 برای این پروژه است؛ زیرا دستگاه می تواند بدون نیاز به ماژول ارتباطی جداگانه به شبکه متصل شود.



شکل (۲-۱) میکروکنترلر ESP32

۲-۳-۲- GPIO

GPIO² مخفف عبارت General Purpose Input/Output است و به پایه های ورودی و خروجی عمومی میکروکنترلر اطلاق می شود. این پایه ها قابلیت برنامه ریزی دارند و می توانند بسته به نیاز سیستم به عنوان ورودی یا خروجی مورد استفاده قرار گیرند. از طریق GPIO می توان وضعیت تجهیزات مختلف را دریافت یا سیگنال های کنترلی مورد نیاز را به تجهیزات خارجی ارسال کرد.

¹ مدار مجتمع یکپارچه ای که بخش های مختلف مورد نیاز برای پردازش و کنترل، از جمله واحد پردازنده، حافظه و برخی رابط های ارتباطی و جانبی را در یک تراشه ترکیب می کند.

² پایه های قابل برنامه ریزی میکروکنترلر که می توانند برای دریافت یا ارسال سیگنال های دیجیتال به تجهیزات و حسگر های خارجی مورد استفاده قرار گیرند.

۳-۳-۲- مبدل آنالوگ به دیجیتال ADC

ADC^۱ یا Analog-to-Digital Converter یکی از واحد های مهم ESP32 است که وظیفه تبدیل سیگنال های ولتاژ آنالوگ به مقادیر دیجیتال قابل پردازش توسط میکروکنترلر را بر عهده دارد. بسیاری از حسگرها خروجی خود را به صورت یک ولتاژ متغیر تولید می کنند و برای پردازش این سیگنال توسط میکروکنترلر، لازم است مقدار آن توسط ADC نمونه برداری و به یک مقدار دیجیتال تبدیل شود.

در پروژه حاضر، خروجی آنالوگ دو حسگر اصلی به ADC متصل شده است. این حسگرها شامل LDR برای اندازه گیری شدت نور محیط و MQ-2 برای تشخیص و اندازه گیری نسبی گازها و دود هستند. مقادیر خوانده شده از این حسگرها پس از تبدیل به داده دیجیتال، توسط ESP32 پردازش شده و می توانند برای پایش وضعیت محیط و اجرای منطق کنترلی سیستم مورد استفاده قرار گیرند.

ADC مورد استفاده در ESP32 دارای وضوح ۱۲ بیت است. بنابراین، در حالت ایده آل مقدار دیجیتال حاصل از تبدیل یک ورودی آنالوگ در بازه ۰ تا ۴۰۹۵ قرار می گیرد. در این حالت، مقدار صفر بیانگر کمترین مقدار قابل اندازه گیری و ۴۰۹۵ بیانگر بیشترین مقدار متناظر با محدوده اندازه گیری ADC است. مقدار واقعی خوانده شده می تواند تحت تاثیر عواملی مانند محدوده ولتاژ ورودی، کالیبراسیون ADC، نویز و شرایط مدار قرار گیرد.

۴-۲- حسگر های محیطی

حسگر های محیطی یکی از اجزای اصلی سامانه طراحی شده در این پروژه هستند و وظیفه دریافت اطلاعات مربوط به شرایط محیط اطراف را بر عهده دارند. داده های حاصل از این حسگرها توسط میکروکنترلر ESP32 دریافت و پردازش شده و در ادامه، برای پایش وضعیت محیط و اجرای عملکردهای مختلف سیستم مورد استفاده قرار می گیرند.

۱-۴-۲- حسگر MQ-2



^۱ مبدلی که سیگنال الکتریکی آنالوگ را به یک مقدار دیجیتال قابل پردازش توسط واحد پردازنده تبدیل می کند.

MQ-2 یکی از حسگر های گازی پر کاربرد از خانواده حسگر های گازی مقاومتی است که به منظور تشخیص وجود برخی گاز های قابل اشتعال و همچنین دود در محیط مورد استفاده قرار می گیرد. این حسگر به دلیل ساختار ساده و هزینه نسبتاً پایین، در پروژه های آموزشی، سامانه های پایش محیطی و برخی کاربردهای آزمایشگاهی مورد استفاده قرار می گیرد. مطابق اطلاعات فنی منتشر شده، این حسگر برای تشخیص موادی مانند LPG، متان، پروپان، بوتان، الکل، هیدروژن و دود کاربرد دارد [7].

عملکرد MQ-2 بر اساس تغییر مقاومت الکتریکی ماده حساس به گاز در اثر قرار گرفتن در معرض گاز های مورد نظر است. با تغییر غلظت گاز در محیط، مقاومت عنصر حساس تغییر کرده و در نتیجه، مشخصه الکتریکی خروجی حسگر نیز تغییر می کند. در مازول های متداول MQ-2، این تغییر می تواند به صورت یک ولتاژ آنالوگ در اختیار میکروکنترلر قرار گیرد.

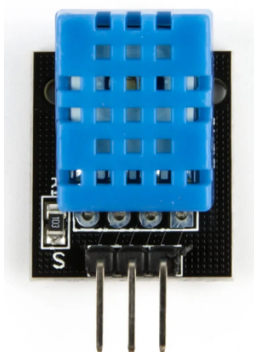
پیش گرمایش MQ-2

حسگر های خانواده MQ برای دستیابی به شرایط کاری نسبتاً پایدار به یک دوره گرم شدن نیاز دارند. به همین دلیل، مقدار خروجی حسگر بلافاصله پس از روشن شدن نمی تواند به عنوان یک مقدار پایدار و قابل اتکا برای تصمیم گیری در نظر گرفته شود. در این پروژه نیز پس از روشن شدن سیستم، یک دوره پیش گرمایش برای حسگر MQ-2 در نظر گرفته شده است تا عنصر حساس حسگر به شرایط کاری مناسب نزدیک شود [7].

محدودیت MQ-2

MQ-2 یک حسگر عمومی برای تشخیص وجود دود و برخی گاز های قابل اشتعال است و مقدار خروجی آن به عوامل مختلفی از جمله نوع و غلظت گاز، دمای محیط، رطوبت، مدت زمان پیش گرمایش و شرایط مدار وابسته است. بنابراین، خروجی این حسگر را نمی توان بدون کالیبراسیون و تحلیل مناسب به عنوان اندازه گیری دقیق غلظت یک گاز خاص در نظر گرفت. همچنین، MQ-2 به تنهایی یک سامانه استاندارد تشخیص و اعلام حریق محسوب نمی شود. در پروژه حاضر، این حسگر به عنوان یکی از منابع اطلاعاتی برای پایش شرایط محیط و تشخیص اولیه دود یا وجود گاز مورد استفاده قرار گرفته است و تصمیم گیری سیستم می تواند با در نظر گرفتن سایر اطلاعات محیطی انجام شود.

۲-۴-۲- حسگر DHT11



شکل (۲-۳) حسگر DHT11

DHT11 یک حسگر دیجیتال برای اندازه‌گیری هم زمان دما و رطوبت نسبی است. بر خلاف حسگر هایی که خروجی آنها به صورت آنالوگ ارائه می شود، DHT11 داده های اندازه گیری شده را به صورت دیجیتال و از طریق یک رابط ارتباطی مشخص به میکروکنترلر منتقل می کند. استفاده از DHT11 در این پروژه امکان پایش دو پارامتر مهم محیطی دما و رطوبت را فراهم می کند.

ارتباط DHT11 با میکروکنترلر از طریق یک خط داده و بر اساس یک پروتکل زمانبندی شده انجام می شود. در این فرایند، میکروکنترلر درخواست دریافت اطلاعات را به حسگر ارسال کرده و سپس DHT11 داده های مربوط به دما و رطوبت را از طریق خط ارتباطی به میکروکنترلر منتقل می کند. در این پروژه برای ساده سازی فرایند ارتباط و دریافت داده ها از DHT11، از کتابخانه Adafruit DHT استفاده شده است. این کتابخانه امکان ایجاد شی DHT و خواندن مستقیم دما و رطوبت را فراهم می کند. مستندات رسمی این کتابخانه توابعی مانند `readHumidity()` و `readTemperature()` را ارائه می دهد [8].

۳-۴-۲- حسگر شدت نور LDR



شکل (۳-۴) حسگر شدت نور

LDR یا Light Dependant Resistor یک مقاومت وابسته به نور است که مقدار مقاومت الکتریکی آن با میزان نور دریافتی تغییر می کند. این ویژگی باعث می شود بتوان از LDR به عنوان یک عنصر ساده برای تشخیص و پایش شدت نور محیط استفاده کرد. از آنجا که میکروکنترلر ESP32 برای دریافت این اطلاعات نیازمند یک سیگنال قابل اندازه گیری است، LDR معمولاً در کنار یک مقاومت دیگر به صورت مدار تقسیم ولتاژ قرار می گیرد. در این ساختار، تغییر مقاومت LDR باعث تغییر ولتاژ خروجی مدار شده و این ولتاژ می تواند توسط ADC میکروکنترلر اندازه‌گیری شود.

۲-۴-۴ حسگر تشخیص حرکت PIR



شکل (۲-۵) حسگر تشخیص حرکت PIR

حسگر PIR یا Passive Infrared Sensor یکی از حسگر های رایج برای تشخیص حرکت اجسام گرم، به ویژه انسان، در محیط است. این حسگر با استفاده از تغییرات تابش مادون قرمز موجود در محدوده دید خود، حرکت را تشخیص می دهد. تمام اجسام دارای دمای بالاتر از صفر مطلق، متناسب با دمای خود تابش الکترومغناطیسی منتشر می کنند که بخش قابل توجهی از آن در محدوده مادون قرمز قرار دارد. بدن انسان نیز به دلیل داشتن دمای بالاتر از محیط، تابش مادون قرمز منتشر می کند. هنگامی که یک جسم گرم در محدوده تشخیص حسگر حرکت می کند، الگوی تابش مادون قرمز دریافت شده توسط حسگر تغییر کرده و این تغییر توسط مدار PIR تشخیص داده می شود. خروجی ماژول های PIR معمولاً به صورت یک سیگنال دیجیتال ارائه می شود. بنابراین، در صورت تشخیص حرکت، وضعیت خروجی تغییر کرده و میکروکنترلر می تواند این تغییر وضعیت را از طریق یکی از پایه های GPIO تشخیص دهد.

۲-۵-۲ بازر



شکل (۲-۶) بازر

بازر یک مبدل الکتریکی به صوتی است که با دریافت یک سیگنال الکتریکی مناسب، انرژی الکتریکی را به ارتعاشات مکانیکی و در نهایت صوت تبدیل می کند. از بازر ها به دلیل ساختار ساده و قابلیت تولید صدای

هشدار، در سامانه های الکترونیکی، تجهیزات اعلام وضعیت و سیستم های هشدار استفاده می شود. در پروژه حاضر، بازر به یکی از پایه های GPIO میکروکنترلر متصل شده و برای ایجاد هشدار صوتی مورد استفاده قرار گرفته است.

۲-۶- ارتباط مادون قرمز



شکل (۲-۷) گیرنده و فرستنده مادون قرمز

مادون قرمز یا ¹Infrared بخشی از طیف امواج الکترومغناطیسی است که طول موج آن بیشتر از نور قرمز مرئی است. این ناحیه از طیف الکترومغناطیسی در بسیاری از کاربرد های الکترونیکی و ارتباطی مورد استفاده قرار می گیرد. یکی از کاربرد های رایج مادون قرمز، استفاده در سیستم های کنترل از راه دور است که در آن اطلاعات و فرمان های دیجیتال با تغییرات زمانی شدت تابش مادون قرمز منتقل می شوند. کنترل از راه دور بسیاری از تجهیزات خانگی، از جمله تلویزیون، سیستم های تهویه مطبوع، پخش کننده های صوتی و تصویری و سایر تجهیزات الکترونیکی، بر پایه ارتباط مادون قرمز انجام می شود.

گیرنده IR وظیفه دریافت تابش مادون قرمز ارسال شده از کنترل کننده و تبدیل آن به یک سیگنال الکتریکی قابل پردازش توسط میکروکنترلر را بر عهده دارد. گیرنده با تشخیص تغییرات شدت تابش و زمان بندی پالس های دریافتی، الگوی سیگنال را در اختیار میکروکنترلر قرار می دهد. در مقابل، فرستنده IR وظیفه تبدیل سیگنال الکتریکی تولید شده توسط میکروکنترلر به تابش مادون قرمز و ارسال آن به سمت دستگاه مقصد را بر عهده دارد.

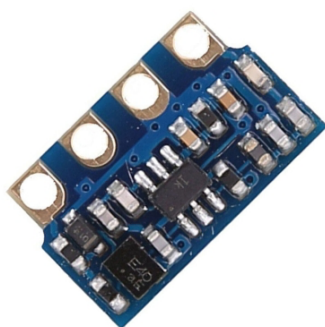
کنترل های از راه دور تجهیزات مختلف از پروتکل های ارتباطی متفاوتی برای کدگذاری و انتقال فرمان ها استفاده می کنند. هر پروتکل، ساختار و نحوه زمان بندی خاصی برای نمایش اطلاعات دارد. از جمله پروتکل های رایج مورد استفاده در تجهیزات الکترونیکی می توان به NEC، Sony، Samsung و Panasonic اشاره کرد. تفاوت میان این پروتکل ها در نحوه کدگذاری داده ها، ساختار فریم، زمان بندی پالس ها و سایر مشخصات سیگنال است.

در این پروژه برای پیاده سازی قابلیت دریافت و ارسال سیگنال های مادون قرمز از کتابخانه IRremoteESP8266 استفاده شده است. این کتابخانه یک پروژه متن باز برای پلتفرم های ESP8266 و ESP32 است که امکانات لازم برای دریافت، پردازش و ارسال سیگنال های مادون قرمز را فراهم می کند.

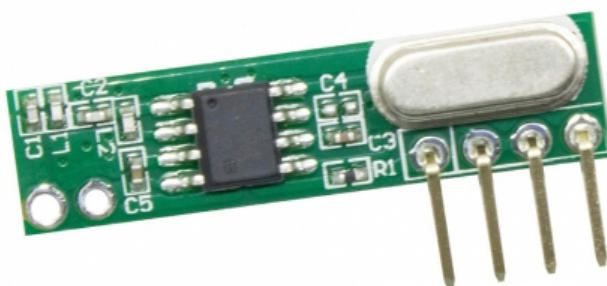
¹ نوعی تابش الکترومغناطیسی در محدوده فروسرخ است که در سامانه های کنترل از راه دور برای ارسال و دریافت فرمان ها مورد استفاده قرار می گیرد.

کتابخانه IRremoteESP8266 از طیف گسترده ای از پروتکل های رایج کنترل از راه دور و انواع مختلف گیرنده ها و فرستنده های مادون قرمز پشتیبانی می کند [6]. به این ترتیب، بخش مادون قرمز سیستم علاوه بر امکان دریافت فرمان های کنترل از راه دور، قابلیت ذخیره و بازپخش فرمان های IR را نیز فراهم می کند. این قابلیت یکی از بخش های اصلی عملکرد کنترلی دستگاه بوده و امکان استفاده از ESP32 به عنوان واسطی برای کنترل تجهیزات الکترونیکی مجهز به گیرنده مادون قرمز را فراهم می سازد.

۲-۷- ارتباط رادیویی RF



شکل (۲-۸) گیرنده رادیویی



شکل (۲-۹) فرستنده رادیویی

فرکانس رادیویی^۱ یا Radio Frequency به بخشی از طیف امواج الکترومغناطیسی گفته می شود که می توان از آن برای انتقال اطلاعات به صورت بی سیم استفاده کرد. ارتباطات RF در بسیاری از سامانه های الکترونیکی و تجهیزات کنترل از راه دور مورد استفاده قرار می گیرند و امکان انتقال داده بدون نیاز به اتصال فیزیکی میان فرستنده و گیرنده را فراهم می کند. یک سیستم رادیویی ساده می تواند از دو بخش اصلی، یعنی فرستنده و گیرنده تشکیل شود. فرستنده وظیفه تبدیل اطلاعات یا کد کنترلی به یک سیگنال رادیویی و ارسال آن را بر عهده دارد. در طرف مقابل، گیرنده سیگنال های رادیویی را دریافت کرده و آن را به یک

^۱ به محدوده ای از امواج الکترومغناطیسی گفته می شود که برای انتقال اطلاعات به صورت بی سیم مورد استفاده قرار می گیرند.

سیگنال الکتریکی قابل پردازش تبدیل می کند. در تجهیزات کنترل از راه دور رایج، باند های فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز از جمله فرکانس های مورد استفاده هستند.

کتابخانه RC Switch یکی از کتابخانه های پر کاربرد برای کار با برخی تجهیزات RF ساده در بلند های فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز است. این کتابخانه امکانات لازم برای دریافت و ارسال کد های رادیویی مورد استفاده در بسیاری از کنترل های از راه دور ساده را فراهم می کند [5]. با استفاده از RC Switch، می توان سیگنال های دریافتی از گیرنده RF را توسط میکروکنترلر دریافت و کد مربوط به آن را استخراج کرد. سپس در صورت نیاز، همان کد می تواند از طریق مازول فرستنده RF مجدداً ارسال شود. این قابلیت امکان پیاده سازی عملکرد دریافت، ذخیره و بازپخش کد های RF را در سیستم فراهم می کند.

۸-۲- ارتباط WiFi

WiFi یک فناوری ارتباطی بی سیم مبتنی بر مجموعه استاندارد های IEEE 802.11 است که امکان برقراری ارتباط و تبادل داده میان تجهیزات مختلف را از طریق شبکه بی سیم فراهم می کند. این فناوری به دلیل سهولت استفاده، پوشش مناسب و پشتیبانی گسترده در تجهیزات الکترونیکی، یکی از روش های متداول برای اتصال تجهیزات هوشمند و سامانه های اینترنت اشیا به شبکه محسوب می شود. میکروکنترلر ESP32 به صورت داخلی از قابلیت WiFi پشتیبانی می کند و امکان برقراری ارتباط با شبکه های بی سیم مبتنی بر استاندارد های 802.11b/g/n را فراهم می سازد [3]. وجود این قابلیت داخلی باعث می شود ESP32 بدون نیاز به استفاده از یک مازول ارتباطی خارجی بتواند به شبکه متصل شده و داده های مورد نیاز را ارسال یا دریافت کند.

ESP32 می تواند در چند حالت مختلف برای برقراری ارتباط WiFi مورد استفاده قرار گیرد. یکی از حالت های اصلی، Station یا STA است. در این حالت، ESP32 همانند یک Client به یک Access Point موجود، مانند روتر یا مودم، متصل می شود. پس از برقراری اتصال، ESP32 می تواند از طریق شبکه به سایر تجهیزات و سرویس های موجود دسترسی داشته باشد [9]. حالت دیگر، Access Point یا AP است. در این حالت، خود ESP32 نقش نقطه دسترسی بی سیم را بر عهده گرفته و یک شبکه WiFi ایجاد می کند. سایر تجهیزات می توانند مستقیماً به شبکه ایجاد شده توسط ESP32 متصل شوند. این حالت به ویژه در شرایطی که دستگاه هنوز به شبکه اصلی دسترسی ندارد یا نیاز به انجام تنظیمات اولیه وجود دارد، کاربرد دارد [9]. همچنین ESP32 از حالت AP+STA پشتیبانی می کند. در این حالت، ESP32 به صورت همزمان از طریق رابط STA به شبکه اصلی متصل شده و در نقش Access Point نیز یک شبکه WiFi ایجاد می کند. بنابراین، سایر تجهیزات می توانند به شبکه ایجاد شده توسط ESP32 متصل شوند، در حالی که خود ESP32 همچنان ارتباط خود را با شبکه اصلی حفظ کرده است [9]. این قابلیت در فرایند پیکربندی اولیه دستگاه می تواند کاربرد ویژه ای داشته باشد.

۱-۸-۲- Captive Portal

¹Captive Portal روشی برای ارائه یک صفحه وب یا رابط تنظیمات به کاربر پس از اتصال به یک شبکه است. در این روش، دستگاه کاربر پس از اتصال به شبکه بی سیم، به جای دسترسی مستقیم به مقصد مورد نظر، به صفحه ای هدایت می شود که می تواند برای انجام تنظیمات یا ورود اطلاعات مورد استفاده قرار گیرد. در پروژه حاضر، از Captive Portal به منظور پیکربندی اولیه دستگاه استفاده شده است. هنگامی که ESP32 برای نخستین بار راه اندازی می شود یا اطلاعات شبکه مورد نیاز را در اختیار ندارد، می تواند در حالت Access Point یک شبکه بی سیم ایجاد کند. کاربر با اتصال تلفن همراه یا رایانه به این شبکه، به صفحه تنظیمات دستگاه دسترسی پیدا کرده و اطلاعات مورد نیاز، مانند مشخصات شبکه WiFi و سایر تنظیمات ارتباطی، را وارد می کند.

۹-۲- پروتکل MQTT

²MQTT یا Message Queuing Telemetry Transport یک پروتکل پیام رسانی سبک و کم حجم است که بر اساس معماری Client/Server و الگوی ارتباطی Publish/Subscribe طراحی شده است. این پروتکل به طور ویژه برای سامانه های اینترنت اشیا، تجهیزات دارای منابع پردازشی و حافظه محدود و شبکه هایی با پهنای باند محدود مورد استفاده قرار می گیرد. استاندارد MQTT نسخه 5.0 توسط سازمان OASIS منتشر شده است [4]. یکی از مهم ترین ویژگی های MQTT، سربار کم و سادگی ساختار پیام ها است. این ویژگی باعث می شود این پروتکل برای ارتباط میان میکروکنترلر ها، حسگرها و سایر تجهیزات هوشمند مناسب باشد. همچنین MQTT با استفاده از معماری Publish/Subscribe، فرستنده و گیرنده پیام را از یکدیگر جدا می کند و مدیریت ارتباط میان آنها را به یک مولفه مرکزی به نام Broker واگذار می کند.

۱-۹-۲- MQTT Broker³

Broker هسته مرکزی ارتباط در معماری MQTT است و وظیفه دریافت، مدیریت و ارسال پیام ها را بر عهده دارد. هنگامی که یک Client پیامی را در یک Topic منتشر می کند، Broker پیام را دریافت کرده و آن را برای Client هایی که آن Topic را Subscribe کرده اند ارسال می کند. Client ای که یک پیام را در یک Topic منتشر می کند، Publisher نامیده می شود. در مقابل، Client ای که برای دریافت پیام های مربوط به یک Topic ثبت نام می کند، Subscriber نام دارد. بنابراین، یک Client می تواند همزمان نقش Publisher و Subscriber را داشته باشد. Topic یک مسیر منطقی برای دسته بندی و شناسایی پیام های

¹ صفحه ای وب است که پس از اتصال کاربر به یک شبکه، برای انجام تنظیمات یا ارائه اطلاعات مورد نیاز پیش از دسترسی عادی به شبکه نمایش داده می شود.

² یک پروتکل ارتباطی سبک مبتنی بر الگوی انتشار/اشتراک است که برای تبادل پیام میان تجهیزات و سامانه های متصل، به ویژه در محیط های اینترنت اشیا، استفاده می شود.

³ نرم افزاری که وظیفه دریافت، مدیریت و توزیع پیام های MQTT را میان Client های منتشرکننده و مشترک بر عهده دارد.

MQTT است. هر پیام در یک Topic مشخص منتشر می شود و Subscriber ها با Subscribe کردن به Topic موردنظر، پیام های مربوط به آن را دریافت می کنند. Topic ها معمولاً به صورت سلسله مراتبی تعریف می شوند تا دسته بندی و مدیریت پیام ها ساده تر شود. ساختار کلی Topic ها به صورت زیر است:

$< topic > / < subtopic > / < subtopic >$

¹ Payload بخش اصلی محتوای پیام MQTT است که داده موردنظر را در خود نگهداری می کند. استاندارد MQTT قالب مشخصی برای محتوای Payload تعیین نمی کند و انتخاب قالب داده به طراح سیستم واگذار شده است. بنابراین، Payload می تواند به صورت متن ساده، JSON یا سایر قالب های مناسب برای کاربرد موردنظر ارسال شود [4].

۲-۹-۲- کیفیت سرویس

یکی از قابلیت های مهم MQTT، پشتیبانی از سطوح مختلف Quality of Service (QoS) است. QoS مشخص می کند که پیام ها با چه سطحی از تضمین تحویل میان Client و Broker منتقل شوند. MQTT سه سطح QoS را تعریف می کند [4]:

QoS 0 - حداکثر یک بار (At Most One):

در این سطح، پیام فقط یکبار ارسال می شود و هیچ تضمینی برای تحویل موفق پیام وجود ندارد. در نتیجه، در صورت بروز خطا یا قطع ارتباط، ممکن است پیام به مقصد نرسد. این سطح کمترین سربار ارتباطی را دارد و برای داده هایی که از دست رفتن برخی نمونه های آنها اهمیت زیادی ندارد، مناسب است.

QoS 1 - حداقل یک بار (At Least Once):

در این سطح، تحویل پیام به مقصد تضمین می شود، اما در برخی شرایط ممکن است یک پیام بیش از یک بار دریافت شود. بنابراین، گیرنده باید در صورت نیاز امکان مدیریت پیام های تکراری را داشته باشد.

QoS 2 - دقیقاً یک بار (Exactly Once):

در این سطح، پروتکل سازوکاری برای اطمینان از تحویل پیام به صورت دقیقاً یک بار در نظر می گیرد. این سطح بیشترین میزان اطمینان و در نتیجه سربار و پیچیدگی بیشتری نسبت به دو سطح دیگر دارد.

۲-۹-۳- ارتباط MQTT از طریق WebSocket

مرورگر های وب به صورت معمول برای برقراری ارتباط مستقیم با MQTT Broker از طریق اتصال TCP خام طراحی نشده اند. به همین دلیل، برای ایجاد ارتباط میان یک برنامه تحت وب و MQTT Broker معمولاً از ²WebSocket استفاده می شود. WebSocket یک ارتباط دو طرفه و پلیدار میان Browser و

¹ بخش اصلی و محتوای داده یک پیام است که اطلاعات موردنظر برای انتقال را در خود جای می دهد؛ در MQTT، Payload می تواند شامل داده های متنی، عددی یا ساختار یافته باشد.

² یک پروتکل ارتباطی است که امکان ایجاد یک ارتباط پلیدار و دو سویه میان Client و Server را فراهم می کند و برای تبادل داده های بلادرنگ مناسب است.

Server ایجاد می کند و امکان ارسال و دریافت داده را بدون نیاز به ایجاد یک اتصال جدید برای هر پیام فراهم می سازد.

در این روش، مرورگر از طریق WebSocket به سرویسی متصل می شود که امکان ارتباط MQTT روی WebSocket را فراهم کرده است. پس از برقراری اتصال، برنامه تحت وب می تواند همانند یک Client MQTT در Topic های موردنظر Subscribe کرده یا پیام هایی را در Topic های مشخص منتشر کند.

۱۰-۲- ذخیره سازی اطلاعات در ESP32

ESP32 دارای حافظه Flash برای نگهداری Firmware و داده های دائمی است. از این حافظه می توان برای اطلاعاتی استفاده کرد که باید بعد از restart نیز باقی بماند. ¹ LittleFS یک فایل سیستم مناسب برای حافظه Flash سیستم های نهفته است. در Arduino-ESP32، LittleFS به عنوان یکی از فایل سیستم های قابل استفاده در ESP32 معرفی شده است [10]

¹ یک فایل سیستم سبک برای حافظه فلش است که امکان ذخیره و بازیابی فایل ها و داده های پایدار را در سامانه های مبتنی بر میکروکنترلر فراهم می کند.

فصل ۳:

تحليل و طراحی سیستم

در این فصل، فرایند تحلیل و طراحی سامانه هوشمند پایش محیط و کنترل تجهیزات مبتنی بر اینترنت اشیا ارائه می شود. هدف از طراحی این سامانه، ایجاد یک هاب هوشمند مبتنی بر میکروکنترلر ESP32 است که بتواند ضمن دریافت و پردازش اطلاعات محیطی، امکان ارسال داده ها از طریق شبکه بی سیم، ایجاد هشدار در شرایط مشخص و همچنین دریافت و بازپخش سیگنال های کنترلی مادون قرمز و فرکانس رادیویی را فراهم کند.

سامانه طراحی شده از مجموعه ای از حسگر های محیطی شامل حسگر دما و رطوبت DHT11، حسگر دود و گاز MQ-2، حسگر نور مبتنی بر LDR و حسگر تشخیص حرکت PIR تشکیل شده است. داده های حاصل از این حسگر ها توسط ESP32 دریافت و پردازش شده و در بازه های زمانی مشخص از طریق شبکه WiFi به MQTT Broker ارسال می شوند.

علاوه بر قابلیت پایش محیط، سامانه دارای زیرسیستم ارتباطی RF در دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز و یک زیرسیستم ارتباط مادون قرمز است. این زیرسیستم ها امکان دریافت، ذخیره سازی و بازپخش برخی سیگنال های کنترلی تجهیزات مختلف را فراهم می کنند. به این ترتیب، ESP32 به عنوان یک واسط میان تجهیزات فیزیکی، شبکه و رابط کاربری عمل می کند.

۳-۱- تحلیل نیازمندی های سیستم

با توجه به اهداف پروژه و قابلیت های مورد انتظار از سامانه، نیازمندی های سیستم به دو گروه اصلی نیازمندی های عملکردی و نیازمندی های غیرعملکردی تقسیم می شوند. نیازمندی های عملکردی بیان کننده وظایف و قابلیت هایی هستند که سامانه باید ارائه دهد و نیازمندی های غیرعملکردی نیز ویژگی های کیفی و الزامات مربوط به نحوه عملکرد، توسعه، نگهداری و پایداری سیستم را مشخص می کنند. در طراحی سامانه حاضر، هدف اصلی ایجاد یک دستگاه هوشمند برای پایش شرایط محیطی، تشخیص وضعیت های غیرعادی، ایجاد هشدار و فراهم کردن امکان کنترل تجهیزات الکترونیکی از طریق شبکه است؛ بنابراین، مجموعه نیازمندی های تعریف شده باید علاوه بر پوشش عملکرد های اصلی دستگاه، امکان ارتباط میان بخش سخت افزاری، نرم افزار Firmware، سرور یا MQTT Broker و رابط کاربری تحت وب را نیز فراهم کند.

از نظر نیازمندی های عملکردی، سامانه باید قادر باشد اطلاعات محیطی مختلف را توسط حسگرهای متصل به ESP32 دریافت و پردازش کند؛ به این صورت که دمای محیط و رطوبت نسبی توسط حسگر DHT11، شدت نور توسط LDR، تغییرات ناشی از وجود دود یا برخی گازها توسط MQ-2 و حرکت در محیط توسط حسگر PIR شناسایی و اندازه گیری شوند. اطلاعات دریافت شده از حسگرها باید توسط میکروکنترلر پردازش شده و در صورت عبور پارامتر های مشخص، به ویژه مقدار مربوط به دود یا گاز، از آستانه تعیین شده، سیستم بتواند وضعیت هشدار را فعال کرده و با استفاده از باز هشدار صوتی ایجاد کند. همچنین سامانه باید امکان انتقال اطلاعات حسگرها و وضعیت فعلی دستگاه را از طریق شبکه WiFi فراهم کرده و بتواند با یک MQTT Broker ارتباط برقرار کند؛ به گونه ای که داده های مربوط به حسگرها و وضعیت سیستم در Topic های مشخص منتشر شده و فرمان های دریافتی از طریق MQTT نیز توسط ESP32 دریافت، پردازش و اجرا شوند.

یکی دیگر از نیازمندی های اصلی سامانه، فراهم کردن امکان کنترل تجهیزات الکترونیکی از طریق سیگنال های RF و IR است. در این راستا، سیستم باید قابلیت دریافت سیگنال های RF در دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز و همچنین دریافت سیگنال های مادون قرمز را داشته باشد و بتواند سیگنال های دریافت شده

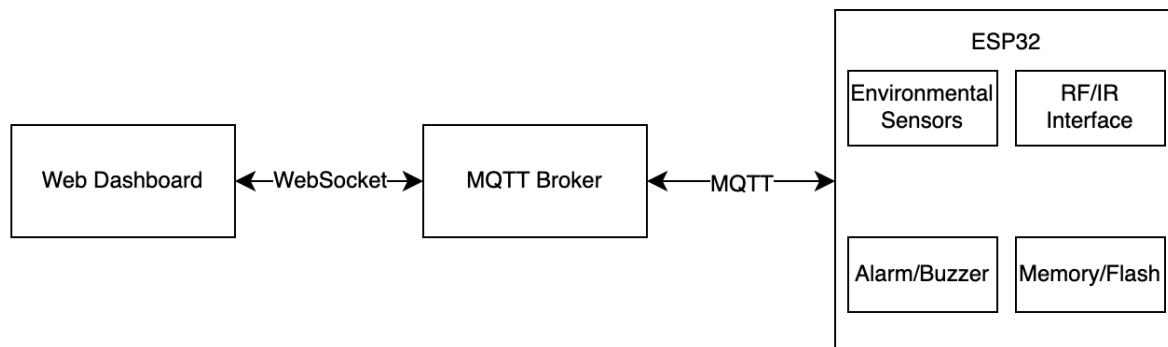
را در حافظه ذخیره کند تا در زمان موردنظر مجدداً از طریق فرستنده مربوطه ارسال شوند. بنابراین، سامانه باید علاوه بر دریافت سیگنال، قابلیت ذخیره سازی و بازپخش سیگنال های RF و IR را نیز داشته باشد و اطلاعات ذخیره شده باید پس از خاموش و روشن شدن دستگاه همچنان قابل دسترسی باشند. همچنین برای جلوگیری از وابستگی تنظیمات دستگاه به کد Firmware، سامانه باید امکان تنظیم اطلاعات شبکه WiFi و مشخصات MQTT را بدون نیاز به تغییر و کامپایل مجدد برنامه فراهم کند؛ به همین منظور، فرآیند پیکربندی اولیه از طریق یک پورتال تنظیمات انجام شده و کاربر می تواند اطلاعات موردنیاز را از طریق رابط ارائه شده وارد یا تغییر دهد.

در کنار عملکردهای محلی، سامانه باید یک رابط کاربری تحت وب نیز در اختیار کاربر قرار دهد تا اطلاعات دریافت شده از حسگرها و وضعیت کلی سیستم در آن نمایش داده شود و کاربر بتواند فرمان های کنترلی موردنظر خود را ارسال کند. ارتباط میان داشبورد تحت وب و بخش ارتباطی سیستم بر پایه MQTT انجام شده و برای فراهم کردن امکان ارتباط MQTT در محیط مرورگر از WebSocket استفاده می شود؛ در نتیجه، اطلاعات حسگرها می توانند در داشبورد نمایش داده شده و فرمان های کاربر نیز از طریق همین مسیر به دستگاه منتقل شوند. این ساختار باعث می شود کاربر بتواند بدون نیاز به دسترسی مستقیم به سخت افزار، وضعیت محیط را مشاهده کرده و تجهیزات متصل به سامانه را کنترل کند.

از نظر نیازمندی های غیرعملکردی، ساختار سیستم باید به گونه ای طراحی شود که امکان توسعه و نگهداری آن در آینده وجود داشته باشد. به همین منظور، Firmware سامانه به صورت ماژولار طراحی شده و بخش هایی مانند مدیریت حسگرها، ارتباط MQTT، ارتباط RF، ارتباط IR، مدیریت هشدار و مدیریت حافظه از یکدیگر تفکیک شده اند تا اضافه کردن قابلیت های جدید یا اصلاح یک بخش، حداقل تأثیر را بر سایر قسمت های سیستم داشته باشد. همچنین اطلاعات مربوط به شبکه و سیگنال های ذخیره شده نباید با خاموش شدن دستگاه از بین بروند؛ بنابراین از سیستم فایل LittleFS برای ذخیره سازی پایدار این اطلاعات استفاده شده است. یکی دیگر از الزامات مهم، پایداری عملکرد محلی سیستم است؛ به این معنا که قطع ارتباط با MQTT Broker یا بروز اختلال در شبکه نباید باعث توقف عملکرد هایی مانند خواندن حسگرها، تشخیص حرکت یا فعال سازی سیستم هشدار شود و بخش های اصلی دستگاه باید بتوانند مستقل از وضعیت ارتباط شبکه به عملکرد خود ادامه دهند.

در نهایت، مجموعه این نیازمندی ها مشخص می کند که سامانه طراحی شده تنها یک دستگاه اندازه گیری ساده نیست، بلکه یک سیستم هوشمند یکپارچه است که وظایف پایش محیط، تشخیص شرایط غیرعادی، ایجاد هشدار، ارتباط شبکه ای، دریافت و بازپخش سیگنال های RF و IR، ذخیره سازی اطلاعات و کنترل تجهیزات از طریق رابط کاربری تحت وب را بر عهده دارد. بنابراین، نیازمندی های تعیین شده به عنوان مبنایی برای طراحی معماری سخت افزار و نرم افزار سیستم مورد استفاده قرار گرفته اند و در مراحل بعدی طراحی و پیاده سازی، عملکرد اجزای مختلف سامانه بر اساس این نیازمندی ها تعیین و ارزیابی خواهد شد.

۳-۲- معماری کلی سیستم



شکل (۳-۱) معماری کلی سیستم

معماری سامانه به گونه ای طراحی شده است که میکروکنترلر ESP32 به عنوان هسته اصلی پردازشی و ارتباطی، در مرکز تعامل میان حسگرها، تجهیزات کنترلی، زیرساخت شبکه و کاربر قرار گیرد. در این معماری، اطلاعات مربوط به شرایط محیطی توسط حسگرهای متصل به ESP32 دریافت شده و پس از پردازش، از طریق شبکه WiFi و پروتکل MQTT به زیرساخت ارتباطی سامانه ارسال می شوند. علاوه بر دریافت و پردازش اطلاعات حسگرها، مسئولیت مدیریت سیستم هشدار، دریافت و ارسال سیگنال های RF و IR، ذخیره سازی اطلاعات و اجرای فرمان های کنترلی را نیز بر عهده دارد. در سمت کاربر، یک داشبورد تحت وب برای نمایش اطلاعات و کنترل سامانه در نظر گرفته شده است. کاربر می تواند از طریق این داشبورد وضعیت فعلی حسگرها و سایر بخش های سیستم را مشاهده کرده و فرمان های کنترلی موردنظر خود را ارسال کند. فرمان های ارسال شده توسط کاربر ابتدا از طریق ارتباط WebSocket به MQTT Broker منتقل شده و سپس در قالب پیام MQTT در اختیار ESP32 قرار می گیرند. ESP32 پس از دریافت پیام، فرمان مربوطه را پردازش کرده و عملیات موردنظر را اجرا می کند. به این ترتیب، ارتباط میان سخت افزار و کاربر به صورت دوطرفه برقرار بوده و هم اطلاعات دستگاه به سمت کاربر ارسال می شوند و هم فرمان های کاربر به دستگاه منتقل می شوند. به صورت کلی، معماری سامانه را می توان از نظر عملکردی به سه بخش اصلی شامل لایه سخت افزار و جمع آوری داده، لایه پردازش و کنترل مبتنی بر ESP32 و لایه ارتباطی و رابط کاربر تقسیم کرد. این تفکیک به منظور مشخص کردن وظایف هر بخش و کاهش وابستگی میان اجزای مختلف سیستم انجام شده است. نقشه شماتیک کامل مدار و نحوه اتصال اجزای سخت افزاری در پیوست ۱-۸ ارائه شده است.

۳-۲-۲- لایه سخت افزار و جمع آوری داده

در لایه سخت افزار و جمع آوری داده، مجموعه ای از حسگرها و ماژول های ارتباطی به ESP32 متصل شده اند که وظیفه آنها دریافت اطلاعات از محیط و برقراری ارتباط با تجهیزات خارجی است. حسگر DHT11 برای اندازه گیری دما و رطوبت نسبی، حسگر MQ-2 برای تشخیص تغییرات ناشی از وجود دود و برخی گازها، حسگر LDR برای پایش شدت نور محیط و حسگر PIR برای تشخیص حرکت مورد استفاده قرار

گرفته اند. خروجی این حسگرها بسته به نوع آنها به صورت دیجیتال یا آنالوگ در اختیار ESP32 قرار گرفته و توسط نرم افزار Firmware پردازش می شوند. علاوه بر حسگرهای محیطی، در این لایه تجهیزات ارتباطی مربوط به کنترل تجهیزات خارجی نیز قرار دارند. سامانه دارای گیرنده و فرستنده RF برای باندهای فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز و همچنین گیرنده و فرستنده مادون قرمز IR است. این تجهیزات امکان دریافت سیگنال های کنترلی، ذخیره سازی اطلاعات مربوط به آنها و ارسال مجدد سیگنال های ذخیره شده را فراهم می کنند. در نتیجه، لایه سخت افزاری علاوه بر جمع آوری اطلاعات محیطی، وظیفه ایجاد واسطه میان ESP32 و تجهیزات خارجی قابل کنترل را نیز بر عهده دارد.

۳-۲-۳- لایه پردازش و کنترل

ESP32 هسته اصلی پردازش و کنترل سامانه است و وظیفه مدیریت بخش عمده ای از عملیات سیستم را بر عهده دارد. داده های دریافت شده از حسگرها در این لایه پردازش شده و وضعیت فعلی محیط تعیین می شود. همچنین منطق مربوط به سیستم هشدار در این بخش اجرا شده و در صورت برقرار شدن شرایط تعریف شده، بازار فعال می شود. مدیریت گیرنده ها و فرستنده های RF و IR، پردازش فرمان های دریافتی از MQTT، مدیریت اطلاعات ذخیره شده و برقراری ارتباط با MQTT Broker نیز در این لایه انجام می شود. Firmware مربوط به ESP32 به صورت ماژولار طراحی شده است تا وظایف مختلف سیستم از یکدیگر تفکیک شوند. برای هر یک از حسگرهای اصلی، از جمله DHT11، MQ-2، LDR و PIR، بخش های نرم افزاری مربوطه در نظر گرفته شده و مدیریت ارتباط RF، ارتباط IR، MQTT، حافظه، هشدار و LED نیز در ماژول های مستقل انجام می شود. این ساختار باعث می شود هر بخش مسئولیت مشخصی داشته باشد و تغییر، اصلاح یا توسعه یک قابلیت، وابستگی کمتری به سایر بخش های نرم افزار ایجاد کند. همچنین این معماری امکان اضافه کردن قابلیت ها و تجهیزات جدید را در مراحل بعدی توسعه ساده تر می کند.

۳-۲-۴- لایه ارتباطی و رابط کاربر

ارتباط میان ESP32 و زیرساخت نرم افزاری سامانه از طریق WiFi برقرار می شود. پس از اتصال ESP32 به شبکه، اطلاعات اندازه گیری شده و وضعیت سیستم در قالب پیام های MQTT به MQTT Broker ارسال می شوند. در این پروژه از broker.emqx.io به عنوان Broker مورد استفاده در ارتباطات MQTT استفاده شده است. ESP32 می تواند اطلاعات حسگرها و وضعیت بخش های مختلف سیستم را در Topic های مشخص منتشر کرده و همچنین پیام ها و فرمان های کنترلی دریافت شده از طریق MQTT را پردازش کند.

داشبورد تحت وب نیز به منظور فراهم کردن امکان پایش و کنترل سیستم برای کاربر طراحی شده است. این داشبورد از طریق WebSocket به MQTT Broker متصل می شود و می تواند پیام های منتشر شده توسط ESP32 را دریافت کرده و اطلاعاتی مانند وضعیت حسگرها، مقادیر اندازه گیری شده و وضعیت هشدار را به کاربر نمایش دهد. از طرف دیگر، فرمان هایی که کاربر از طریق داشبورد ارسال می کند، در قالب پیام MQTT به سمت ESP32 منتقل شده و پس از دریافت توسط دستگاه، توسط بخش مدیریت فرمان

پردازش و اجرا می‌شوند.

۳-۳- طراحی سخت افزار

در طراحی سخت افزاری سامانه، ماژول ESP32-WROOM-32 به عنوان واحد پردازشی و کنترلی اصلی انتخاب شده است. انتخاب این ماژول با توجه به قابلیت پردازشی مناسب، تعداد کافی پایه های ورودی و خروجی، برخورداری از مبدل آنالوگ به دیجیتال (ADC)، حافظه مناسب و همچنین پشتیبانی داخلی از ارتباط WiFi انجام شده است. ESP32 در این سامانه نقش هسته مرکزی را بر عهده دارد و تمامی اجزای سخت افزاری شامل حسگر های محیطی، تجهیزات دریافت و ارسال RF و IR، سیستم هشدار و رابط های ارتباطی به صورت مستقیم یا غیرمستقیم با آن در ارتباط هستند. این ساختار باعث شده است که پردازش اطلاعات، کنترل تجهیزات، ذخیره سازی داده ها و ارتباط با زیرساخت نرم افزاری سیستم در یک واحد مرکزی انجام شود.

۳-۳-۱- واحد پردازش مرکزی و اتصال حسگرها

ماژول ESP32-WROOM-32 وظیفه دریافت اطلاعات حسگرها، پردازش داده های دریافتی، اجرای منطق کنترلی و هشدار، مدیریت تجهیزات RF و IR، ذخیره سازی اطلاعات و برقراری ارتباط با شبکه را بر عهده دارد. قابلیت WiFi داخلی ESP32 برای اتصال سامانه به شبکه محلی و برقراری ارتباط با MQTT Broker مورد استفاده قرار گرفته است و پایه های GPIO نیز برای اتصال تجهیزات دیجیتال و کنترل برخی ماژول ها به کار گرفته شده اند. همچنین برای دریافت خروجی آنالوگ حسگر هایی مانند MQ-2 و LDR از پایه های دارای قابلیت ADC استفاده شده است.

برای اندازه گیری دمای محیط و رطوبت نسبی از حسگر DHT11 استفاده شده است. این حسگر از طریق یک خط داده دیجیتال با ESP32 ارتباط برقرار می کند و اطلاعات اندازه گیری شده را به صورت دیجیتال در اختیار میکروکنترلر قرار می دهد. در پیاده سازی Firmware، برای ارتباط با DHT11 از کتابخانه Adafruit DHT و توابع readTemperature() و readHumidity() استفاده شده است و پایه داده حسگر به GPIO15 متصل شده است. اطلاعات دریافت شده پس از پردازش در Firmware، به صورت جداگانه برای دما و رطوبت مدیریت شده و از طریق MQTT در اختیار سایر بخش های سامانه و داشبورد تحت وب قرار می گیرند.

برای پایش وجود دود و برخی گاز های قابل اشتعال از حسگر MQ-2 استفاده شده است. در این پروژه خروجی آنالوگ این حسگر به GPIO33 متصل شده و مقدار آن توسط ADC داخلی ESP32 اندازه گیری می شود. حسگر MQ-2 برای دستیابی به شرایط کاری مناسب به یک دوره پیش گرمایش نیاز دارد و بنابراین مقدار خروجی آن بلافاصله پس از راه اندازی سیستم به عنوان یک مقدار پایدار در نظر گرفته نمی شود. در طراحی حاضر، فرایند راه اندازی و گرم شدن حسگر مطابق ساختار مدار و Firmware سیستم مدیریت می شود و پس از آماده شدن حسگر، مقدار خروجی آن توسط ADC خوانده می شود. مقدار به دست آمده علاوه بر ارسال از طریق MQTT، با یک آستانه از پیش تعیین شده مقایسه می شود و در صورت عبور مقدار اندازه گیری شده از آستانه، وضعیت هشدار فعال شده و باز به صدا در می آید؛ همچنین وضعیت

هشدار از طریق MQTT به داشبورد ارسال می شود تا کاربر بتواند وضعیت سیستم را مشاهده کند. برای پایش شدت نور محیط نیز از LDR استفاده شده است. از آنجا که خروجی LDR به صورت تغییرات ولتاژ در یک مدار اندازه گیری ایجاد می شود، خروجی آن به یکی از ورودی های ADC متصل شده است. در این پروژه خروجی مربوط به LDR به GPIO32 متصل شده و مقدار آن توسط ADC داخلی ESP32 خوانده می شود. Firmware در بازه های زمانی مشخص مقدار خروجی را نمونه برداری کرده و مقدار حاصل را برای پایش وضعیت روشنایی محیط از طریق MQTT منتشر می کند. به منظور تشخیص حرکت در محیط از یک مازول PIR استفاده شده است که خروجی دیجیتال آن به GPIO13 متصل شده است. هنگامی که حسگر وجود حرکت را در محدوده تحت پوشش خود تشخیص دهد، وضعیت خروجی دیجیتال آن تغییر کرده و ESP32 این تغییر وضعیت را دریافت و پردازش می کند. وضعیت تشخیص حرکت پس از پردازش توسط Firmware از طریق MQTT منتشر می شود و در صورت فعال بودن قابلیت هشدار حرکت، وقوع حرکت می تواند موجب فعال شدن سیستم هشدار صوتی و ارسال وضعیت مربوطه به داشبورد شود.

۲-۳-۳- سیستم ارتباط فرکانس رادیویی

یکی از قابلیت های اصلی سامانه، امکان دریافت، ذخیره سازی و بازپخش سیگنال های فرکانس رادیویی در دو باند ۳۱۵ و ۴۳۳ مگاهرتز است. با توجه به تفاوت فرکانسی این دو گروه، برای هر باند یک گیرنده و یک فرستنده مستقل در نظر گرفته شده است تا ESP32 بتواند دریافت و ارسال سیگنال ها را به صورت جداگانه مدیریت کند. در بخش مربوط به بلند ۳۱۵ مگاهرتز، از مازول گیرنده RXB61 و مازول فرستنده H34S استفاده شده است که خروجی داده گیرنده به GPIO22 و ورودی داده فرستنده به GPIO23 متصل شده اند. برای باند ۴۳۳ مگاهرتز نیز از گیرنده RXB35 و فرستنده H34S استفاده شده است که خروجی گیرنده به GPIO14 و ورودی داده فرستنده به GPIO27 متصل شده اند.

در فرایند دریافت سیگنال، ESP32 اطلاعات دریافتی از گیرنده های RF را دریافت و توسط Firmware پردازش می کند. سیگنال های موردنظر پس از دریافت می توانند با یک نام مشخص در حافظه سیستم ذخیره شوند تا در زمان دیگری امکان استفاده مجدد از آنها وجود داشته باشد. در هنگام ارسال نیز، Firmware بر اساس اطلاعات ذخیره شده و باند فرکانسی مربوط به سیگنال، فرستنده مناسب را انتخاب کرده و داده موردنظر را برای بازپخش به آن ارسال می کند. برای پیاده سازی بخش دریافت و ارسال سیگنال های RF از کتابخانه RCSSwitch استفاده شده است که امکان کار با بسیاری از تجهیزات کنترل از راه دور RF ساده در باندهای موردنظر را فراهم می کند.

۳-۳-۳- سیستم ارتباط مادون قرمز

برای فراهم کردن قابلیت دریافت و بازپخش سیگنال های مادون قرمز، از یک گیرنده IR و مجموعه ای شامل چهار LED مادون قرمز استفاده شده است. گیرنده مادون قرمز به GPIO17 متصل شده و LED های فرستنده مادون قرمز از طریق GPIO4 کنترل می شوند. گیرنده IR سیگنال های ارسال شده توسط کنترل های مادون قرمز را دریافت کرده و اطلاعات مربوط به سیگنال توسط ESP32 پردازش می شود. سیگنال های

دریافت شده می توانند با یک نام مشخص در حافظه سیستم ذخیره شوند تا در زمان دیگری قابل بازیابی و بازپخش باشند. در فرایند ارسال، ESP32 اطلاعات سیگنال ذخیره شده را بازیابی کرده و الگوی مربوطه را از طریق LED های مادون قرمز ارسال می کند؛ به این ترتیب، دستگاه می تواند نقش یک واسط هوشمند برای دریافت و بازپخش فرمان های کنترل از راه دور مادون قرمز را ایفا کند. برای پیاده سازی قابلیت دریافت و ارسال سیگنال های IR از کتابخانه IRremoteESP8266 استفاده شده است.

۴-۳-۳- سیستم هشدار و نمایش وضعیت

برای ایجاد هشدار صوتی در سامانه از یک Active Buzzer استفاده شده است که ورودی آن به GPIO19 متصل شده است. فعال سازی بازر توسط Firmware و بر اساس شرایط تعریف شده در منطق سیستم انجام می شود؛ برای مثال، در صورت عبور مقدار خوانده شده از حسگر MQ-2 از آستانه تعیین شده یا در صورت تشخیص حرکت هنگامی که قابلیت هشدار حرکت فعال باشد، ESP32 می تواند بازر را فعال کرده و هشدار صوتی ایجاد کند. وضعیت هشدار نیز همزمان می تواند از طریق MQTT به داشبورد ارسال شود تا کاربر علاوه بر دریافت هشدار محلی، وضعیت دستگاه را از طریق رابط کاربری نیز مشاهده کند. به منظور ارائه بازخورد بصری از وضعیت عملکرد دستگاه نیز یک LED وضعیت در نظر گرفته شده است. این LED به GPIO25 متصل شده و توسط Firmware برای نمایش وضعیت فعالیت سیستم و همچنین نمایش برخی رویدادهای مهم مانند دریافت یا ارسال سیگنال های RF و IR مورد استفاده قرار می گیرد. استفاده از LED وضعیت باعث می شود کاربر بتواند بدون نیاز به مشاهده داشبورد یا اتصال به شبکه، برخی وضعیت های عملیاتی دستگاه را به صورت مستقیم مشاهده کند.

جدول (۳-۱) پایه های ESP32

ردیف	تجهیز	پایه ESP32	نوع ورودی/خروجی
۱	DHT11	GPIO15	ورودی دیجیتال
۲	MQ-2 Heater	GPIO26	خروجی دیجیتال
۳	MQ-2 ADC	GPIO33	ورودی آنالوگ
۴	LDR ADC	GPIO32	ورودی آنالوگ
۵	PIR	GPIO13	ورودی دیجیتال
۶	RF 315MHz Receiver	GPIO22	ورودی دیجیتال
۷	RF 315MHz Transmitter	GPIO23	خروجی دیجیتال
۸	RF 433MHz Receiver	GPIO14	ورودی دیجیتال
۹	RF 433 MHz Transmitter	GPIO27	خروجی دیجیتال
۱۰	IR Receiver	GPIO17	ورودی دیجیتال
۱۱	IR LED	GPIO4	خروجی دیجیتال
۱۲	Buzzer	GPIO19	خروجی دیجیتال
۱۳	Status LED	GPIO25	خروجی دیجیتال

۴-۳- طراحی نرم افزار و ساختار Firmware

Firmware سامانه بر روی ماژول ESP32-WROOM-32 و با استفاده از محیط توسعه PlatformIO و چارچوب Arduino پیاده سازی شده است. با توجه به تعدد وظایف سیستم و وجود بخش های مختلفی مانند مدیریت حسگرها، ارتباط شبکه، ارتباط MQTT، دریافت و ارسال سیگنال های RF و IR، مدیریت هشدار و ذخیره سازی اطلاعات، ساختار Firmware به صورت ماژولار طراحی شده است. در این ساختار، هر بخش از نرم افزار مسئولیت مشخصی دارد و وظایف سیستم میان ماژول های مختلف تقسیم شده اند؛ به همین دلیل، تغییر یا توسعه یک قابلیت می تواند با کمترین وابستگی به سایر قسمت ها انجام شود و فرایند بررسی، اشکال زدایی و نگهداری Firmware نیز ساده تر خواهد بود.

۴-۳-۱ ساختار ماژولار Firmware

ماژول های مختلف Firmware بر اساس وظایف سیستم از یکدیگر تفکیک شده اند. ماژول main مسئول راه اندازی اولیه و کنترل جریان اصلی برنامه است و ماژول Sensors وظیفه هماهنگی و مدیریت دوره ای حسگرها و رویداد های محیطی را بر عهده دارد. در بخش حسگرها نیز ماژول های اختصاصی DHT، MQ-2، LDR و PIR به ترتیب برای دریافت اطلاعات دما و رطوبت، پایش دود و گاز، اندازه گیری شدت نور و تشخیص حرکت مورد استفاده قرار گرفته اند. ماژول MQTT وظیفه برقراری ارتباط با MQTT Broker و مدیریت ارسال و دریافت پیام ها را بر عهده دارد، در حالی که ماژول های RF و IR برای دریافت، پردازش و ارسال مجدد سیگنال های فرکانس رادیویی و مادون قرمز استفاده می شوند. همچنین ماژول Alarm وضعیت هشدار و عملکرد بازر را مدیریت کرده، ماژول LED مسئول کنترل LED وضعیت است و Provisioning¹ فرایند دریافت و تنظیم اطلاعات WiFi و MQTT را مدیریت می کند. برای ذخیره و بازیابی اطلاعات نیز ماژول Memory با سیستم فایل LittleFS در ارتباط بوده و ماژول Serial برای دریافت، پردازش و نمایش فرمان ها و اطلاعات از طریق ارتباط سریال مورد استفاده قرار گرفته است. در نهایت، پارامتر ها و تنظیمات ثابت مورد نیاز سیستم در ماژول Config نگهداری می شوند.

جدول (۳-۲) وظیفه ماژول های Firmware

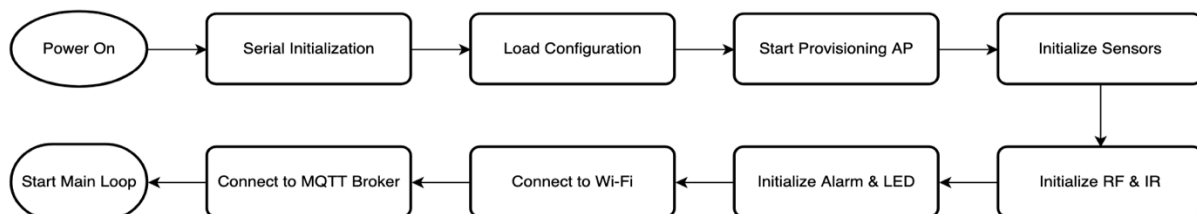
وظیفه	ماژول
راه اندازی و کنترل جریان اصلی برنامه	main
مدیریت دوره ای حسگر ها و رویداد های محیطی	Sensors
دریافت دما و رطوبت	DHT
دریافت مقدار گاز و دود	MQ-2
دریافت مقدار نور	LDR
تشخیص حرکت	PIR

¹ فرایند آماده سازی و پیکربندی اولیه یک دستگاه برای اتصال به شبکه و سرویس های مورد نیاز، از جمله دریافت، اعتبارسنجی و ذخیره تنظیمات ارتباطی است.

برقراری ارتباط با Broker و ارسال و دریافت پیام	MQTT
دریافت و ارسال سیگنال های RF	RF
دریافت و ارسال سیگنال های مادون قرمز	IR
مدیریت وضعیت هشدار و بازر	Alarm
مدیریت LED وضعیت	LED
پیکربندی WiFi و MQTT	Provisioning
ذخیره و بازیابی اطلاعات در LittleFS	Memory
چاپ و دریافت و پردازش فرمان های متنی	Serial
نگهداری تنظیمات و پارامتر های ثابت	Config

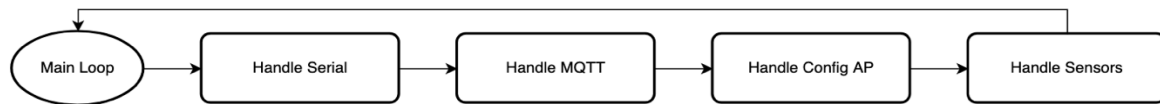
۲-۴-۳- راه اندازی و اجرای اصلی سیستم

پس از روشن شدن ESP32، برنامه وارد تابع `setup()` می شود که وظیفه آماده سازی اولیه اجزای مختلف سامانه را بر عهده دارد. در مرحله نخست، ارتباط سریال برای نمایش اطلاعات راه اندازی و دریافت فرمان های احتمالی برقرار می شود و سپس تنظیمات و پارامتر های مورد نیاز سیستم از حافظه خوانده می شوند. در صورتی که اطلاعات مورد نیاز شبکه WiFi و MQTT موجود نباشند یا نیاز به پیکربندی مجدد وجود داشته باشد، فرآیند Provisioning اجرا شده و پورتال تنظیمات برای دریافت اطلاعات شبکه و مشخصات MQTT در اختیار کاربر قرار می گیرد. پس از تکمیل پیکربندی، حسگرها و ماژول های ارتباطی مورد نیاز راه اندازی شده و سیستم تلاش می کند ابتدا به شبکه WiFi و سپس به MQTT Broker متصل شود. پس از تکمیل موفقیت آمیز این مراحل، سیستم آماده ورود به چرخه اصلی اجرای برنامه خواهد بود.



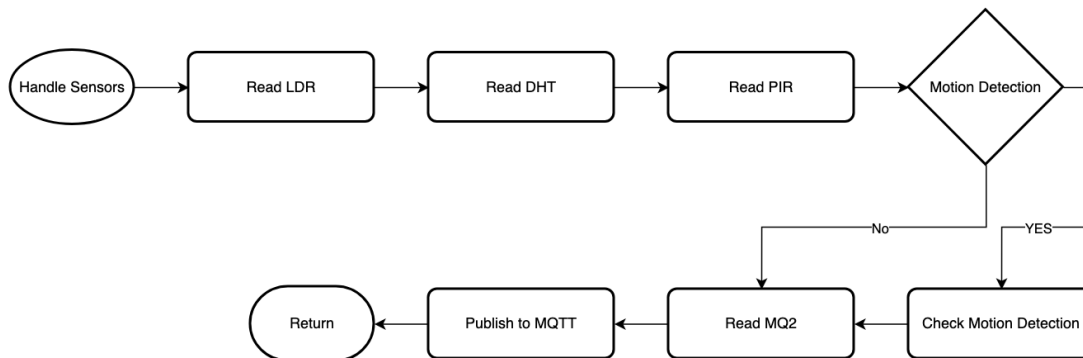
شکل (۳-۲) روند راه اندازی سیستم

پس از پایان عملیات راه اندازی، برنامه وارد تابع `loop()` می شود. این تابع به صورت مداوم اجرا شده و وظایف اصلی سامانه را مدیریت می کند. در هر چرخه، وضعیت ارتباط سریال و MQTT بررسی شده و در صورت دریافت فرمان جدید، پیام مربوطه پردازش می شود؛ سپس ماژول مدیریت حسگرها وضعیت حسگر های مختلف را بررسی کرده و در صورت رسیدن زمان نمونه برداری، مقادیر جدید را دریافت می کند. اطلاعات حاصل از حسگرها پس از پردازش در Topic های مربوط به MQTT منتشر می شوند و رویداد هایی مانند تشخیص حرکت یا عبور مقدار MQ-2 از آستانه تعیین شده نیز در همین فرایند بررسی و مدیریت می شوند. ساختار اجرای برنامه به گونه ای طراحی شده است که عملیات مختلف بدون توقف طولانی در یک بخش انجام شوند و اختلال در ارتباط شبکه، تا حد امکان موجب توقف عملکرد های محلی دستگاه نشود.



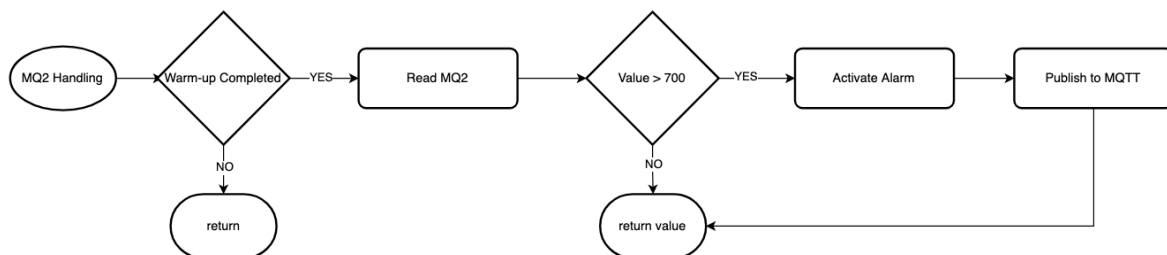
شکل (۳-۳) روند اجرای حلقه اصلی سیستم

۳-۴-۳- مدیریت حسگرها و منطق هشدار



شکل (۳-۴) روند مدیریت حسگرها

مدیریت حسگرهای محیطی توسط ماژول Sensors انجام می شود و این ماژول نقش هماهنگ کننده میان درایورهای مربوط به DHT11، MQ-2، LDR و PIR را بر عهده دارد. اطلاعات حسگرها در بازه های زمانی مشخص دریافت شده و پس از پردازش، مقدار هر حسگر در Topic مربوطه از طریق MQTT منتشر می شود. در نسخه فعلی Firmware، بازه زمانی اصلی برای نمونه برداری و انتشار اطلاعات حسگرها ۱۰ ثانیه در نظر گرفته شده است. این روش باعث می شود داده های محیطی به صورت دوره ای بروزرسانی شوند، در حالی که پردازش مداوم و غیرضروری حسگرها کاهش پیدا می کند.



شکل (۳-۵) روند مدیریت حسگر MQ-۲

در مورد حسگر MQ-2، به دلیل نیاز این حسگر به پیش گرمایش، Firmware پس از راه اندازی دستگاه یک دوره اولیه را برای آماده سازی آن در نظر می گیرد و تا پایان این دوره مقدار حسگر به عنوان داده قابل اتکا برای تصمیم گیری هشدار استفاده نمی شود. پس از پایان زمان پیش گرمایش، خروجی آنالوگ MQ-2 توسط ADC خوانده شده و مقدار به دست آمده علاوه بر انتشار از طریق MQTT، با آستانه تعیین شده در

Firmware مقایسه می شود. در نسخه فعلی پروژه، زمان پیش گرمایش اولیه به دلیل محدودیت زمانی آزمایش ها ۵ دقیقه و مقدار آستانه ۷۰۰ در نظر گرفته شده است. در صورت عبور مقدار خوانده شده از این آستانه، ماژول Alarm فعال شده و وضعیت هشدار علاوه بر فعالسازی بازر، از طریق MQTT نیز به داشبورد ارسال می شود. لازم به ذکر است که این مقادیر با توجه به شرایط آزمایش و هدف نمونه اولیه انتخاب شده اند و مقدار مناسب آستانه MQ-2 در یک محیط واقعی باید بر اساس شرایط محیط، کالیبراسیون حسگر و آزمون های عملی تعیین شود.

دما و رطوبت محیط توسط حسگر DHT11 اندازه گیری می شوند و داده های آن در فواصل زمانی تعیین شده توسط ماژول Sensors دریافت می شوند. برای ارتباط با این حسگر از کتابخانه Adafruit DHT استفاده شده است و Firmware با استفاده از توابع readTemperature() و readHumidity() به ترتیب مقدار دما و رطوبت نسبی را دریافت می کند. مقدار شدت نور نیز توسط LDR و ورودی ADC خوانده شده و مانند سایر داده های محیطی پس از پردازش در اختیار ماژول MQTT قرار می گیرد. حسگر PIR نیز به صورت دیجیتال بررسی شده و تغییر وضعیت خروجی آن به عنوان رویداد تشخیص حرکت در نظر گرفته می شود. قابلیت تشخیص حرکت در Firmware قابل فعال یا غیرفعال شدن است و این کار از طریق فرمان های motion on و motion off انجام می شود. در صورتی که قابلیت تشخیص حرکت فعال باشد و PIR وقوع حرکت را گزارش کند، رویداد مربوطه توسط سیستم پردازش شده و می تواند باعث فعال شدن وضعیت هشدار و ارسال پیام مربوطه به MQTT شود.

مدیریت وضعیت هشدار توسط ماژول Alarm انجام می شود و این ماژول مسئول کنترل Active Buzzer و تعیین وضعیت هشدار سیستم است. در پیاده سازی فعلی دو رویداد اصلی می توانند موجب فعال شدن هشدار شوند که شامل عبور مقدار MQ-2 از آستانه تعیین شده و تشخیص حرکت در صورت فعال بودن قابلیت تشخیص حرکت است. علاوه بر هشدار های خودکار، امکان کنترل مستقیم بازر نیز از طریق فرمان های buzzer on و buzzer off فراهم شده است. هنگامی که وضعیت هشدار فعال می شود، بازر بر اساس منطق زمانی تعریف شده توسط Firmware کنترل می شود. در نسخه فعلی چرخه زمانی فعال و غیرفعال شدن آن ۲ ثانیه در نظر گرفته شده است. این ساختار باعث می شود سیستم بتواند هم در واکنش به رویدادهای محیطی و هم در پاسخ به فرمان مستقیم کاربر، هشدار صوتی ایجاد کند.

۵-۳- طراحی ارتباط WiFi و MQTT

ارتباط شبکه ای سامانه بر پایه قابلیت WiFi داخلی ESP32 طراحی شده است و این ارتباط، مسیر اصلی تبادل اطلاعات میان دستگاه، MQTT Broker و داشبورد تحت وب را فراهم می کند. ESP32 پس از راه اندازی و دریافت تنظیمات شبکه، به شبکه WiFi موردنظر متصل شده و از طریق این ارتباط امکان ارسال اطلاعات حسگرها، وضعیت هشدار و سایر داده های سامانه به زیرساخت نرم افزاری را فراهم می کند. همچنین همین مسیر برای دریافت فرمان های کنترلی از سمت کاربر مورد استفاده قرار می گیرد؛ بنابراین ارتباط شبکه ای سیستم به صورت دوطرفه طراحی شده است.

۱-۵-۳- پیکربندی و ارتباط WiFi

در سامانه حاضر از حالت WIFI_AP_STA استفاده شده است. در این حالت، ESP32 می تواند به صورت همزمان در نقش Station به شبکه WiFi موردنظر متصل شده و در کنار آن یک Access Point اختصاصی نیز ایجاد کند. این قابلیت در فرآیند Provisioning مورد استفاده قرار می گیرد و به دستگاه اجازه می دهد بدون نیاز به تغییر یا کامپایل مجدد Firmware، اطلاعات مربوط به شبکه WiFi و سایر تنظیمات موردنیاز را از کاربر دریافت کند. پس از انجام پیکربندی و برقراری اتصال به شبکه اصلی، ESP32 از ارتباط WiFi به عنوان مسیر ارتباطی خود با MQTT Broker استفاده می کند.

استفاده از این ساختار باعث شده است که فرآیند راه اندازی اولیه دستگاه از تنظیمات ثابت Firmware جدا شود و کاربر بتواند اطلاعات شبکه را در زمان راه اندازی یا در صورت نیاز در مراحل بعدی تغییر دهد. در نتیجه، دستگاه برای استفاده در شبکه های مختلف نیازی به تغییر کد برنامه نخواهد داشت و اطلاعات پیکربندی شده نیز می توانند در حافظه داخلی دستگاه ذخیره شده و در راه اندازی های بعدی مورد استفاده قرار گیرند.

۲-۵-۳- ارتباط MQTT

برای تبادل اطلاعات میان ESP32، MQTT Broker و داشبورد تحت وب از پروتکل MQTT استفاده شده است. این پروتکل بر اساس مدل Publish/Subscribe عمل می کند و باعث می شود فرستنده و گیرنده پیام ها مستقیماً به یکدیگر وابسته نباشند. در این معماری، ESP32 اطلاعات حسگرها و وضعیت سیستم را در Topic های مشخص منتشر می کند و داشبورد یا سایر Client های متصل به Broker می توانند Topic های موردنظر را Subscribe کرده و پیام های مربوطه را دریافت کنند. از طرف دیگر، داشبورد می تواند فرمان های کنترلی را در Topic های مشخص منتشر کند و ESP32 با Subscribe کردن به این Topic ها، پیام های دریافت شده را پردازش و اجرا کند.

در این پروژه از broker.emqx.io به عنوان MQTT Broker استفاده شده است. برای جلوگیری از پراکندگی Topic ها و ایجاد یک ساختار منظم، یک Topic پایه با مقدار homeintelligence در نظر گرفته شده است و نام دستگاه نیز توسط کاربر تعیین می شود. بر این اساس، Topic های مختلف سامانه از یک ساختار مشترک پیروی می کنند و نوع داده یا عملکرد هر Topic از بخش انتهایی مسیر آن مشخص می شود. به صورت کلی می توان ساختار Topic را به صورت زیر در نظر گرفت:

homeintelligence / < device > / < subtopic >

برای اطلاعات حسگرها، هر نوع داده در یک Topic مجزا منتشر می شود تا دریافت و پردازش آن برای داشبورد و سایر Client ها ساده باشد. ساختار Topic های مربوط به حسگرهای اصلی سامانه در جدول زیر ارائه شده است

جدول (۳-۳) Topic های مربوط به حسگرها

Topic	داده
.../sensor/TEMP	دما

رطوبت	.../sensor/HUM
شدت نور	.../sensor/LDR
تشخیص حرکت	.../sensor/PIR
دود و گاز	.../sensor/MQ2

در کنار اطلاعات حسگرها، برای اطلاع رسانی رویداد های مهم سیستم از Topic مربوط به هشدار با ساختار `.../alarm` استفاده شده است. این Topic برای ارسال پیام هایی که نشان دهنده وقوع یا تغییر وضعیت یک رویداد هشدار هستند مورد استفاده قرار می گیرد. برای مثال، در صورت عبور مقدار MQ-2 از آستانه تعیین شده، پیام `MQ2 threshold exceed` منتشر می شود و در صورت تشخیص حرکت نیز پیام `Motion Detected` به داشبورد ارسال می شود. همچنین تغییر وضعیت قابلیت تشخیص حرکت با پیام های `Motion ON` و `Motion OFF` اطلاع رسانی می شود.

برای دریافت فرمان های کنترلی از سمت داشبورد نیز Topic با ساختار `.../command` در نظر گرفته شده است. Firmware این Topic را `Subscribe` کرده و در صورت دریافت پیام جدید، آن را به بخش مدیریت فرمان منتقل می کند. این بخش بر اساس محتوای پیام، عملیات متناظر را شناسایی و اجرا می کند. به این ترتیب، قابلیت هایی مانند فعال یا غیرفعال کردن تشخیص حرکت، کنترل بازر و سایر عملیات کنترلی می توانند از طریق یک مسیر ارتباطی مشخص مدیریت شوند.

علاوه بر داده های حسگر و فرمان های کنترلی، سامانه باید فهرست سیگنال های ذخیره شده RF و IR را نیز در اختیار داشبورد قرار دهد. برای این منظور از Topic های `.../RF/list` و `.../IR/list` استفاده شده است. پیام های منتشر شده در این Topic ها در قالب JSON ارسال می شوند و شامل اطلاعات مربوط به سیگنال های ذخیره شده هستند. داشبورد با دریافت این اطلاعات می تواند فهرست سیگنال های موجود را به کاربر نمایش داده و امکان انتخاب سیگنال موردنظر برای انجام عملیات بعدی، مانند ارسال مجدد یا مدیریت اطلاعات ذخیره شده، را فراهم کند.

۳-۶- طراحی سیستم RF

یکی از قابلیت های اصلی سامانه، امکان دریافت، ذخیره سازی و بازپخش سیگنال های رادیویی در دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز است. با توجه به اینکه گیرنده ها و فرستنده های مورد استفاده در هر باند به صورت تک فرکانس عمل می کنند، برای هر یک از این دو باند یک مسیر مستقل برای دریافت و ارسال سیگنال در نظر گرفته شده است. به این ترتیب، سامانه می تواند سیگنال های مربوط به هر دو باند را دریافت کرده و پس از شناسایی و ذخیره اطلاعات آنها، در زمان موردنیاز سیگنال را از طریق فرستنده متناسب با باند فرکانسی آن بازپخش کند. مدیریت ارتباط با ماژول های RF و دریافت و ارسال سیگنال ها در Firmware توسط کتابخانه `RCSwitch` انجام شده است.

۳-۶-۱- دریافت و ذخیره سازی سیگنال RF

فرایند دریافت سیگنال RF از طریق فرمان `rf rx <name>` آغاز می شود که در آن `<name>` نامی است که

کاربر برای سیگنال موردنظر تعیین می کند. پس از اجرای این فرمان، Firmware هر دو مسیر دریافت RF مربوط به باندهای ۳۱۵ و ۴۳۳ مگاهرتز را فعال کرده و وضعیت گیرنده ها را بررسی می کند. هنگامی که یک سیگنال توسط یکی از گیرنده ها دریافت شود، اطلاعات مربوط به سیگنال توسط کتابخانه RCSSwitch استخراج شده و باند فرکانسی مربوط به گیرنده ای که سیگنال را دریافت کرده است نیز مشخص می شود. این اطلاعات سپس در قالب ساختار مورد استفاده Firmware قرار گرفته و با نام تعیین شده توسط کاربر در حافظه دائمی دستگاه ذخیره می شود.

اطلاعات سیگنال های RF در فایل rf_signals.json در سیستم فایل LittleFS نگهداری می شوند. استفاده از حافظه دائمی باعث می شود اطلاعات سیگنال های ذخیره شده با خاموش یا راه اندازی مجدد ESP32 از بین نروند و دستگاه بتواند پس از راه اندازی مجدد، اطلاعات قبلی را از حافظه بازبازی کند. همچنین ذخیره نام سیگنال در کنار اطلاعات مربوط به باند فرکانسی باعث می شود در هنگام بازپخش، سیستم بتواند بدون دریافت مجدد اطلاعات از کاربر، فرستنده مناسب را انتخاب کند.

۲-۶-۳- بازپخش و مدیریت سیگنال RF

برای بازپخش یک سیگنال ذخیره شده از فرمان rf tx <name> استفاده می شود. پس از دریافت این فرمان، Firmware نام سیگنال موردنظر را در اطلاعات ذخیره شده جست و جو کرده و پس از بازبازی اطلاعات آن، باند فرکانسی مربوط به سیگنال را بررسی می کند. سپس بر اساس باند ثبت شده، فرستنده مناسب انتخاب شده و اطلاعات سیگنال از طریق آن ارسال می شود. در نتیجه، کاربر هنگام بازپخش سیگنال نیازی به مشخص کردن فرکانس ندارد و انتخاب مسیر ارسال به صورت خودکار توسط Firmware انجام می شود. برای مدیریت سیگنال های ذخیره شده نیز فرمان های rf list و rf del <name> در نظر گرفته شده اند. فرمان rf list فهرست سیگنال های موجود در حافظه را نمایش داده و اطلاعات این فهرست را از طریق MQTT نیز در اختیار داشبورد قرار می دهد تا کاربر بتواند سیگنال های ذخیره شده را مشاهده و انتخاب کند. فرمان rf del <name> نیز برای حذف یک سیگنال مشخص از حافظه استفاده می شود.

۷-۳- طراحی سیستم مادون قرمز

سیستم مادون قرمز سامانه از دو بخش اصلی گیرنده و فرستنده تشکیل شده است و وظیفه آن دریافت، پردازش، ذخیره سازی و بازپخش فرمان های مادون قرمز مورد استفاده در تجهیزات مختلف است. در این بخش، گیرنده مادون قرمز سیگنال ارسال شده توسط کنترل های از راه دور را دریافت کرده و اطلاعات آن توسط ESP32 پردازش می شود؛ سپس سیگنال دریافت شده با یک نام مشخص در حافظه دائمی دستگاه ذخیره شده و در زمان موردنیاز می توان آن را بازبازی و از طریق LED های مادون قرمز بازپخش کرد. برای پیاده سازی فرایند دریافت و ارسال سیگنال های IR از کتابخانه IRremoteESP8266 استفاده شده است که امکان کار با تعدادی از پروتکل های رایج کنترل از راه دور را فراهم می کند.

۱-۷-۳- دریافت و ذخیره سازی سیگنال IR

فرایند دریافت سیگنال مادون قرمز با اجرای فرمان `ir rx <name>` آغاز می شود که در آن `<name>` نامی است که کاربر برای سیگنال موردنظر تعیین می کند. پس از اجرای فرمان، Firmware وارد حالت دریافت شده و منتظر دریافت یک سیگنال مادون قرمز می ماند. با دریافت سیگنال، اطلاعات مربوط به آن توسط گیرنده دریافت شده و کتابخانه IRremoteESP8266 داده های مربوط به پروتکل و محتوای سیگنال را استخراج و برای پردازش در اختیار Firmware قرار می دهد. سامانه قادر به شناسایی و پردازش تعدادی از پروتکل های رایج مانند NEC، SONY، SAMSUNG و PANASONIC است و در صورتی که پروتکل دریافت شده توسط کتابخانه شناسایی یا پشتیبانی نشود، اطلاعات خام سیگنال نیز قابل دریافت و ذخیره سازی خواهد بود.

پس از دریافت سیگنال، اطلاعات آن با نام تعیین شده توسط کاربر در حافظه دائمی دستگاه و در قالب JSON ذخیره می شود. اطلاعات مربوط به سیگنال های IR در فایل `ir_signals.json` در سیستم فایل LittleFS نگهداری می شوند. ذخیره سازی در حافظه دائمی باعث می شود سیگنال های ثبت شده پس از خاموش و روشن شدن دستگاه نیز حفظ شوند و در راه اندازی های بعدی امکان بازیابی و استفاده مجدد از آنها وجود داشته باشد.

۲-۷-۳- بازپخش و مدیریت سیگنال IR

برای بازپخش یک سیگنال ذخیره شده از فرمان `ir tx <name>` استفاده می شود. پس از دریافت این فرمان، Firmware اطلاعات سیگنال موردنظر را از حافظه بازیابی کرده و بر اساس اطلاعات ذخیره شده، روش مناسب برای ارسال آن را انتخاب می کند. در صورتی که پروتکل سیگنال توسط کتابخانه پشتیبانی شود، داده های مربوط به آن با استفاده از پروتکل شناسایی شده تولید و از طریق LED های مادون قرمز ارسال می شوند؛ در غیر این صورت، اطلاعات خام ذخیره شده مورد استفاده قرار گرفته و سیگنال به صورت خام بازپخش می شود. به این ترتیب، سامانه علاوه بر پشتیبانی از پروتکل های شناخته شده، امکان ذخیره و بازپخش سیگنال هایی را که پروتکل آنها قابل شناسایی نیست نیز فراهم می کند.

برای مدیریت سیگنال های ذخیره شده نیز دو فرمان `ir list` و `ir del <name>` در نظر گرفته شده است. فرمان `ir list` فهرست سیگنال های موجود در حافظه را نمایش داده و اطلاعات مربوط به این فهرست را از طریق MQTT نیز در اختیار داشبورد قرار می دهد تا کاربر بتواند سیگنال های ثبت شده را مشاهده و انتخاب کند. فرمان `ir del <name>` نیز برای حذف یک سیگنال مشخص از حافظه استفاده می شود.

۸-۳- طراحی سیستم ذخیره سازی

برای ذخیره سازی اطلاعاتی که باید پس از خاموش و روشن شدن دستگاه نیز حفظ شوند، از سیستم فایل LittleFS موجود در ESP32 استفاده شده است. استفاده از این سیستم فایل امکان ذخیره سازی اطلاعات به صورت دائمی و بدون نیاز به حافظه خارجی را فراهم می کند و باعث می شود تنظیمات دستگاه و اطلاعات سیگنال های ثبت شده پس از راه اندازی مجدد نیز قابل بازیابی باشند. در این پروژه اطلاعات دائمی سامانه

در قالب فایل های JSON سازمان دهی شده است تا ساختار داده ها ساده، قابل خواندن و قابل پردازش توسط Firmware باشد. در مجموع، چهار فایل اصلی برای نگهداری اطلاعات دائمی سامانه در نظر گرفته شده است که هر یک وظیفه مشخصی بر عهده دارند. فایل provisioning.json برای نگهداری اطلاعات مربوط به شبکه WiFi و MQTT، فایل settings.json برای تنظیمات داخلی دستگاه، فایل rf_signals.json برای نگهداری سیگنال های RF و فایل ir_signals.json برای نگهداری سیگنال های مادون قرمز استفاده می شود.

۳-۸-۱- ذخیره تنظیمات و اطلاعات دستگاه

اطلاعات مربوط به پیکربندی شبکه در فایل provisioning.json و به صورت JSON ذخیره می شوند. این اطلاعات شامل SSID و رمز عبور شبکه WiFi، آدرس و پورت MQTT Broker، نام کاربری و رمز عبور MQTT در صورت استفاده و نام اختصاصی دستگاه است. نگهداری این اطلاعات در حافظه دائمی باعث می شود دستگاه پس از راه اندازی مجدد بتواند تنظیمات قبلی را بازیابی کرده و بدون نیاز به وارد کردن مجدد اطلاعات شبکه، فرایند اتصال را انجام دهد.

تنظیمات مربوط به وضعیت داخلی دستگاه نیز در فایل settings.json ذخیره می شوند. در نسخه فعلی پروژه، یکی از مهم ترین اطلاعات این فایل وضعیت فعال یا غیرفعال بودن قابلیت تشخیص حرکت است. به این ترتیب، تنظیماتی که باید پس از خاموش و روشن شدن دستگاه حفظ شوند، از اطلاعات مربوط به پیکربندی شبکه و اطلاعات سیگنال ها جدا نگهداری شده و هر گروه از داده ها فایل مخصوص به خود را دارد.

۳-۸-۲- ذخیره سازی سیگنال RF و IR

اطلاعات مربوط به سیگنال های RF در فایل rf_signals.json ذخیره می شوند. برای هر سیگنال، علاوه بر نام تعیین شده توسط کاربر، اطلاعات مورد نیاز برای بازپخش آن نیز در حافظه نگهداری می شود. این اطلاعات شامل کد سیگنال، تعداد بیت، پروتکل، طول پالس و باند فرکانسی است. ذخیره باند فرکانسی در کنار سایر اطلاعات سیگنال اهمیت دارد، زیرا سیستم هنگام بازپخش می تواند بر اساس این مقدار، فرستنده مناسب مربوط به باند ۳۱۵ یا ۴۳۳ مگاهرتز را به صورت خودکار انتخاب کند.

سیگنال های مادون قرمز نیز در فایل ir_signals.json ذخیره می شوند. اطلاعات اصلی هر سیگنال شامل نام سیگنال، پروتکل، کد سیگنال و تعداد بیت است و در مواردی که پروتکل سیگنال قابل شناسایی یا پشتیبانی نباشد، اطلاعات خام سیگنال نیز در ساختار ذخیره سازی نگهداری می شود تا امکان بازپخش آن در آینده وجود داشته باشد. این روش باعث شده است که سیستم تنها به پروتکل های شناخته شده وابسته نباشد و بتواند سیگنال های خام را نیز برای استفاده مجدد ذخیره کند.

۳-۹- طراحی سیستم Captive Portal و Provisioning

یکی از قابلیت های نرم افزاری سامانه، امکان پیکربندی اطلاعات مربوط به شبکه WiFi و MQTT بدون نیاز به تغییر یا کامپایل مجدد Firmware است. این قابلیت با استفاده از مکانیزم Provisioning پیاده سازی شده است و به کاربر اجازه می دهد در زمان راه اندازی دستگاه، اطلاعات مورد نیاز برای اتصال به شبکه و MQTT Broker را از طریق یک رابط تحت وب وارد و ذخیره کند. برای اجرای این فرآیند، ESP32 در حالت WIFI_AP_STA قرار می گیرد؛ به این ترتیب دستگاه علاوه بر تلاش برای اتصال به شبکه WiFi تنظیم شده، یک Access Point اختصاصی نیز ایجاد می کند که برای دسترسی کاربر به بخش پیکربندی مورد استفاده قرار می گیرد.

جدول (۳-۴) مشخصات AP

مقدار	مشخصه
Intelligence_Setup	SSID
12345678	Password
192.168.4.1	IP

پس از اتصال کاربر به Access Point دستگاه، یک Captive Portal در اختیار وی قرار می گیرد که به عنوان رابط پیکربندی سامانه عمل می کند. این صفحه امکان وارد کردن اطلاعات مورد نیاز برای اتصال ESP32 به شبکه WiFi و MQTT Broker را فراهم می کند و کاربر می تواند اطلاعاتی مانند نام شبکه، رمز عبور WiFi، آدرس و پورت MQTT Broker و در صورت نیاز اطلاعات احراز هویت MQTT را در آن وارد کند. پس از ارسال و تأیید اطلاعات، Firmware داده های دریافت شده را در قالب JSON در فایل provisioning.json موجود در سیستم فایل LittleFS ذخیره کرده و سپس دستگاه را راه اندازی مجدد می کند.

۳-۱۰- طراحی Web Dashboard

داشبورد تحت وب سامانه با هدف نمایش اطلاعات محیطی، مشاهده وضعیت عملکرد دستگاه و ارسال فرمان های کنترلی به ESP32 طراحی شده است. این داشبورد به صورت Serverless پیاده سازی شده و برای تبادل اطلاعات مستقیماً از طریق WebSocket به MQTT Broker متصل می شود؛ بنابراین برای ارتباط میان رابط کاربر و ESP32 نیازی به ایجاد یک سرور واسط اختصاصی وجود ندارد. از آنجا که مرورگر های وب به صورت مستقیم از ارتباط MQTT بر بستر TCP پشتیبانی نمی کنند، MQTT Broker مورد استفاده باید قابلیت WebSocket را فعال کرده باشد تا داشبورد بتواند از طریق این پروتکل به Broker متصل شده و پیام های MQTT را دریافت و ارسال کند.

پس از برقراری اتصال، داشبورد Topic های مربوط به اطلاعات حسگرها را Subscribe کرده و داده های منتشر شده توسط ESP32 را به صورت لحظه ای دریافت می کند. اطلاعات مربوط به دما، رطوبت، شدت نور، وضعیت تشخیص حرکت و مقدار مربوط به دود و گاز در بخش های مربوطه نمایش داده می شوند و با دریافت هر پیام جدید، مقدار نمایش داده شده در رابط کاربر بروزرسانی می شود؛ بنابراین برای مشاهده تغییرات نیازی به Refresh کردن صفحه وجود ندارد. علاوه بر اطلاعات حسگرها، داشبورد پیام های مربوط

به وضعیت هشدار را نیز دریافت کرده و در صورت وقوع رویداد هایی مانند عبور مقدار MQ-2 از آستانه تعیین شده یا تشخیص حرکت در حالت فعال، وضعیت هشدار را به کاربر نمایش می دهد. داشبورد علاوه بر نمایش اطلاعات، امکان ارسال فرمان های کنترلی به ESP32 را نیز فراهم می کند. این فرمان ها از طریق MQTT و در Topic مربوط به ./command... ارسال شده و پس از دریافت توسط ESP32، توسط بخش مدیریت فرمان پردازش و اجرا می شوند. فرمان های buzzer on و buzzer off برای کنترل بزر و فرمان های motion on و motion off برای فعال یا غیرفعال کردن قابلیت تشخیص حرکت مورد استفاده قرار می گیرند. همچنین فرمان های مربوط به مدیریت سیگنال های RF و فرمان های مربوط به سیستم مادون قرمز که امکان دریافت، بازپخش، حذف و مشاهده فهرست سیگنال های ذخیره شده را از طریق رابط کاربر فراهم می کند.

فصل ۴:

پیاده سازی سامانه

در این فصل، فرایند پیاده سازی سامانه تشریح می شود. پس از بررسی مبانی نظری و طراحی سامانه در فصل های پیشین، در این فصل نحوه تبدیل طراحی ارائه شده به یک سامانه عملیاتی مورد بررسی قرار می گیرد. پیاده سازی انجام شده شامل دو بخش اصلی Firmware دستگاه و رابط کاربری تحت وب است. Firmware بر روی برد ESP32 و با استفاده از چارچوب Arduino توسعه داده شده و وظیفه مدیریت حسگرها، پردازش داده های دریافتی، کنترل سیستم هشدار، دریافت و ارسال سیگنال های RF و IR، مدیریت ارتباط WiFi و MQTT و همچنین ذخیره سازی اطلاعات را بر عهده دارد. در بخش سخت افزاری، ESP32 به مجموعه ای از حسگرها و تجهیزات جانبی شامل حسگر دما و رطوبت DHT11، حسگر گاز و دود MQ-2، حسگر نور مبتنی بر LDR، حسگر تشخیص حرکت PIR، گیرنده و فرستنده RF در باند های ۳۱۵ و ۴۳۳ مگاهرتز، گیرنده و فرستنده مادون قرمز و بازو متصل شده است. اطلاعات دریافت شده از حسگرها توسط ESP32 پردازش شده و در فواصل زمانی مشخص فرستاده می شود. در بخش ارتباطی دستگاه با استفاده از WiFi به شبکه متصل شده و برای تبادل داده و دریافت فرمان های کنترلی از پروتکل MQTT استفاده می کند. MQTT Broker مورد استفاده در پیاده سازی نهایی پروژه، سرویس عمومی broker.emqx.io است. همچنین برای پیکربندی اولیه و تغییر تنظیمات شبکه، زیرسامانه مبتنی بر Access Point و Captive Portal در Firmware پیاده سازی شده است. یکی دیگر از قابلیت های اصلی سامانه، امکان دریافت، ذخیره و بازپخش سیگنال های RF و IR است. این قابلیت دستگاه را از یک سامانه صرفاً نظارتی به یک واسط ارتباطی میان تجهیزات الکترونیکی و زیرساخت هوشمند تبدیل می کند. در نهایت، یک Web Dashboard برای مشاهده داده های حسگرها، دریافت هشدارها و ارسال فرمان های کنترلی طراحی شده است. این داشبورد مستقیماً با MQTT Broker ارتباط برقرار کرده و امکان تعامل کاربر با دستگاه را بدون نیاز به یک Backend اختصاصی فراهم می کند.

۴-۱- محیط و ابزار های توسعه

توسعه و پیاده سازی Firmware سامانه بر روی ماژول ESP32-WROOM-32 با استفاده از محیط توسعه PlatformIO و چارچوب نرم افزاری Arduino انجام شده است. تنظیمات مربوط به محیط توسعه، مشخصات سخت افزاری برد، Framework مورد استفاده، سرعت ارتباط سریال، سیستم فایل و کتابخانه های مورد نیاز پروژه در فایل platformio.ini تعریف شده اند. این فایل به عنوان فایل اصلی پیکربندی پروژه در PlatformIO، محیط مورد نیاز برای ساخت و بارگذاری Firmware را مشخص می کند و باعث می شود وابستگی های نرم افزاری و تنظیمات مربوط به برد به صورت یکپارچه در پروژه تعریف شوند. در این پروژه از پلتفرم espressif32 برای توسعه نرم افزار مربوط به خانواده ESP32 استفاده شده و Framework انتخاب شده نیز Arduino است. پارامتر platform پلتفرم مورد استفاده را مشخص می کند و پارامتر board نیز تنظیمات مربوط به برد را تعیین می کند که در پروژه حاضر با توجه به استفاده از برد عمومی ESP32 DevKit، مقدار esp32dev برای آن در نظر گرفته شده است.

برای مشاهده پیام های خروجی Firmware و بررسی وضعیت عملکرد دستگاه، ارتباط سریال با سرعت 115200 baud تنظیم شده است. این ارتباط در مراحل توسعه، آزمایش و خطایابی سامانه نقش مهمی دارد و امکان مشاهده اطلاعات مربوط به راه اندازی اولیه دستگاه، وضعیت حسگرها، فرآیند اتصال به شبکه WiFi و MQTT Broker، دریافت و اجرای فرمان ها و همچنین پیام های مربوط به عملکرد بخش های مختلف Firmware را فراهم می کند. استفاده از Serial Monitor باعث می شود در زمان توسعه بتوان وضعیت داخلی

سیستم را بدون نیاز به ابزار های خارجی بررسی کرد و در صورت بروز خطا در بخش هایی مانند اتصال شبکه، دریافت سیگنال های RF و IR یا اجرای فرمان ها، اطلاعات لازم برای تشخیص مشکل در اختیار توسعه دهنده قرار گیرد.

در بخش ذخیره سازی اطلاعات دائمی نیز از سیستم فایل LittleFS استفاده شده است که امکان استفاده از حافظه داخلی ESP32 برای نگهداری داده هایی را فراهم می کند که باید پس از خاموش و روشن شدن دستگاه همچنان باقی بمانند. در این پروژه از LittleFS برای ذخیره اطلاعات مربوط به تنظیمات Provisioning، تنظیمات داخلی دستگاه و همچنین سیگنال های ذخیره شده RF و IR استفاده شده است. فایل های مربوط به این اطلاعات در زمان راه اندازی توسط Firmware بررسی و در صورت وجود، اطلاعات مورد نیاز از آنها بازیابی می شود؛ بنابراین تغییر وضعیت تغذیه یا راه اندازی مجدد دستگاه موجب از بین رفتن اطلاعات ذخیره شده نخواهد شد.

پس از آماده سازی محیط توسعه و نصب کتابخانه ها و وابستگی های تعریف شده در فایل platformio.ini، فرایند ساخت Firmware توسط PlatformIO انجام می شود. در این مرحله کد های منبع پروژه به همراه کتابخانه های مورد استفاده کامپایل شده و فایل اجرایی مناسب برای ESP32 تولید می شود و پس از موفقیت آمیز بودن فرایند ساخت، Firmware از طریق رابط ارتباطی مناسب بر روی برد بارگذاری می شود. پس از بارگذاری نیز می توان با استفاده از Serial Monitor پیام های خروجی دستگاه را مشاهده و عملکرد بخش های مختلف سیستم را بررسی کرد. در نتیجه، محیط توسعه پروژه علاوه بر فراهم کردن ابزار های مورد نیاز برای برنامه نویسی و ساخت Firmware، امکانات لازم برای مدیریت وابستگی ها، بارگذاری برنامه، ذخیره سازی اطلاعات و بررسی عملکرد سامانه در مرحله توسعه و آزمایش را نیز در اختیار قرار می دهد.



شکل (۴-۱) روند ساخت و بارگذاری Firmware

۴-۲- پیاده سازی سخت افزار

در پیاده سازی سخت افزاری سامانه، هر یک از حسگرها و تجهیزات جانبی به یکی از پایه های GPIO ماژول ESP32 متصل شده و ارتباط میان سخت افزار و Firmware بر اساس این نگاشت انجام می شود. به منظور جلوگیری از پراکندگی شماره پایه ها در بخش های مختلف برنامه و همچنین ساده تر شدن فرایند تغییر اتصالات در صورت نیاز، تمامی پایه های مورد استفاده در فایل پیکربندی config.h تعریف شده اند؛ بنابراین هر ماژول نرم افزاری به جای استفاده مستقیم از شماره پایه، از ثابت تعریف شده مربوط به آن تجهیز استفاده می کند و در نتیجه تغییر یک اتصال سخت افزاری تنها با اصلاح مقدار مربوطه در فایل پیکربندی امکان پذیر خواهد بود. نگاشت کامل پایه های مورد استفاده در جدول (۳-۱) ارائه شده است

۴-۲-۱ پیاده سازی حسگر های محیطی

برای اندازه گیری دما و رطوبت محیط از حسگر DHT11 و کتابخانه Adafruit DHT استفاده شده است. در

زمان راه اندازی Firmware، تابع `setupDHT()` فراخوانی شده و با اجرای `dht.begin()` ارتباط نرم افزار با حسگر آماده می شود. پس از راه اندازی، اطلاعات دما و رطوبت توسط توابع جداگانه دریافت می شوند و در صورت نامعتبر بودن مقدار خوانده شده، پیام خطا در Serial Monitor نمایش داده می شود.

```
void setupDHT()
{
    dht.begin();
}

float readHumidity()
{
    float input = dht.readHumidity();
    if (isnan(input))
    {
        Serial.println("Failed to read humidity from DHT sensor");
    }
    return input;
}

float readTemperature()
{
    float input = dht.readTemperature();
    if (isnan(input))
    {
        Serial.println("Failed to read temperature from DHT sensor");
    }
    return input;
}
```

در این پیاده سازی، تابع `setupDHT()` تنها یک بار در مرحله راه اندازی اجرا می شود، در حالی که توابع `readHumidity()` و `readTemperature()` در زمان نمونه برداری حسگر فراخوانی شده و مقدار رطوبت و دمای اندازه گیری شده را بر می گردانند. استفاده از `isnan()` نیز باعث می شود در صورت عدم دریافت داده معتبر از حسگر، وضعیت خطا مشخص شده و از پردازش بی اطلاع یک مقدار نامعتبر جلوگیری شود.

برای اندازه گیری شدت نور محیط از ماژول LDR استفاده شده است که خروجی آن به ورودی آنالوگ ESP32 متصل شده و مقدار آن توسط ADC خوانده می شود. از آنجا که مقدار خروجی LDR به صورت ولتاژ آنالوگ است، برای تبدیل این مقدار به داده قابل پردازش توسط Firmware از تابع `analogRead()` استفاده شده است.

```
void setupLDR()
{
    pinMode(LDR_PIN, INPUT);
}

int readLDR()
{
    int input = analogRead(LDR_PIN);
    return input;
}
```

تابع `readLDR()` با فراخوانی `analogRead()` مقدار ADC مربوط به پایه `LDR_PIN` را دریافت کرده و مقدار خام تبدیل شده را بر می گرداند. این مقدار در ادامه توسط ماژول مدیریت حسگرها مورد استفاده قرار گرفته و

از طریق MQTT برای نمایش در داشبورد ارسال می شود. برای تشخیص دود و برخی گازهای قابل اشتعال از حسگر MQ-2 استفاده شده است. خروجی آنالوگ حسگر به ورودی ADC متصل شده و مقدار آن به صورت یک مقدار دیجیتال توسط ESP32 خوانده می شود. از آنجا که مقدار خام حسگر به تنهایی نشان دهنده وقوع شرایط هشدار نیست، پس از دریافت مقدار ADC، مقدار آن با آستانه از پیش تعیین شده مقایسه می شود.

```
void setupMQ2()
{
    pinMode(MQ2_PIN, INPUT);
}

int readMQ2()
{
    return analogRead(MQ2_PIN);
}

bool checkThreshold(int value)
{
    if (value > MQ2_THRESHOLD)
    {
        return true;
    }
    return false;
}
```

در تابع readMQ2() مقدار خروجی آنالوگ حسگر توسط ADC خوانده می شود و در تابع checkThreshold() مقدار دریافت شده با ثابت MQ2_THRESHOLD مقایسه می شود. در صورتی که مقدار اندازه گیری شده از آستانه تعیین شده بیشتر باشد، مقدار true بازگردانده شده و سیستم می تواند این وضعیت را به عنوان شرایط هشدار در نظر بگیرد. منطق مربوط به فعال کردن هشدار و ارسال وضعیت آن از طریق MQTT در ماژول مدیریت حسگرها و سیستم هشدار انجام می شود.

برای تشخیص حرکت از ماژول PIR استفاده شده است که خروجی دیجیتال آن به پایه مشخص شده در ESP32 متصل شده است. در زمان راه اندازی، پایه مربوط به PIR به عنوان ورودی تنظیم شده و وضعیت فعال بودن قابلیت تشخیص حرکت نیز از حافظه دائمی بازیابی می شود تا تنظیم انتخاب شده توسط کاربر پس از راه اندازی مجدد دستگاه حفظ شود.

```
void setupPIR()
{
    pinMode(PIR_PIN, INPUT);

    motionDetectionEnabled = loadMotionDetection();
    if (motionDetectionEnabled)
    {
        Serial.println("Motion detection restored to ON");
    }
}

int readPIR()
{
    int input = digitalRead(PIR_PIN);
}
```

```

if (input == HIGH)
{
    return 1;
}
return 0;
}

```

پس از راه اندازی، وضعیت خروجی PIR توسط تابع readPIR() بررسی می شود. در این تابع، مقدار HIGH به عنوان تشخیص حرکت و مقدار LOW به عنوان عدم تشخیص حرکت در نظر گرفته شده است. به منظور مدیریت وضعیت قابلیت تشخیص حرکت، دو متغیر motionDetectionEnabled و motionDetectionTriggered در Firmware تعریف شده اند که متغیر نخست وضعیت فعال یا غیرفعال بودن قابلیت تشخیص حرکت را مشخص کرده و متغیر دوم وضعیت تحریک شدن سیستم در نتیجه تشخیص حرکت را نگهداری می کند. فعال و غیرفعال کردن این قابلیت نیز از طریق توابع مربوطه انجام شده و علاوه بر تغییر وضعیت داخلی، تنظیم جدید در حافظه ذخیره می شود تا در راه اندازی بعدی نیز قابل بازیابی باشد. به صورت مشابه، برای غیرفعال کردن قابلیت تشخیص حرکت، وضعیت داخلی تغییر کرده و مقدار جدید در حافظه ذخیره می شود. در نتیجه، وضعیت تشخیص حرکت تنها در صورتی می تواند موجب ایجاد هشدار شود که مقدار motionDetectionEnabled فعال باشد و در صورت شناسایی حرکت، وضعیت سیستم توسط ماژول مدیریت حسگرها پردازش و در صورت نیاز از طریق MQTT به داشبورد اعلام خواهد شد. جزئیات پیاده سازی این بخش در فایل src/PIR/pir.cpp قرار دارد.

۴-۲-۲- پیاده سازی سیستم هشدار صوتی

برای ایجاد هشدار صوتی از یک Active Buzzer استفاده شده است که با منطق Active-Low کنترل می شود؛ به این معنی که قرار گرفتن پایه خروجی در سطح منطقی LOW باعث فعال شدن بازو و قرار گرفتن آن در سطح HIGH موجب غیرفعال شدن آن می شود. در زمان راه اندازی Firmware، پایه بازو به عنوان خروجی تنظیم شده و برای جلوگیری از فعال شدن ناخواسته بازو، وضعیت آن در حالت خاموش قرار می گیرد.

```

void setupBuzzer()
{
    pinMode(BUZZER_PIN, OUTPUT);
    stopAlarm();
}

void startAlarm()
{
    alarmActive = true;
    buzzerOn = true;
    lastBuzzerToggle = millis();
    digitalWrite(BUZZER_PIN, LOW);
}

void stopAlarm()
{
    alarmActive = false;
}

```

```
buzzerOn = false;
digitalWrite(BUZZER_PIN, HIGH);
}
```

تابع `startAlarm()` وظیفه فعال کردن وضعیت هشدار را بر عهده دارد و با توجه به Active-Low بودن بازر، پایه مربوطه را در سطح LOW قرار می دهد، در حالی که تابع `stopAlarm()` با قرار دادن پایه در سطح HIGH بازر را غیرفعال می کند. از آنجا که هشدار صوتی سامانه به صورت پیوسته فعال نمی شود و برای ایجاد الگوی هشدار، بازر باید در بازه های زمانی مشخص روشن و خاموش شود، تابع `updateAlarm()` در حلقه اصلی Firmware فراخوانی می شود و بر اساس وضعیت زمانی سیستم، خروجی بازر را تغییر می دهد. در نتیجه، در صورت فعال بودن هشدار، بازر مطابق چرخه زمانی تعریف شده فعال و غیرفعال شده و در صورت پایان شرایط هشدار، به حالت خاموش باز می گردد. جزئیات کامل پیاده سازی سیستم هشدار صوتی در فایل `src/Alarm/alarm.cpp` قرار دارد.

۴-۳- پیاده سازی سیستم ارتباط رادیویی RF

یکی از قابلیت های اصلی سامانه، امکان دریافت، ذخیره سازی و بازپخش سیگنال های رادیویی RF است که با هدف فراهم کردن امکان کنترل تجهیزات الکتریکی مجهز به کنترل از راه دور در این پروژه پیاده سازی شده است. در طراحی انجام شده، سیستم RF از دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز پشتیبانی می کند و برای هر یک از این باندها یک مسیر مستقل برای دریافت و ارسال سیگنال در نظر گرفته شده است. از آنجا که گیرنده ها و فرستنده های مورد استفاده به صورت تک فرکانس عمل می کنند، برای مدیریت هر باند یک نمونه مستقل از کلاس `RCSwitch` در Firmware ایجاد شده است و بدین ترتیب سیستم می تواند سیگنال های دو باند را به صورت مستقل دریافت و بازپخش کند. مدیریت ارتباط با ماژول های RF نیز با استفاده از کتابخانه `RCSwitch` انجام شده است که امکان دریافت و ارسال کدهای رایج تجهیزات RF را در اختیار Firmware قرار می دهد.

فرایند راه اندازی سیستم RF در تابع `setupRF()` انجام می شود و در این مرحله گیرنده و فرستنده مربوط به هر دو بلند فعال می شوند. در نتیجه، پس از اجرای این تابع، هر دو گیرنده برای دریافت سیگنال و هر دو فرستنده برای ارسال سیگنال آماده خواهند بود. پایه های مورد استفاده برای این تجهیزات نیز در فایل پیکربندی پروژه تعریف شده اند تا نگاشت پایه ها از منطق اصلی برنامه جدا باشد و در صورت تغییر مدار سخت افزاری، نیاز به تغییر بخش های مختلف Firmware وجود نداشته باشد.

```
void setupRF()
{
    rfSwitch433.enableReceive(RF_RX_433_PIN);
    rfSwitch433.enableTransmit(RF_TX_433_PIN);
    rfSwitch315.enableReceive(RF_RX_315_PIN);
    rfSwitch315.enableTransmit(RF_TX_315_PIN);
}
```

برای نگهداری اطلاعات سیگنال های دریافت شده، ساختاری با نام `RFSignal` تعریف شده است که پارامترهای اصلی موردنیاز برای شناسایی، ذخیره و بازپخش سیگنال را در خود نگهداری می کند. در این ساختار، متغیر `name` نام اختصاص داده شده به سیگنال، `code` کد دریافت شده، `bits` تعداد بیت های سیگنال، `protocol` شماره پروتکل تشخیص داده شده، `pulse` طول پالس و `band` باند فرکانسی سیگنال را مشخص می کند. اطلاعات سیگنال های دریافت شده در آرایه `signals` نگهداری می شوند و تعداد عناصر معتبر این

آرایه نیز توسط متغیر signalCount مشخص می شود. استفاده از این ساختار باعث شده است اطلاعات مربوط به هر سیگنال به صورت یک مجموعه منسجم نگهداری شود و عملیات جست و جو، حذف، ذخیره سازی و بازپخش آن ساده تر انجام گیرد.

```
RFSignal signals[MAX_SIGNALS];
size_t signalCount = 0;

struct RFSignal
{
    String name;
    uint32_t code;
    uint16_t bits;
    uint16_t protocol;
    uint16_t pulse;
    uint16_t band;
};
```

۴-۳-۱- دریافت و ذخیره سازی سیگنال RF

فرایند دریافت سیگنال توسط تابع readRFSignal() انجام می شود. این تابع، وضعیت گیرنده های ۳۱۵ و ۴۳۳ مگاهرتز را بررسی کرده و در صورت دریافت سیگنال، گیرنده ای که سیگنال را دریافت کرده است شناسایی می کند. پس از شناسایی گیرنده فعال، پارامتر های سیگنال از طریق API کتابخانه RCSSwitch استخراج شده و در صورت معتبر بودن داده، اطلاعات مربوط به سیگنال در اولین موقعیت آزاد آرایه signals قرار می گیرد. پس از پلایان پردازش نیز گیرنده به وضعیت مناسب بازگردانده می شود تا امکان دریافت سیگنال بعدی فراهم باشد.

مدیریت کلی فرایند دریافت و ذخیره سازی توسط تابع handleRFReceive() انجام می شود؛ به این صورت که ابتدا وضعیت سیستم از نظر فعال بودن حالت دریافت بررسی شده و پس از دریافت موفق سیگنال، حالت دریافت غیرفعال شده و اطلاعات به دست آمده برای ذخیره سازی دائمی در اختیار بخش مدیریت حافظه قرار می گیرد. بنابراین، دریافت سیگنال و ذخیره سازی آن به صورت دو مرحله مجزا انجام شده است که این موضوع باعث تفکیک مناسب مسئولیت ها در ساختار Firmware می شود. جزئیات کامل پیاده سازی این بخش در فایل های src/RF/general.cpp و src/RF/receiver.cpp قابل مشاهده است.

۴-۳-۲- بازپخش و مدیریت سیگنال RF

برای بازپخش سیگنال های ذخیره شده، تابع transmitRFSignal() مورد استفاده قرار می گیرد. در این فرایند ابتدا وجود سیگنال موردنظر و معتبر بودن پارامتر های ذخیره شده بررسی شده و سپس بر اساس مقدار band، فرستنده مناسب انتخاب می شود. پس از انتخاب فرستنده، پروتکل و طول پالس ذخیره شده به آن اختصاص داده شده و در نهایت کد و تعداد بیت های سیگنال برای ارسال در اختیار کتابخانه RCSSwitch قرار می گیرد. به این ترتیب، کاربر هنگام ارسال مجدد یک سیگنال تنها کافی است نام آن را مشخص کند و انتخاب فرستنده و پارامتر های مورد نیاز توسط Firmware انجام می شود.

به منظور دسترسی به سیگنال های ذخیره شده، تابع `findRFSignalByName()` برای جست و جوی سیگنال بر اساس نام آن پیاده سازی شده است. این تابع عناصر معتبر آرایه `signals` را از ابتدا تا انتها بررسی می کند و در صورت پیدا شدن سیگنالی با نام موردنظر، مکان آن را بر می گرداند؛ در صورتی که سیگنال موردنظر در حافظه وجود نداشته باشد، مقدار `nullptr` بازگردانده می شود.

```
RFSignal *findRFSignalByName(const String &name)
{
    for (size_t i = 0; i < signalCount; i++)
    {
        if (signals[i].name == name)
        {
            return &signals[i];
        }
    }
    return nullptr;
}
```

برای حذف یک سیگنال نیز ابتدا با استفاده از همین تابع وجود سیگنال بررسی می شود و پس از پیدا شدن آن، موقعیت سیگنال در آرایه مشخص می شود. سپس عناصر قرارگرفته پس از آن یک موقعیت به سمت چپ منتقل می شوند تا فضای خالی ایجاد شده حذف شود و مقدار `signalCount` نیز یک واحد کاهش پیدا می کند. در ادامه، آرایه جدید در حافظه دائمی ذخیره می شود.

```
RFSignal *signal = findRFSignalByName(name);
if (signal == nullptr)
{
    Serial.println("RF signal not found");
    return;
}

size_t index = signal - signals;
for (size_t i = index; i + 1 < signalCount; i++)
{
    signals[i] = signals[i + 1];
}
signalCount--;

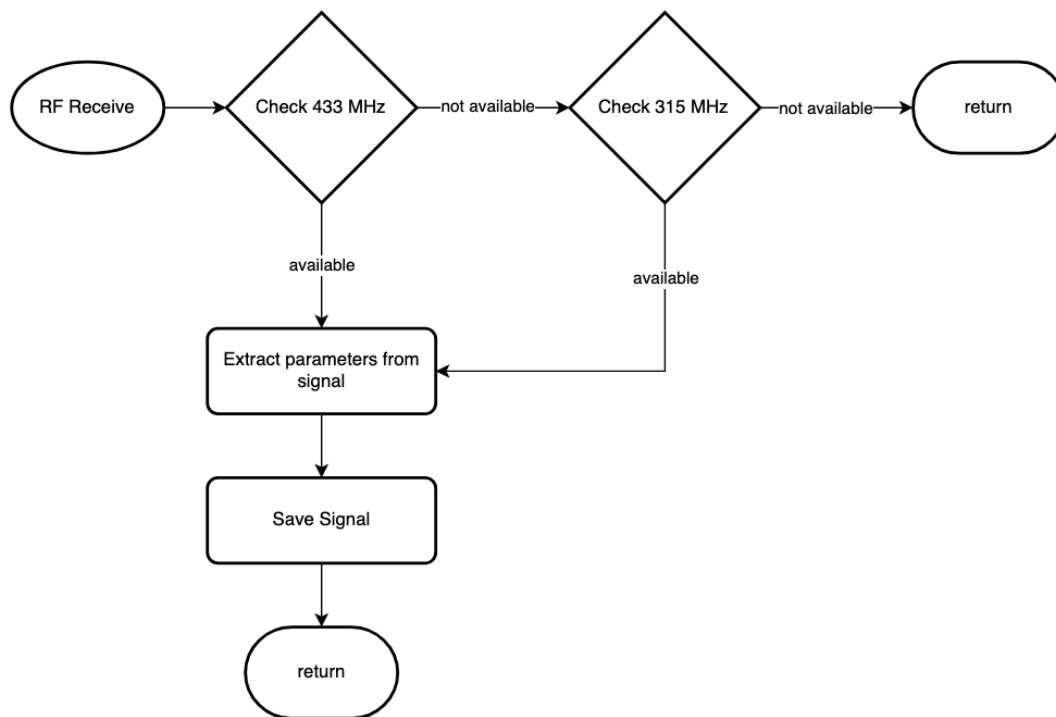
if (saveRFSignals())
{
    Serial.print("Deleted RF signal: ");
    Serial.println(name);
}
else
{
    loadRFSignals();
    Serial.println("Failed to delete RF signal");
}
```

همچنین برای نمایش فهرست سیگنال های ذخیره شده، تابع `listRFSignals()` پیاده سازی شده است که اطلاعات سیگنال ها را شامل نام، باند فرکانسی، کد، تعداد بیت و پروتکل استخراج کرده و علاوه بر نمایش در Serial Monitor، اطلاعات را در قالب JSON از طریق MQTT منتشر می کند. اطلاعات این بخش از طریق Topic مربوط به RF/list منتشر می شود.

جزئیات کامل پیاده سازی این بخش در فایل `src/RF/transmitter.cpp` و `src/RF/general.cpp` قابل مشاهده است.

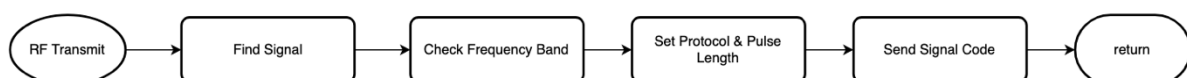
۴-۳-۳- منطق کلی دریافت و ارسال RF

فرایند دریافت سیگنال در سامانه به صورت زیر است:



شکل (۴-۲) فرایند دریافت سیگنال RF

فرایند ارسال سیگنال در سامانه به صورت زیر است:



شکل (۴-۳) فرایند ارسال سیگنال RF

۴-۴- پیاده سازی سیستم مادون قرمز IR

یکی دیگر از قابلیت های اصلی سامانه، امکان دریافت، ذخیره سازی و بازپخش فرمان های مادون قرمز است که با هدف فراهم کردن امکان کنترل تجهیزاتی مانند تلویزیون، سیستم های تهویه، پخش کننده ها و سایر تجهیزات خانگی مجهز به کنترل از راه دور مادون قرمز در پروژه پیاده سازی شده است. در این بخش، یک گیرنده مادون قرمز برای دریافت سیگنال های ارسالی از کنترل ها و چهار LED مادون قرمز برای بازپخش

سیگنال های ذخیره شده مورد استفاده قرار گرفته است. پردازش، تشخیص پروتکل و ارسال سیگنال های IR توسط کتابخانه IRremoteESP8266 انجام می شود و ساختار نرم افزاری سیستم به گونه ای طراحی شده است که علاوه بر سیگنال های قابل Decode، امکان ذخیره و بازپخش سیگنال های ناشناخته به صورت داده خام نیز وجود داشته باشد.

برای مدیریت بخش دریافت و ارسال، یک شی از کلاس IRrecv و یک شی از کلاس IRsend ایجاد شده است که به ترتیب مسئول دریافت و Decode کردن سیگنال های مادون قرمز و آماده سازی و ارسال مجدد آنها هستند. راه اندازی این دو بخش در تابع setupIR() انجام می شود. با اجرای این تابع، گیرنده IR برای دریافت سیگنال آماده شده و بخش ارسال نیز راه اندازی می شود تا در ادامه Firmware بتواند بر اساس فرمان های دریافت شده، عملیات دریافت یا ارسال سیگنال را انجام دهد.

```
IRrecv irReceiver(IR_RX_PIN);
IRsend irTransmitter(IR_TX_PIN);

void setupIR()
{
    irReceiver.enableIRIn();
    irTransmitter.begin();
}
```

برای نگهداری اطلاعات سیگنال های دریافت شده، ساختاری با نام IRSignal تعریف شده است که علاوه بر اطلاعات Decode شده، داده خام سیگنال را نیز در خود نگهداری می کند. در این ساختار، متغیر name نام اختصاص داده شده به سیگنال، protocol نوع پروتکل تشخیص داده شده، value مقدار Decode شده، bits تعداد بیت های سیگنال، rawLength تعداد نمونه های داده خام و rawData آرایه مربوط به داده خام سیگنال را نگهداری می کند. استفاده همزمان از اطلاعات Decode شده و داده خام اهمیت دارد، زیرا همه سیگنال های مادون قرمز الزاماً توسط کتابخانه به یک پروتکل شناخته شده قابل تشخیص نیستند و در چنین شرایطی برای بازپخش صحیح سیگنال باید اطلاعات خام آن حفظ شود. سیگنال های دریافت شده در آرایه irSignals نگهداری شده و تعداد سیگنال های معتبر نیز توسط متغیر irSignalCount مشخص می شود.

برای ذخیره اطلاعات سیگنال های مادون قرمز، ساختار IRSignal تعریف شده است:

```
IRSignal irSignals[MAX_SIGNALS];
size_t irSignalCount = 0;

struct IRSignal
{
    String name;
    decode_type_t protocol;
    uint64_t value;
    uint16_t bits;
    uint16_t rawLength;
    uint16_t rawData[MAX_IR_RAW_LENGTH];
};
```

۴-۴-۱- دریافت و ذخیره سازی سیگنال IR

فرایند دریافت و Decode سیگنال مادون قرمز توسط تابع readIRSignal() انجام می شود. در این فرایند

ابتدا یک ساختار decode_results برای نگهداری نتیجه دریافت و Decode ایجاد شده و گیرنده IR بررسی می کند که آیا سیگنال جدیدی دریافت شده است یا خیر. پس از دریافت سیگنال، پارامترهای مربوط به وضعیت دریافت از جمله overflow و طول داده خام بررسی می شوند تا از معتبر بودن اطلاعات اطمینان حاصل شود. در صورت معتبر بودن داده، یک موقعیت آزاد در آرایه irSignals اختصاص داده شده و اطلاعات مربوط به پروتکل، مقدار Decode شده، تعداد بیت ها و داده خام سیگنال در ساختار مربوطه ذخیره می شوند. یکی از نکات مهم در این پیاده سازی، ذخیره سازی داده خام در کنار اطلاعات Decode شده است؛ زیرا ممکن است پروتکل یک کنترل از راه دور توسط کتابخانه IRremoteESP8266 شناسایی نشود، اما شکل موج دریافت شده همچنان قابل بازپخش باشد. بنابراین، نگهداری آرایه rawData باعث می شود سامانه محدود به پروتکل های شناخته شده نباشد و بتواند در صورت ناشناخته بودن پروتکل نیز سیگنال دریافت شده را در قالب خام ذخیره کند.

مدیریت کلی فرایند دریافت توسط تابع handleIRReceive() انجام می شود. این تابع ابتدا بررسی می کند که سیستم در حالت دریافت IR قرار داشته باشد و در صورت دریافت موفق سیگنال، حالت دریافت را غیرفعال کرده و اطلاعات به دست آمده را برای ذخیره سازی دائمی در اختیار بخش مدیریت حافظه قرار می دهد. در نتیجه، دریافت یک سیگنال پس از اجرای فرمان مربوطه تنها تا زمان دریافت اولین سیگنال معتبر ادامه پیدا می کند و پس از ذخیره اطلاعات، سیستم به حالت عادی اجرای Firmware باز می گردد. جزئیات پیاده سازی این بخش در فایل های src/IR/receiver.cpp و src/IR/general.cpp قابل مشاهده است.

۲-۴-۴- بازپخش و مدیریت سیگنال IR

برای بازپخش سیگنال های ذخیره شده، تابع transmitIRSignal() مورد استفاده قرار می گیرد. در ابتدای این فرایند وجود سیگنال موردنظر و معتبر بودن اطلاعات آن بررسی شده و سپس بر اساس مقدار protocol، روش مناسب برای ارسال انتخاب می شود. در صورتی که پروتکل سیگنال توسط Firmware پشتیبانی شود، اطلاعات Decode شده با استفاده از روش ارسال مربوط به همان پروتکل منتقل می شود و در صورتی که پروتکل ناشناخته باشد یا امکان ارسال آن به صورت Decode شده وجود نداشته باشد، داده خام ذخیره شده در rawData با فرکانس حامل ۳۸ کیلوهرتز بازپخش می شود.

در نسخه فعلی Firmware، پروتکل های رایج NEC، SONY، SAMSUNG و PANASONIC پشتیبانی می شوند و برای سایر سیگنال ها، در صورت وجود داده خام معتبر، روش ارسال خام مورد استفاده قرار می گیرد. این روش باعث می شود قابلیت بازپخش سامانه تنها به پروتکل های شناخته شده محدود نباشد و امکان ذخیره و استفاده مجدد از برخی سیگنال هایی که توسط کتابخانه قابل Decode نیستند نیز فراهم شود.

برای پیدا کردن سیگنال بر اساس نام، تابع findIRSignalByName() پیاده سازی شده است. این تابع عناصر معتبر آرایه irSignals را از ابتدا تا انتها بررسی می کند و در صورت پیدا شدن سیگنالی با نام موردنظر، مکان آن را بر می گرداند؛ در صورتی که سیگنال موردنظر در حافظه وجود نداشته باشد، مقدار nullptr بازگردانده می شود.

```
IRSignal *findIRSignalByName(const String &name)
{
    for (size_t i = 0; i < irSignalCount; i++)
```

```

{
    if (irSignals[i].name == name)
    {
        return &irSignals[i];
    }
}
return nullptr;
}

```

برای حذف یک سیگنال نیز با استفاده از همین تابع وجود سیگنال بررسی می شود و پس از پیدا شدن آن، موقعیت سیگنال در آرایه مشخص می شود. سپس عناصر قرارگرفته پس از آن یک موقعیت به سمت چپ منتقل می شوند تا فضای خالی ایجاد شده حذف شود و مقدار irSignalCount نیز یک واحد کاهش پیدا می کند. در ادامه، آرایه جدید در حافظه دائمی ذخیره می شود.

```

IRSignal *signal = findIRSignalByName(name);
if (signal == nullptr)
{
    Serial.println("IR signal not found");
    return;
}

size_t index = signal - irSignals;
for (size_t i = index; i + 1 < irSignalCount; i++)
{
    irSignals[i] = irSignals[i + 1];
}
irSignalCount--;

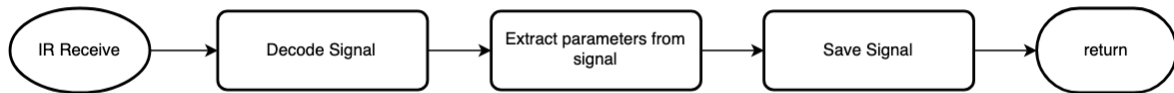
if (saveIRSignals())
{
    Serial.print("Deleted IR signal: ");
    Serial.println(name);
}
else
{
    loadIRSignals();
    Serial.println("Failed to delete IR signal");
}

```

همچنین برای نمایش فهرست سیگنال های ذخیره شده، تابع listIRSignals() در نظر گرفته شده است که اطلاعات هر سیگنال شامل نام، پروتکل، تعداد بیت و طول داده خام را استخراج کرده و علاوه بر نمایش در Serial Monitor، آنها را در قالب یک شی JSON آماده کرده و از طریق MQTT منتشر می کند. اطلاعات فهرست سیگنال ها از طریق Topic مربوط به IR/list در اختیار داشبورد قرار می گیرد و در نتیجه کاربر می تواند بدون دسترسی مستقیم به حافظه دستگاه، سیگنال های ذخیره شده را مشاهده و مدیریت کند. جزئیات پیاده سازی در فایل src/IR/transmitter.cpp و src/IR/general.cpp قابل مشاهده است.

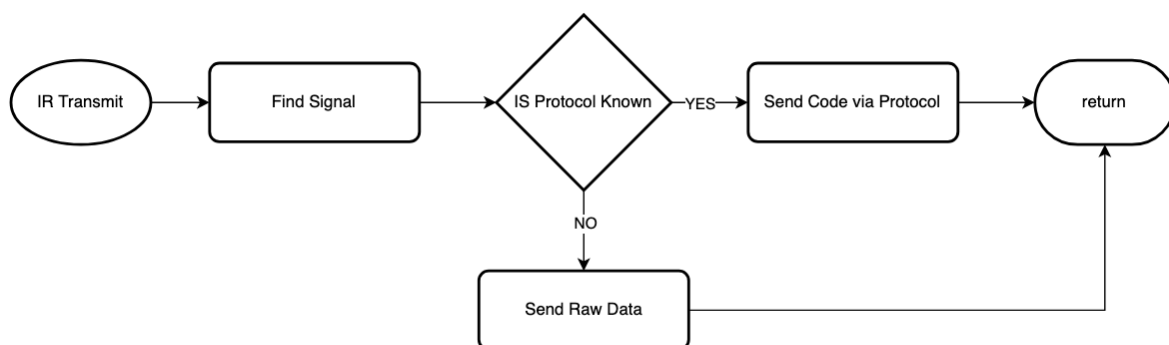
۴-۴-۳- منطق کلی دریافت و ارسال IR

فرایند دریافت سیگنال در سامانه به صورت زیر است:



شکل (۴-۴) فرایند دریافت سیگنال IR

فرایند ارسال سیگنال در سامانه به صورت زیر است:



شکل (۴-۵) فرایند ارسال سیگنال IR

۴-۵- پیاده سازی ارتباط WiFi و MQTT

به منظور ایجاد ارتباط میان دستگاه و زیرساخت نرم افزاری سامانه، از دو فناوری WiFi و MQTT استفاده شده است که در این ساختار، ارتباط WiFi وظیفه اتصال ESP32 به شبکه محلی و فراهم کردن مسیر ارتباطی میان دستگاه و MQTT Broker را بر عهده دارد و پروتکل MQTT نیز برای تبادل اطلاعات حسگرها، وضعیت سیستم و فرمان های کنترلی میان ESP32، Broker و داشبورد تحت وب مورد استفاده قرار می گیرد. پیاده سازی این بخش به گونه ای انجام شده است که قطع یا اختلال موقت در ارتباط شبکه باعث توقف عملکرد بخش های محلی سیستم نشده و Firmware بتواند در زمان مناسب برای برقراری مجدد ارتباط تلاش کند.

۴-۵-۱- ایجاد و راه اندازی MQTT Client

برای پیاده سازی ارتباط MQTT از کتابخانه PubSubClient استفاده شده است و در ابتدای مازول، یک شی از کلاس WiFiClient به عنوان زیرساخت ارتباط شبکه و یک شی از کلاس PubSubClient برای مدیریت ارتباط MQTT ایجاد شده است؛ همچنین به منظور جداسازی منطق دریافت پیام از بخش ارتباطی، یک Callback برای دریافت فرمان های MQTT تعریف شده است.

```
static WiFiClient wifiClient;
static PubSubClient mqttClient(wifiClient);
static MqttCommandHandler mqttCommandHandler = nullptr;
```

وظیفه راه اندازی اولیه ارتباط MQTT توسط تابع `setupMQTT()` انجام می شود که در آن Handler مربوط به فرمان ها ثبت شده، مشخصات MQTT Broker از تنظیمات Provisioning یا مقادیر پیش فرض خوانده شده و سپس مشخصات سرور برای MQTT Client تنظیم می شود.

```
void setupMQTT(MqttCommandHandler commandHandler)
{
    mqttCommandHandler = commandHandler;

    const char *host = provisioningConfig.mqttHost.length() > 0
        ? provisioningConfig.mqttHost.c_str()
        : MQTT_HOST;
    uint16_t port = provisioningConfig.mqttPort > 0
        ? provisioningConfig.mqttPort
        : MQTT_PORT;

    WiFi.setAutoReconnect(true);
    connectWiFi();
    mqttClient.setServer(host, port);
    mqttClient.setCallback(mqttCallback);
}
```

۲-۵-۴- برقراری و مدیریت اتصال WiFi و MQTT

اتصال ESP32 به شبکه WiFi توسط تابع `connectWiFi()` انجام می شود که ابتدا وضعیت فعلی شبکه را بررسی کرده و در صورت متصل بودن دستگاه، از ایجاد اتصال مجدد جلوگیری می کند؛ در غیر این صورت، مشخصات شبکه از تنظیمات ذخیره شده Provisioning دریافت شده و در صورت نبود اطلاعات معتبر، مقادیر پیشفرض مورد استفاده قرار می گیرند و در نهایت اتصال به شبکه آغاز می شود.

```
static void connectWiFi()
{
    if (WiFi.status() == WL_CONNECTED)
    {
        return;
    }

    const char *ssid = provisioningConfig.wifiSSID.length() > 0
        ? provisioningConfig.wifiSSID.c_str()
        : WIFI_SSID;
    const char *wpass = provisioningConfig.wifiPassword.length() > 0
        ? provisioningConfig.wifiPassword.c_str()
        : WIFI_PASSWORD;

    WiFi.begin(ssid, wpass);
}
```

پس از برقراری ارتباط WiFi، اتصال به MQTT Broker توسط تابع `connectMQTT()` انجام می شود. در

این تابع ابتدا مشخصات Broker شامل آدرس، پورت، نام کاربری و رمز عبور از تنظیمات Provisioning یا مقادیر پیشفرض دریافت شده و سپس با توجه به وجود یا عدم وجود اطلاعات احراز هویت، روش مناسب برای اتصال انتخاب می‌شود. پس از اتصال موفق، دستگاه در Topic مربوط به فرمان‌ها Subscribe می‌کند تا بتواند فرمان‌های ارسال شده از سمت داشبورد یا سایر Publisherها را دریافت کند. به این ترتیب، ESP32 علاوه بر نقش Publisher برای ارسال اطلاعات حسگرها، به عنوان Subscriber نیز در سیستم MQTT فعالیت می‌کند.

```
static bool connectMQTT()
{
    const char *host = provisioningConfig.mqttHost.length() > 0
        ? provisioningConfig.mqttHost.c_str()
        : MQTT_HOST;
    uint16_t port = provisioningConfig.mqttPort > 0
        ? provisioningConfig.mqttPort
        : MQTT_PORT;
    const char *mqttUser = provisioningConfig.mqttUser.length() > 0
        ? provisioningConfig.mqttUser.c_str()
        : MQTT_USER;
    const char *mqttPass = provisioningConfig.mqttPassword.length() > 0
        ? provisioningConfig.mqttPassword.c_str()
        : MQTT_PASSWORD;

    bool connected = false;
    if (strlen(mqttUser) > 0)
    {
        connected = mqttClient.connect(MQTT_CLIENT_ID, mqttUser, mqttPass);
    }
    else
    {
        connected = mqttClient.connect(MQTT_CLIENT_ID);
    }

    if (!connected)
    {
        Serial.print("MQTT connect failed, state=");
        Serial.println(mqttClient.state());
        return false;
    }

    String commandTopic = topicFor(MQTT_COMMAND_TOPIC);
    mqttClient.subscribe(commandTopic.c_str());
    return true;
}
```

به منظور جلوگیری از توقف برنامه در صورت قطع ارتباط، مدیریت اتصال در طول اجرای تابع loop() توسط handleMQTT() انجام می‌شود. این تابع ابتدا وضعیت WiFi و سپس وضعیت MQTT Broker را بررسی کرده و در صورت قطع بودن هر یک، پس از گذشت بازه زمانی مشخص شده، مجدداً برای برقراری ارتباط تلاش می‌کند؛ در صورتی که ارتباط برقرار باشد نیز تابع mqttClient.loop() اجرا می‌شود تا پیام‌های جدید دریافت و Callbackهای مربوطه پردازش شوند.

```
void handleMQTT()
{
```

```

if (WiFi.status() != WL_CONNECTED)
{
    unsigned long now = millis();
    if (now - lastReconnectAttempt >= MQTT_RECONNECT_INTERVAL)
    {
        lastReconnectAttempt = now;
        connectMQTT();
    }
    return;
}

if (!mqttClient.connected())
{
    unsigned long now = millis();
    if (now - lastReconnectAttempt >= MQTT_RECONNECT_INTERVAL)
    {
        lastReconnectAttempt = now;
        connectMQTT();
    }
    return;
}

mqttClient.loop();
}

```

۳-۵-۴- دریافت و انتشار فرمان های MQTT

برای دریافت فرمان های MQTT از Callback تعریف شده در زمان راه اندازی Client استفاده شده است. این تابع Payload دریافتی را از آرایه بایت به یک رشته متنی تبدیل کرده، فاصله های اضافی را حذف می کند و سپس فرمان حاصل را به mqttCommandHandler منتقل می نماید تا منطق مربوط به پردازش فرمان در بخش دیگری از Firmware انجام شود. در این ساختار، ماژول MQTT مسئول تشخیص نوع فرمان یا اجرای مستقیم آن نیست، بلکه تنها وظیفه دریافت پیام و انتقال آن به Handler مربوطه را بر عهده دارد؛ این جداسازی موجب کاهش وابستگی میان بخش ارتباطی و منطق کنترلی سیستم شده و امکان توسعه فرمان های جدید را بدون ایجاد تغییرات گسترده در ماژول MQTT فراهم می کند.

```

static void mqttCallback(char *topic, byte *payload, unsigned int length)
{
    String command;
    command.reserve(length + 1);

    for (unsigned int i = 0; i < length; i++)
    {
        command += static_cast<char>(payload[i]);
    }

    command.trim();
}

```

```

Serial.print("MQTT command on ");
Serial.print(topic);
Serial.print(": ");
Serial.println(command);

if (command.length() > 0 && mqttCommandHandler != nullptr)
{
    mqttCommandHandler(command);
}
}

```

برای انتشار اطلاعات نیز تابع publishMQTT() در نظر گرفته شده است که ابتدا وضعیت اتصال به Broker را بررسی کرده و در صورت برقرار بودن ارتباط، Topic نهایی را با استفاده از Topic پایه سیستم ایجاد و Payload را در آن منتشر می‌کند.

```

bool publishMQTT(const String &subTopic, const String &payload, bool retained)
{
    if (!mqttClient.connected())
    {
        return false;
    }

    String topic = topicFor(subTopic);
    return mqttClient.publish(topic.c_str(), payload.c_str(), retained);
}

```

۴-۶- پیاده سازی پیکربندی اولیه و مدیریت اتصال به شبکه

به منظور جلوگیری از وابستگی مستقیم تنظیمات شبکه و MQTT Broker به Firmware و فراهم کردن امکان استفاده از سامانه در شبکه های مختلف، یک زیر سامانه مستقل با عنوان Provisioning در Firmware طراحی و پیاده سازی شده است که وظیفه آن، دریافت، اعتبارسنجی، ذخیره سازی و بازیابی اطلاعات موردنیاز برای اتصال ESP32 به شبکه WiFi و MQTT Broker است؛ به کمک این زیرسامانه، کاربر می تواند بدون نیاز به تغییر در کد برنامه یا کامپایل مجدد Firmware، مشخصات شبکه، اطلاعات MQTT و نام دستگاه را از طریق یک رابط تحت وب تنظیم کرده و تنظیمات جدید را در حافظه دائمی دستگاه ذخیره کند.

۴-۶-۱- ساختار اطلاعات و راه اندازی

اطلاعات مربوط به پیکربندی ارتباطی دستگاه در ساختاری با نام ProvisioningConfig نگهداری می شوند که شامل نام و رمز عبور شبکه WiFi، آدرس و پورت MQTT Broker، اطلاعات احراز هویت MQTT و نام اختصاصی دستگاه است.

```

struct ProvisioningConfig
{
    String wifiSSID;
    String wifiPassword;
}

```

```
String mqttHost;
uint16_t mqttPort;
String mqttUser;
String mqttPassword;
String client;
};
```

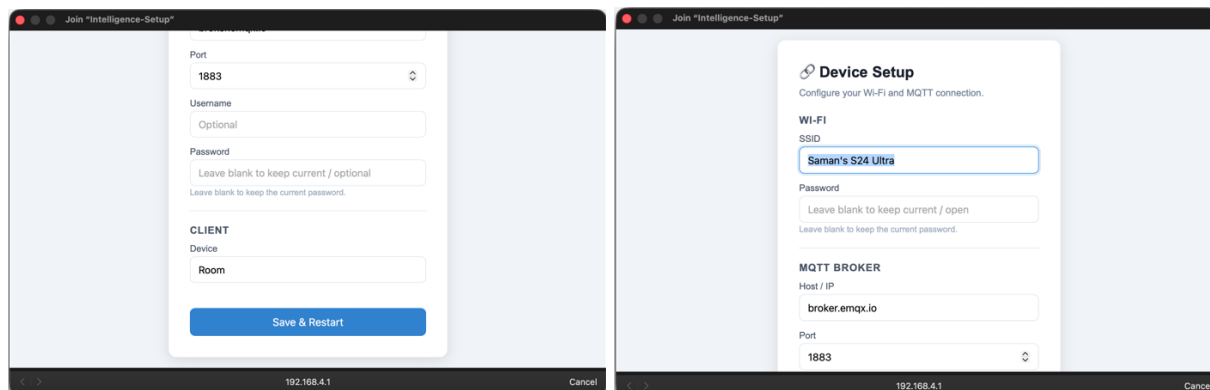
ساختار فوق باعث می شود تمام اطلاعات مورد نیاز برای برقراری ارتباط شبکه در یک ساختار مشخص و متمرکز قرار داشته باشد و سایر بخش های Firmware بتوانند بدون درگیر شدن با جزئیات مربوط به دریافت یا ذخیره سازی اطلاعات، تنظیمات مورد نیاز خود را از این ساختار دریافت کنند. مشخصات متغیرهای این ساختار در جدول زیر ارائه شده است.

جدول (۴-۱) ساختار داده تنظیمات Provisioning

متغیر	نوع داده	کاربرد	
wifiSSID	String	نام AP	اجباری
wifiPassword	String	رمز عبور WiFi	اجباری
mqttHost	String	آدرس MQTT Broker	اجباری
mqttPort	uint16_t	پورت MQTT Broker	اجباری
mqttUser	String	نام کاربری MQTT Broker	اختیاری
mqttPassword	String	رمز عبور MQTT Broker	اختیاری
client	String	نام دستگاه	اجباری

برای فراهم کردن امکان پیکربندی دستگاه، ESP32 در حالت WIFI_AP_STA قرار می گیرد که در این حالت می تواند به صورت همزمان به شبکه WiFi اصلی به عنوان Station متصل شده و یک Access Point اختصاصی نیز برای انجام تنظیمات ایجاد کند؛ بنابراین فعال بودن AP به اتصال دستگاه به شبکه اصلی وابسته نیست و کاربر می تواند حتی در زمانی که دستگاه به شبکه متصل است نیز به رابط Provisioning دسترسی داشته باشد و تنظیمات آن را تغییر دهد. مشخصات Access Point ایجاد شده در جدول (3-4) ذکر شده است. پس از اتصال کاربر به این Access Point، یک Web Server روی ESP32 اجرا شده و صفحه پیکربندی از طریق آدرس 192.168.4.1 در اختیار کاربر قرار می گیرد.

۴-۶-۲- پیاده سازی Captive Portal و مدیریت درخواست های HTTP



شکل (۴-۶) صفحه تنظیمات

برای ساده تر شدن فرایند پیکربندی، در پروژه از مکانیزم Captive Portal استفاده شده است. برای این منظور یک DNS Server ایجاد شده و تمامی درخواست های DNS به آدرس IP مربوط به Access Point هدایت می شوند. در مازول Provisioning مسیرهای مختلفی برای مدیریت درخواست های HTTP تعریف شده است که مسیر / وظیفه نمایش صفحه اصلی تنظیمات، مسیر /save/ وظیفه دریافت اطلاعات ارسال شده توسط کاربر و مسیر handleNotFound نیز وظیفه هدایت درخواست های ناشناخته به صفحه پیکربندی را بر عهده دارد؛ علاوه بر این، به منظور شناسایی خودکار Captive Portal توسط سیستم عامل های مختلف، مسیر هایی مانند generate_204/ برای Android، مسیر hotspot-detect.html/ برای Apple و مسیر های ncsi.txt/ و connecttest.txt/ برای Windows نیز به صفحه پیکربندی هدایت می شوند.

```
server.on("/", HTTP_GET, handleRoot);
// OS connectivity-check probes - redirecting them (instead of returning the
// expected "success"/204 response) is what makes the captive portal pop up
// automatically on iOS / Android / Windows.
server.on("/generate_204", HTTP_GET, redirectToPortal); // Android
server.on("/gen_204", HTTP_GET, redirectToPortal); // Android (alt)
server.on("/hotspot-detect.html", HTTP_GET, redirectToPortal); // Apple
server.on("/ncsi.txt", HTTP_GET, redirectToPortal); // Windows
server.on("/connecttest.txt", HTTP_GET, redirectToPortal); // Windows
server.on("/redirect", HTTP_GET, redirectToPortal); // Windows
server.on("/save", HTTP_POST, handleSave);
server.onNotFound(handleNotFound);
```

با استفاده از این ساختار، زمانی که کاربر به Access Point دستگاه متصل می شود، سیستم عامل با ارسال درخواست های بررسی اتصال، وجود یک شبکه دارای Captive Portal را تشخیص داده و ESP32 به جای پاسخ عادی، کاربر را به صفحه پیکربندی هدایت می کند؛ در نتیجه، در بسیاری از دستگاه ها صفحه تنظیمات بدون نیاز به وارد کردن دستی آدرس IP در مرورگر نمایش داده می شود و فرایند پیکربندی برای کاربر ساده تر خواهد بود.

۳-۶-۴- دریافت، اعتبارسنجی و ذخیره تنظیمات

پس از نمایش صفحه پیکربندی، کاربر اطلاعات مربوط به شبکه WiFi و MQTT Broker را وارد کرده و با ارسال فرم، درخواست POST به مسیر /save ارسال می شود؛ تابع `handleSave()` اطلاعات دریافتی را پردازش کرده، یک نمونه جدید از `ProvisioningConfig` ایجاد می کند و سپس مقادیر وارد شده را از نظر وجود پارامترهای ضروری بررسی می نماید، به طوری که در صورت خالی بودن نام شبکه WiFi، آدرس MQTT Broker یا نام دستگاه، درخواست معتبر تشخیص داده نشده و با خطای HTTP 400 رد می شود. یکی از نکات مهم در این پیاده سازی، نحوه مدیریت رمز های عبور است؛ در صورتی که کاربر فیلد مربوط به رمز عبور WiFi یا MQTT را خالی ارسال کند، مقدار قبلی رمز عبور حفظ می شود و به این ترتیب، رمز های ذخیره شده در حافظه نیازی به نمایش مجدد در رابط کاربری ندارند و کاربر برای تغییر سایر تنظیمات مجبور به وارد کردن مجدد رمز فعلی نخواهد بود.

پس از تأیید اعتبار اطلاعات، تنظیمات جدید توسط تابع `saveProvisioningConfig()` در حافظه دائمی سیستم ذخیره شده و سپس دستگاه Restart می شود تا Firmware با استفاده از تنظیمات جدید، فرایند اتصال به شبکه و MQTT Broker را از ابتدا انجام دهد. اطلاعات ذخیره شده پس از راه اندازی مجدد توسط سیستم `Provisioning` بازیابی شده و در اختیار ماژول های WiFi و MQTT قرار می گیرند؛ بنابراین تنظیمات اعمال شده پس از خاموش و روشن شدن دستگاه نیز حفظ خواهند شد. جزئیات کامل پیاده سازی این بخش در فایل `src/Provisioning/provisioning.cpp` قابل مشاهده است.

۷-۴- پیاده سازی مدیریت حافظه و ذخیره سازی اطلاعات

در این پروژه، به منظور حفظ اطلاعات مورد نیاز سامانه پس از خاموش و روشن شدن ESP32، از سیستم فایل LittleFS استفاده شده است که امکان ذخیره سازی اطلاعات در حافظه داخلی دستگاه را بدون نیاز به حافظه خارجی فراهم می کند. ماژول مدیریت حافظه مسئولیت ذخیره سازی و بازیابی اطلاعات مختلف سامانه شامل تنظیمات `Provisioning`، وضعیت فعال یا غیرفعال بودن تشخیص حرکت و همچنین سیگنال های ذخیره شده RF و IR را بر عهده دارد و به این ترتیب سایر ماژول های Firmware می توانند بدون درگیر شدن با جزئیات نحوه دسترسی به حافظه، اطلاعات مورد نیاز خود را از طریق توابع مربوطه ذخیره یا بازیابی کنند.

برای ساختاردهی اطلاعات ذخیره شده نیز از کتابخانه `ArduinoJson` استفاده شده است؛ در این روش، اطلاعات پیش از ذخیره سازی به ساختار JSON تبدیل شده و در فایل مربوطه قرار می گیرند و هنگام راه اندازی یا نیاز به بازیابی اطلاعات، فایل JSON خوانده شده و داده های آن مجدداً به ساختارها و متغیر های مورد استفاده Firmware تبدیل می شوند. این روش علاوه بر ساده تر کردن مدیریت اطلاعات، باعث می شود ساختار فایل های ذخیره شده خوانا و قابل توسعه باشد.

پیش از هرگونه عملیات خواندن یا نوشتن، آماده بودن سیستم فایل توسط تابع داخلی `ensureLittleFS()` بررسی می شود تا در صورت Mount نبودن LittleFS، عملیات راه اندازی مورد نیاز انجام شده و از اجرای عملیات روی یک سیستم فایل آماده نشده جلوگیری شود.

۴-۷-۱- ذخیره و بازیابی وضعیت دستگاه

یکی از اطلاعاتی که باید پس از Restart دستگاه حفظ شود، وضعیت فعال یا غیرفعال بودن قابلیت تشخیص حرکت است؛ به همین منظور این مقدار در فایل settings.json / ذخیره می شود و تابع saveMotionDetection() وظیفه تبدیل وضعیت فعلی به داده JSON و ذخیره آن را بر عهده دارد، در حالی که تابع loadMotionDetection() هنگام راه اندازی دستگاه فایل را خوانده و وضعیت قبلی را بازیابی می کند و در صورتی که فایل تنظیمات وجود نداشته باشد، مقدار پیشفرض برای وضعیت تشخیص حرکت برابر با غیرفعال در نظر گرفته می شود.

جزئیات پیاده سازی این بخش در فایل src/Memmmory/memmmory.cpp قابل مشاهده است.

۴-۷-۲- ذخیره و بازیابی سیگنال های RF و IR

سیگنال های RF دریافت شده توسط سیستم در فایل rf_signals.json و سیگنال های مادون قرمز دریافت شده نیز در فایل ir_signals.json ذخیره می شوند و ساختار ذخیره سازی آنها به گونه ای طراحی شده است که اطلاعات اصلی مورد نیاز برای بازپخش سیگنال در کنار نام اختصاصی آن نگهداری شود. برای ذخیره سیگنال های RF، تابع saveRFSignals() ابتدا یک آرایه JSON ایجاد کرده و سپس برای هر عنصر موجود در آرایه signals یک شی JSON ایجاد می کند و پارامترهایی مانند نام، کد، تعداد بیت، پروتکل، طول پالس و باند فرکانسی را در آن قرار می دهد و در نهایت آرایه ایجاد شده در فایل rf_signals.json ذخیره می شود. به صورت مشابه، تابع loadRFSignals() هنگام راه اندازی دستگاه فایل ذخیره شده را باز کرده و آرایه JSON را می خواند و سپس اطلاعات هر عنصر را استخراج کرده و در آرایه signals قرار می دهد تا سیگنال ها برای استفاده در عملیات جست و جو و بازپخش در دسترس Firmware قرار داشته باشند.

در مورد سیگنال های IR نیز فرایند مشابهی انجام می شود، با این تفاوت که علاوه بر اطلاعات Decode شده مانند پروتکل، مقدار سیگنال و تعداد بیت، اطلاعات خام سیگنال نیز در فایل ذخیره می شود تا در صورت ناشناخته بودن پروتکل یا عدم پشتیبانی کتابخانه از آن، امکان بازپخش سیگنال همچنان وجود داشته باشد. در زمان راه اندازی نیز loadIRSignals() اطلاعات موجود در ir_signals.json را بازیابی کرده و در آرایه irSignals قرار می دهد تا سیگنال های قبلی IR در اختیار سیستم قرار داشته باشند.

جزئیات پیاده سازی این بخش در فایل src/Memmmory/memmmory.cpp قابل مشاهده است.

۴-۸- راه اندازی و هماهنگی ماژول ها

هسته اصلی Firmware شامل دو تابع setup() و loop() است که به ترتیب وظیفه راه اندازی اولیه و اجرای مداوم سامانه را بر عهده دارند. تابع setup() تنها یک بار پس از روشن شدن ESP32 اجرا شده و در آن ارتباط سریال، تنظیمات ذخیره شده، Access Point مربوط به Provisioning، حسگرها و ارتباط MQTT راه اندازی می شوند؛ پس از پایان این مرحله، کنترل برنامه به تابع loop() منتقل می شود که به صورت پیوسته اجرا شده و وظایف مختلف سامانه را در طول زمان مدیریت می کند.

```
void setup()
```

```
{
  Serial.begin(115200);
  delay(200);

  loadProvisioningConfig();
  startConfigAP();

  setupSensors();
  setupMQTT(handleMqttCommand);
  printHelp();
}
```

در ابتدای فرایند راه اندازی، ارتباط سریال با سرعت 115200 baud برقرار می شود تا پیام های مربوط به وضعیت سیستم و عملیات انجام شده در Serial Monitor قابل مشاهده باشند. سپس تنظیمات مربوط به شبکه WiFi و MQTT که در حافظه داخلی ذخیره شده اند، توسط تابع loadProvisioningConfig() بازبینی شده و Access Point اختصاصی سیستم توسط تابع startConfigAP() فعال می شود. در ادامه، تابع setupSensors() وظیفه آماده سازی حسگرها و تجهیزات جانبی شامل DHT11، MQ-2، LDR، PIR، Buzzer، سیستم RF، سیستم IR و LED وضعیت را بر عهده دارد و اطلاعات سیگنال های ذخیره شده RF و IR نیز در همین مرحله از حافظه بازبینی می شوند. پس از آماده سازی اجزای سخت افزاری، ارتباط MQTT با استفاده از تابع setupMQTT() تنظیم شده و در نهایت فهرست فرمان های قابل استفاده توسط تابع printHelp() در اختیار کاربر قرار می گیرد.

```
void loop()
{
  handleSerial();
  handleMQTT();
  handleConfigAP();
  handleSensors();
}
```

پس از تکمیل عملیات راه اندازی، تابع loop به صورت مداوم اجرا می شود و هماهنگی میان بخش های مختلف Firmware را انجام می دهد. در هر چرخه اجرای این تابع، ابتدا ورودی های دریافت شده از Serial Monitor پردازش شده، سپس وضعیت ارتباط MQTT بررسی و پیام های جدید دریافت می شوند. در ادامه درخواست های مربوط به پورتال پیکربندی پردازش شده و در نهایت تابع handleSensors() وظیفه مدیریت حسگرها، وضعیت هشدار، LED، دریافت سیگنال های RF و IR و انتشار اطلاعات حسگرها را انجام می دهد. این ساختار باعث می شود وظایف مختلف سیستم به صورت مستقل در ماژول های مربوطه پیاده سازی شده و تابع loop() تنها نقش هماهنگ کننده میان این بخش ها را داشته باشد.

```
void setupSensors()
{
  setupMQ2();
  setupDHT();
  setupPIR();
  setupBuzzer();
  setupRF();
  setupIR();
  setupLDR();
  nextMQ2ReadAt = millis() + MQ2_PREHEAT_DURATION;
  loadRFSignals();
}
```

```
loadIRSignals();
setupStatusLED();
}
```

راه اندازی تجهیزات سخت افزاری توسط تابع `setupSensors()` انجام می شود. در این تابع، ابتدا اجزای مربوط به حسگرهای MQ-2، DHT11، PIR و LDR راه اندازی شده و سپس سیستم هشدار صوتی، گیرنده ها و فرستنده های RF و سیستم مادون قرمز آماده به کار می شوند. برای حسگر MQ-2، زمان اولین خوانش با توجه به مدت زمان پیش گرمایش تعیین می شود. در ادامه سیگنال های ذخیره شده RF و IR از حافظه داخلی بازیابی شده و LED وضعیت نیز راه اندازی می شود.

۴-۸-۱- مدیریت فرمان های سیستم

یکی از بخش های مهم Firmware، امکان کنترل قابلیت های مختلف دستگاه از طریق فرمان های متنی است. این فرمان ها می توانند از طریق Serial Monitor یا MQTT دریافت شده و پس از پردازش توسط Command Handler، عملیات متناظر با آنها اجرا شود. ساختار فرمان ها به گونه ای طراحی شده است که قابلیت های مختلف سامانه، از جمله سیستم RF، سیستم IR، هشدار صوتی و تشخیص حرکت، از طریق یک رابط فرمانی مشترک قابل کنترل باشند.

۴-۸-۱-۱- مدیریت فرمان های RF و IR

در صورت دریافت فرمان `rf rx <name>` ابتدا نام سیگنال بررسی شده و در صورت معتبر بودن درخواست، سیستم وارد حالت دریافت RF می شود تا سیگنال دریافتی با نام تعیین شده ذخیره شود. در فرمان `rf tx <name>`، سیگنال موردنظر در میان سیگنال های ذخیره شده جست و جو شده و در صورت وجود، توسط فرستنده مناسب بازپخش می شود. فرمان `rf list` نیز فهرست سیگنال های ذخیره شده را نمایش داده و اطلاعات آنها را از طریق MQTT در اختیار داشبورد قرار می دهد، در حالی که فرمان `rf del <name>` برای حذف سیگنال مشخص از حافظه مورد استفاده قرار می گیرد.

فرمان های مربوط به سیستم مادون قرمز نیز از ساختار مشابهی استفاده می کنند. به این ترتیب، کاربر می تواند یک سیگنال IR را با نام دلخواه دریافت و ذخیره کند، سیگنال ذخیره شده را بازپخش کند، سیگنال موردنظر را حذف کرده یا فهرست سیگنال های ذخیره شده را مشاهده کند.

جزئیات پردازش این فرمان ها در فایل `src/Serial/serial.cpp` پیاده سازی شده است.

۴-۸-۱-۲- مدیریت فرمان های سیستم هشدار و تشخیص حرکت

در صورت دریافت `buzzer on`، تابع `startAlarm()` اجرا شده و هشدار صوتی فعال می شود و در صورت دریافت `buzzer off`، تابع `stopAlarm()` وظیفه غیرفعال کردن آن را بر عهده دارد. علاوه بر کنترل مستقیم توسط کاربر، فعال شدن هشدار می تواند در نتیجه رویداد های حسگر MQ-2 یا تشخیص حرکت نیز انجام شود. با اجرای `motion on` قابلیت تشخیص حرکت فعال و با اجرای `motion off` غیرفعال می شود. وضعیت این قابلیت علاوه بر تغییر در حافظه اجرایی، در حافظه دائمی نیز ذخیره می شود تا پس از راه اندازی مجدد دستگاه، وضعیت قبلی قابل بازیابی باشد.

جزئیات پردازش این فرمان ها در فایل `src/Serial/serial.cpp` پیاده سازی شده است.

۲-۸-۴- مدیریت حسگرها و وظایف دوره ای

مدیریت دوره ای حسگرها و سایر وظایف وابسته به زمان در تابع `handleSensors()` انجام می شود. این تابع در هر چرخه از `loop()` فراخوانی شده و وضعیت دریافت RF و IR، سیستم هشدار، LED های وضعیت و تشخیص حرکت را بررسی می کند و در صورت رسیدن زمان نمونه برداری، اطلاعات حسگرها را دریافت و منتشر می کند.

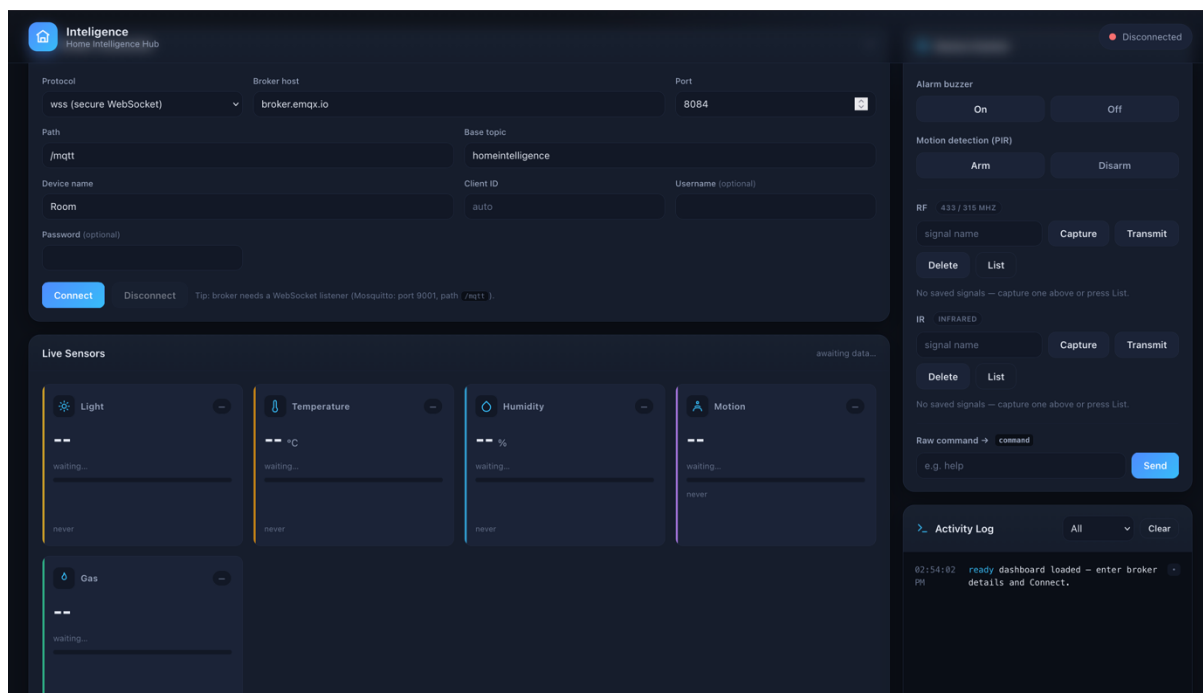
```
void handleSensors()
{
    handleRFReceive(receivingRF, receiveName);
    handleIRReceive(receivingIR, receiveName);
    updateAlarm();
    updateStatusLED();
    updatePulseLED();
    checkMotionDetection();
    printSensors();
}
```

در ابتدای اجرای تابع، وضعیت دریافت سیگنال های RF و IR بررسی می شود تا در صورت قرار داشتن سیستم در حالت دریافت، داده جدید پردازش و ذخیره شود. سپس وضعیت هشدار و LED های سیستم بروزرسانی شده و قابلیت تشخیص حرکت بررسی می شود. در نهایت تابع `printSensors()` وظیفه بررسی زمان نمونه برداری و دریافت مقادیر حسگرها را بر عهده می گیرد و داده های به دست آمده را برای انتشار از طریق MQTT آماده می کند.

برای جلوگیری از خوانش بیش از اندازه حسگرها و ارسال مداوم پیام به MQTT Broker، نمونه برداری حسگرها با یک بازه زمانی مشخص انجام می شود. در پایاده سازی فعلی، اطلاعات حسگرهای DHT11، LDR و PIR با فاصله زمانی ۱۰ ثانیه بررسی و در صورت رسیدن زمان نمونه برداری، مقادیر جدید از طریق MQTT منتشر می شوند. حسگر MQ-2 به دلیل نیاز به پیش گرمایش، منطق زمانی مستقلی دارد و اولین خوانش آن پس از پایان مدت پیش گرمایش انجام می شود.

جزئیات پایاده سازی این بخش در فایل `src/Sensors/sensors.cpp` قابل مشاهده است.

۴-۹- پیاده سازی داشبورد وب و رابط کاربری



شکل (۴-۷) داشبورد وب

به منظور فراهم کردن یک رابط کاربری مناسب برای مشاهده وضعیت سامانه، دریافت اطلاعات حسگرها، مشاهده رویداد های هشدار و ارسال فرمان های کنترلی، یک داشبورد تحت وب برای پروژه طراحی و پیاده سازی شده است. این داشبورد به صورت Server-less طراحی شده و برای تبادل اطلاعات میان رابط کاربری و ESP32، مستقیماً به MQTT Broker متصل می شود؛ از آنجا که مرورگر ها امکان برقراری ارتباط مستقیم با MQTT روی پروتکل TCP را ندارند، ارتباط داشبورد با Broker از طریق MQTT over WebSocket انجام می شود. در نتیجه، داشبورد بدون نیاز به یک Backend اختصاصی می تواند پیام های منتشر شده توسط ESP32 را دریافت کرده و فرمان های موردنظر کاربر را نیز در Topic مربوط به دستگاه منتشر کند.

۴-۹-۲- اتصال به MQTT Broker و دریافت اطلاعات

در بخش تنظیمات داشبورد، پارامتر های موردنیاز برای برقراری ارتباط با MQTT Broker در اختیار کاربر قرار گرفته است. کاربر می تواند آدرس Broker، پورت ارتباطی، نام کاربری و رمز عبور و همچنین نوع ارتباط WebSocket را مشخص کند. داشبورد از دو حالت WebSocket و WebSocket Secure (WSS) پشتیبانی می کند که امکان استفاده از ارتباط معمولی یا ارتباط رمزنگاری شده را فراهم می سازد. پس از برقراری اتصال، داشبورد در Topic های مربوط به دستگاه Subscribe می کند و پیام های دریافتی را به صورت لحظه ای پردازش می نماید. داده های مربوط به حسگرهای DHT11، LDR، PIR و MQ-2 در Topic های مشخص منتشر شده و داشبورد پس از دریافت هر پیام، Topic آن را بررسی کرده و مقدار دریافت

شده را در بخش مربوط به همان حسگر نمایش می دهد. به این ترتیب، تغییر مقادیر حسگرها بدون نیاز به Refresh کردن صفحه در رابط کاربری نمایش داده می شود. برای افزایش قابلیت بررسی و تحلیل داده ها، برای حسگرهای دارای خروجی عددی، نمودار تغییرات نیز در نظر گرفته شده است تا کاربر بتواند روند تغییرات اطلاعات حسگر را در طول زمان مشاهده کند. همچنین به منظور جلوگیری از نمایش داده هایی که ممکن است مربوط به زمان گذشته باشند، زمان آخرین دریافت پیام هر حسگر در داشبورد ثبت می شود و در صورتی که بیشتر از ۳۰ ثانیه از آخرین پیام دریافت شده گذشته باشد، داده مربوطه به عنوان Stale Data یا داده قدیمی علامت گذاری می شود.

۳-۹-۴- ارسال فرمان و مدیریت وضعیت سامانه

داشبورد علاوه بر نمایش اطلاعات، امکان ارسال فرمان های کنترلی به ESP32 را نیز فراهم می کند. فرمان ها در Topic مربوط به دستورات منتشر شده و پس از دریافت توسط ESP32، به Command Handler منتقل می شوند تا عملیات متناظر با آنها اجرا شود. از طریق این بخش، کاربر می تواند قابلیت های مختلف سیستم را بدون استفاده از Serial Monitor کنترل کند. با استفاده از این فرمان ها، کاربر می تواند از طریق رابط گرافیکی داشبورد یک سیگنال جدید دریافت و ذخیره کند، سیگنال ذخیره شده را بازپخش نماید، سیگنال مشخصی را حذف کند یا فهرست سیگنال های موجود را مشاهده نماید. فهرست سیگنال ها پس از دریافت از Topic های RF/list و IR/list در بخش مربوطه نمایش داده می شود و به این ترتیب مدیریت سیگنال های ذخیره شده بدون نیاز به دسترسی مستقیم به دستگاه امکان پذیر است. همچنین کنترل سیستم هشدار و قابلیت تشخیص حرکت نیز از طریق داشبورد انجام می شود. برای نمایش رویداد های مهم، یک بخش هشدار در قسمت بالایی داشبورد طراحی شده است که پیام های مربوط به تشخیص حرکت و عبور مقدار MQ-2 از آستانه تعیین شده را به کاربر نمایش می دهد. این بخش باعث می شود رویداد های مهم سیستم در کنار اطلاعات عادی حسگرها قابل مشاهده بوده و کاربر بتواند در کوتاه ترین زمان نسبت به وضعیت غیرعادی واکنش نشان دهد.

فصل ۵:

آزمون و ارزیابی سیستم

پس از طراحی و پیاده سازی بخش های سخت افزاری و نرم افزاری سامانه، عملکرد کلی سیستم در شرایط مختلف مورد آزمون و ارزیابی قرار گرفت تا از صحت عملکرد اجزای مختلف و هماهنگی میان ماژول های سخت افزاری و نرم افزاری اطمینان حاصل شود. در این فرایند، عملکرد حسگرها، سیستم هشدار، ارتباط WiFi و MQTT، قابلیت Provisioning، دریافت و بازپخش سیگنال های RF و IR، ذخیره سازی اطلاعات در حافظه داخلی و همچنین عملکرد داشبورد وب بررسی شد. آزمون ها به گونه ای انجام شدند که علاوه بر بررسی عملکرد مستقل هر بخش، نحوه تعامل ماژول ها با یکدیگر و واکنش سامانه در شرایط مختلف نیز مورد ارزیابی قرار گیرد.

۵-۱- روش آزمون و ارزیابی سامانه

به منظور ارزیابی عملکرد سامانه و بررسی صحت عملکرد بخش های مختلف آن، آزمون ها بر روی برد توسعه ESP32 DevKit و با استفاده از تجهیزات سخت افزاری به کار رفته در نمونه نهایی انجام شده است. در بخش اندازه گیری پارامتر های محیطی، از حسگر DHT11 برای اندازه گیری دما و رطوبت، ماژول LDR برای اندازه گیری شدت نور، خروجی آنالوگ حسگر MQ-2 برای تشخیص دود و گاز های قابل اشتعال و ماژول PIR برای تشخیص حرکت استفاده شده است. همچنین برای بررسی عملکرد سیستم هشدار، یک Active Buzzer و برای آزمایش قابلیت کنترل تجهیزات دارای کنترل از راه دور، مجموعه فرستنده و گیرنده RF در دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز و مجموعه گیرنده و فرستنده مادون قرمز مورد استفاده قرار گرفته است. آزمون های نرم افزاری نیز با استفاده از MQTT Broker، داشبورد وب و رابط Serial Monitor انجام شده اند تا علاوه بر عملکرد مستقل هر بخش، ارتباط میان اجزای مختلف سامانه نیز مورد بررسی قرار گیرد.

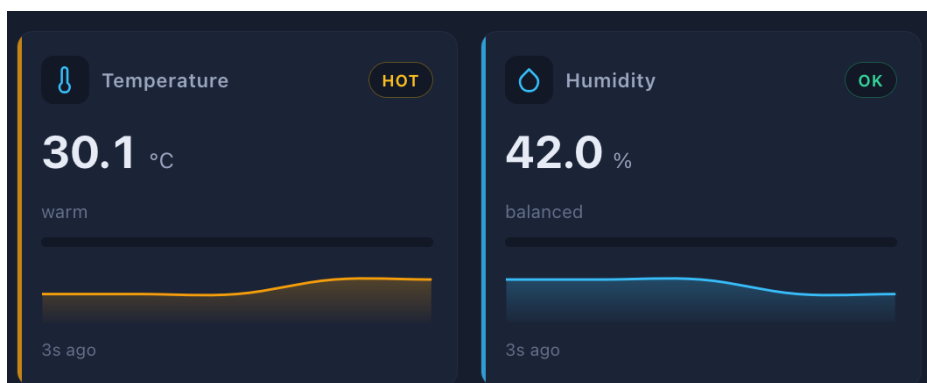
برای ارزیابی سامانه، معیار هایی در نظر گرفته شده است که هم عملکرد سخت افزار و هم عملکرد ارتباطی و نرم افزاری سیستم را پوشش می دهند. در بخش سخت افزاری، صحت دریافت اطلاعات از حسگر های محیطی، تشخیص صحیح شرایط هشدار و عملکرد سیستم هشدار صوتی بررسی شده است. در بخش ارتباطی، صحت اتصال ESP32 به شبکه Wi-Fi، برقراری ارتباط با MQTT Broker و دریافت صحیح فرمان های کنترلی مورد ارزیابی قرار گرفته است. همچنین قابلیت دریافت، ذخیره سازی و بازپخش سیگنال های RF در هر دو باند ۳۱۵ و ۴۳۳ مگاهرتز و قابلیت دریافت، ذخیره سازی و بازپخش سیگنال های IR مورد آزمون قرار گرفته است. در ادامه، عملکرد سیستم Provisioning شامل ایجاد Access Point، نمایش Captive Portal، دریافت و ذخیره تنظیمات و برقراری اتصال بر اساس تنظیمات جدید بررسی شده و در نهایت عملکرد داشبورد وب از نظر دریافت زنده اطلاعات حسگرها، نمایش هشدارها، ارسال فرمان ها و مدیریت سیگنال های ذخیره شده ارزیابی شده است.

۵-۲- آزمون عملکرد حسگرها

به منظور بررسی صحت عملکرد بخش اندازه گیری و تشخیص شرایط محیطی، هر یک از حسگر های مورد استفاده در سامانه به صورت مستقل مورد آزمون قرار گرفتند. در این آزمون ها ابتدا رفتار حسگر در شرایط عادی بررسی شده و سپس با ایجاد تغییر در پارامتر محیطی موردنظر، پاسخ حسگر و نحوه پردازش اطلاعات توسط Firmware ارزیابی شد. علاوه بر بررسی مقدار خام دریافت شده از حسگرها، نحوه ارسال اطلاعات از

طریق MQTT و عملکرد منطق هشدار در شرایط تعیین شده نیز مورد بررسی قرار گرفت.

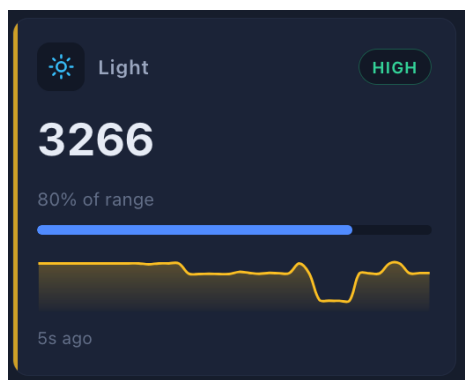
۵-۲-۱- آزمون حسگر دما و رطوبت



شکل (۵-۱) نتایج حسگر دما و رطوبت

برای بررسی عملکرد حسگر DHT11، دستگاه در شرایط محیطی عادی قرار گرفت و مقادیر دما و رطوبت به صورت دوره ای توسط Firmware از حسگر دریافت و در خروجی Serial Monitor نمایش داده شد. همچنین صحت انتشار مقادیر دریافت شده در Topic های مربوط به دما و رطوبت MQTT مورد بررسی قرار گرفت. با مشاهده تغییرات منطقی و پایدار مقادیر و دریافت صحیح داده ها، مشخص شد که ارتباط ESP32 با حسگر DHT11 و فرایند خواندن و ارسال اطلاعات به درستی انجام می شود و حسگر در شرایط آزمون عملکرد مطلوبی دارد.

۵-۲-۲- آزمون حسگر نور

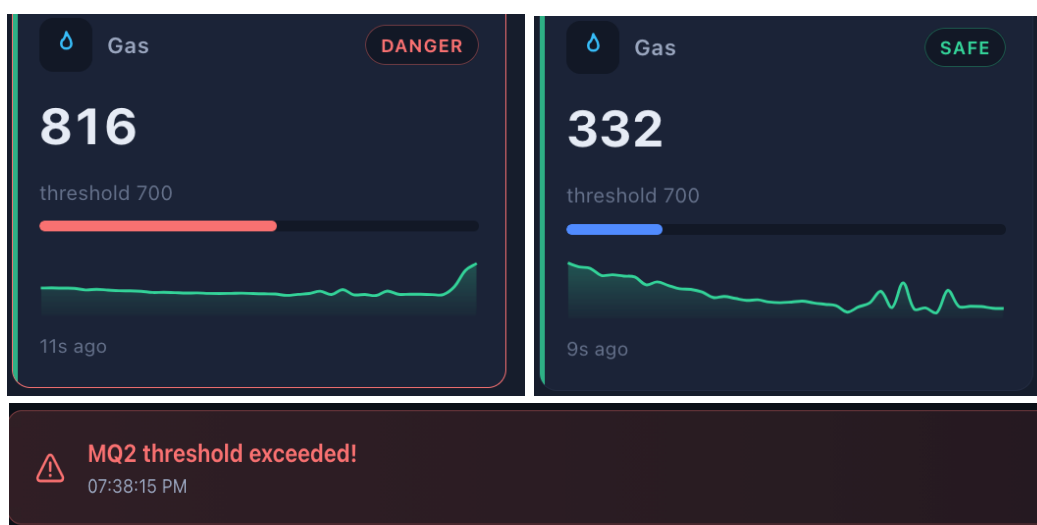


شکل (۵-۲) نتیجه حسگر نور

برای بررسی عملکرد ماژول LDR، شرایط نوری محیط در چند وضعیت مختلف تغییر داده شد و مقدار

خروجی آنالوگ حسگر توسط مبدل ADC مربوط به ESP32 اندازه گیری گردید. در این آزمون انتظار می رود با تغییر شدت نور محیط، مقدار خوانده شده از ورودی آنالوگ نیز تغییر کند و این تغییرات در خروجی سیستم قابل مشاهده باشد. نتایج به دست آمده نشان داد که با تغییر شرایط نوری، مقدار ADC نیز تغییر کرده و بنابراین حسگر LDR قادر است تغییرات شدت نور محیط را به درستی تشخیص داده و مقدار متناظر را در اختیار Firmware قرار دهد.

۵-۲-۳- آزمون حسگر دود و آتش



شکل (۵-۳) نتایج حسگر دود و آتش

به منظور ارزیابی عملکرد حسگر MQ-2، ابتدا دستگاه در شرایط عادی محیط قرار داده شد و مقدار خروجی آنالوگ حسگر اندازه گیری گردید؛ سپس با قرار دادن حسگر در معرض دود و شرایط آزمایشی مربوط به آتش، افزایش مقدار خروجی و واکنش منطق تشخیص هشدار مورد بررسی قرار گرفت. در شرایط عادی، مقدار خوانده شده از حسگر در حدود بازه ۳۰۰ تا ۳۵۰ قرار داشت، در حالی که در شرایط آزمایش آتش مقدار خروجی به بیش از ۷۰۰ افزایش یافت که در پیاده سازی سیستم به عنوان آستانه تشخیص وضعیت آتش در نظر گرفته شده است. با عبور مقدار حسگر از این آستانه، سیستم علاوه بر تشخیص شرایط هشدار، وضعیت Alarm را فعال کرده و پیام مربوط به وقوع هشدار را از طریق MQTT منتشر کرد؛ بنابراین عملکرد حسگر MQ-2 و ارتباط آن با منطق تشخیص هشدار مطابق انتظار ارزیابی شد.

۵-۲-۴- آزمون حسگر تشخیص حرکت



شکل (۵-۴) نتایج حسگر تشخیص حرکت

برای بررسی عملکرد ماژول PIR، ابتدا دستگاه در محیطی بدون حرکت قرار داده شد تا وضعیت اولیه حسگر مشخص شود و سپس با ایجاد حرکت در محدوده تشخیص حسگر، تغییر وضعیت خروجی آن و واکنش Firmware مورد بررسی قرار گرفت. نتایج آزمون نشان داد که حسگر PIR در شرایط تعیین شده قادر به تشخیص حرکت بوده و تغییر وضعیت خروجی آن توسط ESP32 به درستی شناسایی می‌شود. همچنین در صورتی که قابلیت تشخیص حرکت در تنظیمات سیستم فعال باشد، پس از شناسایی حرکت، منطق هشدار اجرا شده و پیام مربوط به وقوع رویداد از طریق MQTT ارسال می‌شود؛ در نتیجه عملکرد حسگر PIR و ارتباط آن با سیستم هشدار نیز مطابق طراحی انجام شده مورد تأیید قرار گرفت.

۵-۳- آزمون سامانه هشدار

در این آزمون، عملکرد سامانه هشدار صوتی و نحوه واکنش آن به وضعیت های مختلف سیستم مورد بررسی قرار گرفت. در مرحله نخست، عملکرد Buzzer از طریق فرمان های کنترلی سیستم مورد آزمایش قرار گرفت. علاوه بر کنترل مستقیم توسط کاربر، عملکرد ارتباط Buzzer با رویداد های تشخیص داده شده توسط حسگرها نیز بررسی شد؛ به این صورت که با عبور مقدار خروجی حسگر MQ-2 از آستانه تعیین شده و همچنین در صورت فعال بودن قابلیت تشخیص حرکت و شناسایی حرکت، وضعیت هشدار فعال شده و Buzzer مطابق منطق تعریف شده شروع به فعالیت می‌کند. نتایج آزمون نشان داد که سامانه هشدار صوتی مطابق طراحی انجام شده عمل می‌کند و در هنگام فعال شدن وضعیت هشدار، Buzzer به صورت متناوب فعال و غیرفعال شده و با غیرفعال کردن هشدار نیز عملکرد آن متوقف می‌شود. همچنین ارتباط میان رویداد های حسگرها و سیستم هشدار به درستی برقرار بوده و در شرایط تعیین شده، هشدار صوتی و پیام مربوط به رویداد از طریق سامانه ارسال می‌شود.

۴-۵- آزمون اتصال به WiFi و MQTT Broker

```
Connecting WiFi to Saman's S24 Ultra
Connecting MQTT to broker.emqx.io:1883
MQTT subscribed: homeintelligence/Room/command
```

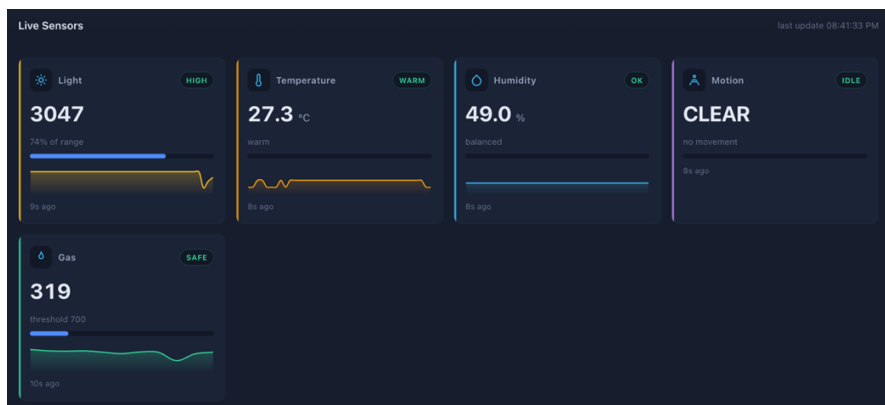
شکل (۵-۵) نتیجه آزمون اتصال به شبکه و MQTT Broker

در این آزمون، عملکرد سامانه در برقراری ارتباط با شبکه WiFi و سپس اتصال به MQTT Broker مورد بررسی قرار گرفت. هدف از انجام این آزمون، اطمینان از صحت عملکرد بخش ارتباطی ESP32 و همچنین بررسی امکان تبادل اطلاعات میان دستگاه و MQTT Broker بود. در این فرایند، ابتدا دستگاه با استفاده از تنظیمات ذخیره شده در بخش Provisioning برای اتصال به شبکه WiFi تلاش می کند و پس از برقراری موفقیت آمیز ارتباط، اتصال به MQTT Broker برقرار می شود. پس از اتصال به Broker، امکان انتشار پیام های مربوط به اطلاعات حسگرها و همچنین دریافت پیام های کنترلی از طریق Topic مربوطه مورد آزمایش قرار گرفت.

همانطور که مشاهده می شود، دستگاه ابتدا با موفقیت به شبکه WiFi متصل شده و پس از آن ارتباط با MQTT Broker نیز برقرار شده است. همچنین آزمون ارسال و دریافت پیام نشان داد که دستگاه قادر است اطلاعات را به Broker منتشر کرده و پیام های ارسال شده از سمت Broker را نیز دریافت و پردازش کند. بنابراین، ارتباط دو طرفه میان ESP32 و MQTT Broker مطابق طراحی انجام شده به درستی برقرار بوده و زیرساخت ارتباطی موردنیاز برای تبادل اطلاعات و فرمان های کنترلی سامانه عملکرد صحیحی دارد.

۵-۵- آزمون داشبورد سامانه

در این آزمون، عملکرد بخش های مختلف داشبورد وب و نحوه ارتباط آن با ESP32 از طریق MQTT مورد بررسی قرار گرفت. هدف از انجام این آزمون، اطمینان از صحت دریافت و نمایش اطلاعات حسگرها، نمایش رویداد های هشدار و همچنین امکان ارسال فرمان های کنترلی از رابط کاربری به دستگاه بود. با توجه به اینکه داشبورد به صورت مستقیم از طریق MQTT over WebSocket با Broker ارتباط برقرار می کند، عملکرد ارتباط میان داشبورد، Broker و ESP32 نیز در جریان این آزمون مورد ارزیابی قرار گرفت.



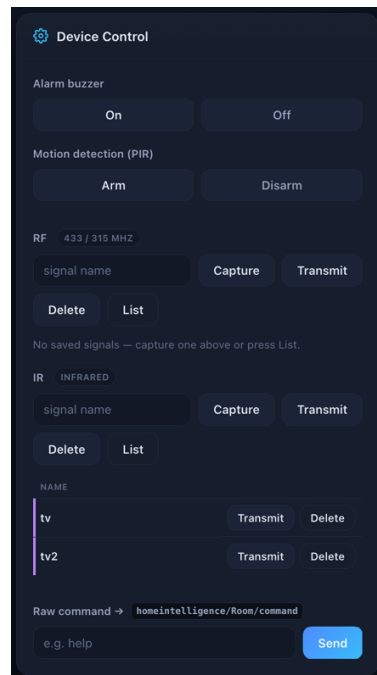
شکل (۵-۶) نمایش اطلاعات حسگرها در داشبورد

در مرحله نخست، بخش نمایش اطلاعات حسگرها مورد بررسی قرار گرفت. پس از برقراری ارتباط داشبورد با MQTT Broker، اطلاعات ارسال شده توسط ESP32 از Topic های مربوط به حسگرها دریافت شده و مقادیر دما، رطوبت، شدت نور، وضعیت حرکت و مقدار حسگر MQ-2 در بخش های مربوطه نمایش داده شدند. همچنین تغییر مقادیر حسگرها بدون نیاز به بارگذاری مجدد صفحه در داشبورد نمایش داده شد که نشان دهنده عملکرد صحیح ارتباط بلادرنگ میان دستگاه و رابط کاربری است.



شکل (۷-۵) نمایش هشدار در داشبورد

در مرحله دوم، عملکرد بخش نمایش وضعیت هشدار مورد بررسی قرار گرفت. با ایجاد شرایط هشدار توسط حسگرهای سامانه، پیام مربوط به رویداد از طریق MQTT منتشر شده و داشبورد پس از دریافت آن، وضعیت هشدار را در بخش مربوطه نمایش داد. نتایج این آزمون نشان داد که رویداد های هشدار به درستی از دستگاه به Broker و سپس به داشبورد منتقل شده و در رابط کاربری نمایش داده می شوند.



شکل (۸-۵) ارسال فرمان از داشبورد

در مرحله پایانی، قابلیت ارسال فرمان های کنترلی از داشبورد به ESP32 مورد آزمایش قرار گرفت. در این آزمون، فرمان هایی مانند فعال و غیرفعال کردن Buzzer، کنترل قابلیت تشخیص حرکت و همچنین فرمان های مربوط به دریافت، ارسال، حذف و نمایش سیگنال های RF و IR از طریق داشبورد ارسال شدند. ESP32 پس از دریافت پیام از Topic فرمان، آن را پردازش کرده و عملیات مربوطه را اجرا کرد. بنابراین، نتایج آزمون نشان داد که مسیر ارتباطی از داشبورد تا MQTT Broker و سپس ESP32 به درستی عمل کرده و داشبورد قادر است علاوه بر نمایش اطلاعات، کنترل بخش های مختلف سامانه را نیز انجام دهد.

۵-۶- RF آزمون سیستم

به منظور ارزیابی عملکرد بخش ارتباط رادیویی سامانه، قابلیت های دریافت، ذخیره سازی، بازپخش و مدیریت سیگنال های RF مورد آزمایش قرار گرفت. در این آزمون، عملکرد ارتباط میان گیرنده های RF، Firmware و حافظه داخلی دستگاه بررسی شد و همچنین امکان کنترل و مدیریت سیگنال های ذخیره شده از طریق فرمان های سیستم مورد ارزیابی قرار گرفت.

۵-۶-۱ آزمون دریافت سیگنال RF

```
MQTT command on homeintelligence/Room/command: rf rx test
Waiting for RF signal named: test
Saved RF signal: test (433 MHz)
```

شکل (۵-۹) دریافت سیگنال RF

در این آزمون، قابلیت دریافت سیگنال RF از طریق یک ریموت کنترل مورد بررسی قرار گرفت. برای انجام آزمون، سیستم در حالت دریافت قرار داده شده و سپس یک فرمان از ریموت کنترل ارسال شد تا گیرنده های RF سیگنال را دریافت و پارامترهای مربوط به آن را استخراج کنند. پس از دریافت موفقیت آمیز سیگنال، اطلاعات آن شامل کد، تعداد بیت، پروتکل، طول پالس و باند فرکانسی شناسایی شده در ساختار مربوط به سیگنال ذخیره شد و نام تعیین شده توسط کاربر نیز به آن اختصاص داده شد. همانطور که مشاهده می شود، سیگنال ارسال شده از ریموت به درستی توسط سیستم دریافت و پردازش شده و اطلاعات آن با نام تعیین شده در حافظه دستگاه ذخیره شده است؛ بنابراین، عملکرد بخش دریافت و ذخیره سازی اولیه سیگنال RF مطابق طراحی انجام شده صحیح است.

۵-۶-۲ آزمون ارسال سیگنال RF

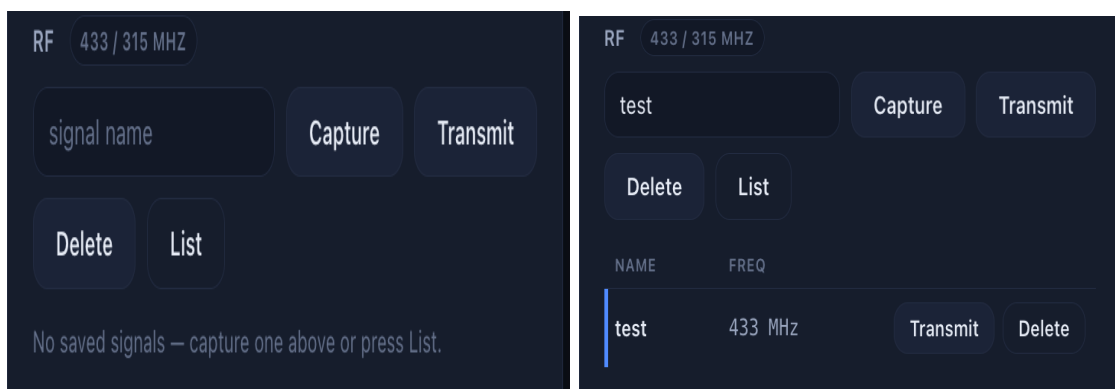
```
MQTT command on homeintelligence/Room/command: rf tx test
Transmitted RF signal: test
```

شکل (۵-۱۰) ارسال سیگنال RF

در این آزمون، قابلیت بازپخش سیگنال RF ذخیره شده مورد بررسی قرار گرفت. برای انجام آزمون، نام یکی از سیگنال های ذخیره شده به سیستم ارسال شد و Firmware پس از پیدا کردن سیگنال موردنظر، باند فرکانسی ذخیره شده را بررسی کرده و فرستنده متناسب با آن را انتخاب کرد. سپس پارامترهای مربوط به سیگنال، شامل پروتکل، طول پالس، کد و تعداد بیت، برای بازپخش مورد استفاده قرار گرفتند. همانطور که مشاهده می شود، سیگنال ذخیره شده بدون نیاز به تعیین مجدد فرکانس توسط کاربر، به درستی از طریق

فرستنده مربوطه ارسال شده و عملکرد بازپخش سیگنال RF با موفقیت انجام شده است. این نتیجه نشان می‌دهد که اطلاعات ذخیره شده سیگنال برای بازسازی و ارسال مجدد آن به درستی مورد استفاده قرار می‌گیرد.

۳-۶-۵- آزمون مدیریت سیگنال RF



```
MQTT command on homeintelligence/Room/command: rf del test
Deleted RF signal: test
```

شکل (۵-۱۱) مدیریت سیگنال RF

در این آزمون، قابلیت مدیریت سیگنال‌های ذخیره شده شامل نمایش فهرست سیگنال‌ها و حذف یک سیگنال مشخص مورد بررسی قرار گرفت. ابتدا با استفاده از فرمان مربوط به فهرست سیگنال‌ها، اطلاعات سیگنال‌های موجود در حافظه بازیابی و نمایش داده شد و سپس یکی از سیگنال‌های ذخیره شده با استفاده از نام آن برای حذف انتخاب شد. پس از حذف، اطلاعات سیگنال‌های موجود دوباره نمایش داده شد. همانطور که مشاهده می‌شود، فهرست سیگنال‌های ذخیره شده به درستی نمایش داده شده و اطلاعات مربوط به سیگنال‌ها قابل مشاهده است. همچنین پس از اجرای فرمان حذف، سیگنال موردنظر از آرایه سیگنال‌ها و حافظه دائمی حذف شده و در فهرست سیگنال‌های موجود نیز نمایش داده نمی‌شود. بنابراین، قابلیت‌های اصلی مدیریت سیگنال‌های RF مطابق طراحی انجام شده به درستی عمل می‌کنند.

۷-۵- آزمون سیستم IR

به منظور ارزیابی عملکرد بخش ارتباط مادون قرمز سامانه، قابلیت‌های دریافت، ذخیره سازی، بازپخش و مدیریت سیگنال‌های IR مورد آزمایش قرار گرفت. در این آزمون، عملکرد گیرنده و فرستنده مادون قرمز و همچنین نحوه پردازش اطلاعات دریافت شده توسط Firmware و ذخیره آنها در حافظه داخلی مورد بررسی قرار گرفت تا صحت عملکرد این بخش در شرایط عملی ارزیابی شود.

۵-۷-۱- آزمون دریافت سیگنال IR

```
MQTT command on homeintelligence/Room/command: ir rx test
Waiting for IR signal named: test
NEC
1068E11E
32
68
Saved IR signal: test
```

شکل (۵-۱۲) دریافت سیگنال IR

در این آزمون، قابلیت دریافت سیگنال مادون قرمز از طریق یک کنترل از راه دور IR مورد بررسی قرار گرفت. برای انجام آزمون، سیستم در حالت دریافت قرار داده شد و سپس یکی از فرمان های کنترل از راه دور به سمت گیرنده مادون قرمز ارسال گردید. پس از دریافت سیگنال، اطلاعات آن توسط کتابخانه مربوطه پردازش شده و در صورت شناسایی پروتکل، اطلاعات Decode شده و همچنین داده خام سیگنال در ساختار مربوطه ذخیره شد. همانطور که مشاهده می شود، سیگنال ارسال شده از کنترل IR به درستی توسط گیرنده دریافت و پردازش شده و اطلاعات آن با نام تعیین شده توسط کاربر در حافظه دستگاه ذخیره شده است. بنابراین، عملکرد بخش دریافت و ذخیره سازی سیگنال های مادون قرمز مطابق طراحی انجام شده صحیح است.

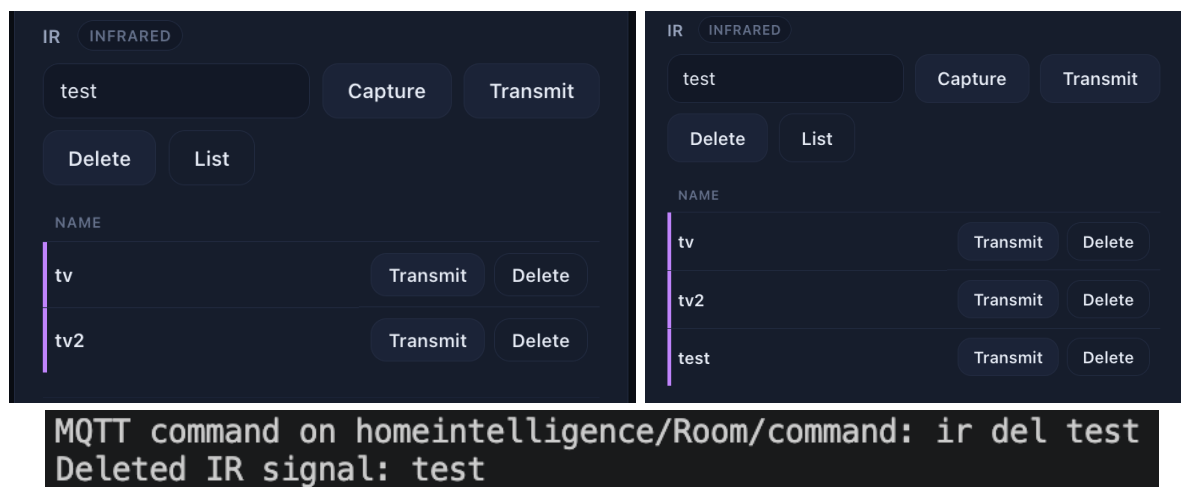
۵-۷-۲- آزمون ارسال سیگنال IR

```
MQTT command on homeintelligence/Room/command: ir tx test
Transmitting NEC signal:
Transmitted IR signal: test
```

شکل (۵-۱۳) ارسال سیگنال IR

در این آزمون، قابلیت بازپخش سیگنال مادون قرمز ذخیره شده مورد بررسی قرار گرفت. برای انجام آزمون، نام یکی از سیگنال های ذخیره شده به سیستم ارسال شد و Firmware پس از پیدا کردن سیگنال موردنظر، اطلاعات مربوط به پروتکل و داده ذخیره شده را بازیابی کرده و روش مناسب برای ارسال آن را انتخاب کرد. در صورتی که پروتکل سیگنال توسط سیستم پشتیبانی شود، سیگنال بر اساس اطلاعات Decode شده ارسال شده و در غیر این صورت، داده خام ذخیره شده برای بازپخش مورد استفاده قرار می گیرد. همانطور که مشاهده می شود، سیگنال ذخیره شده به درستی توسط LED های مادون قرمز بازپخش شده و فرمان مربوطه به تجهیز مقصد منتقل شده است. این نتیجه نشان می دهد که اطلاعات ذخیره شده سیگنال IR به درستی بازیابی شده و فرایند بازپخش سیگنال مطابق طراحی سیستم انجام می شود.

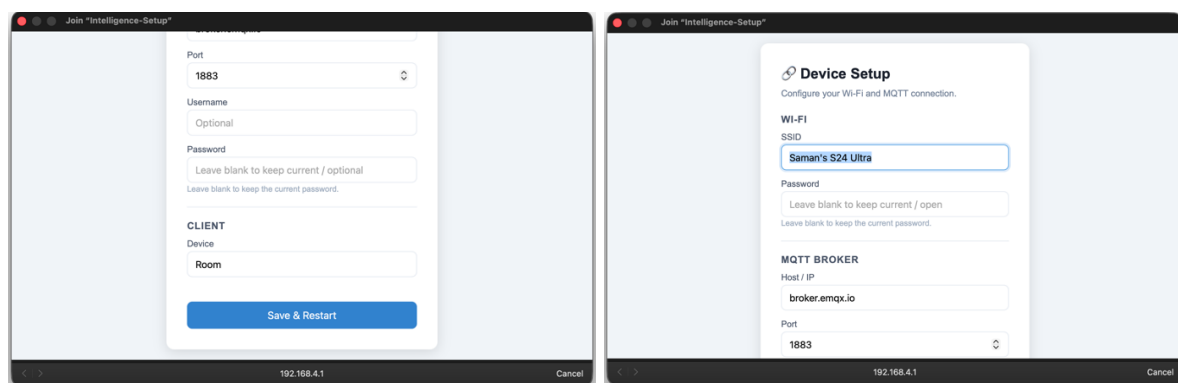
۳-۷-۵- آزمون مدیریت سیگنال IR

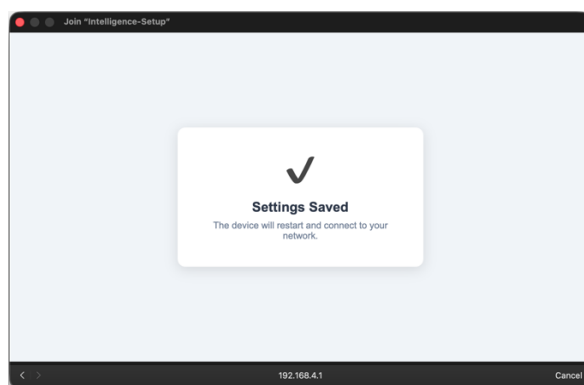


شکل (۱۴-۵) مدیریت سیگنال IR

در این آزمون، قابلیت مدیریت سیگنال های مادون قرمز ذخیره شده، شامل نمایش فهرست سیگنال ها و حذف یک سیگنال مشخص، مورد بررسی قرار گرفت. ابتدا با استفاده از فرمان مربوط به نمایش فهرست، اطلاعات سیگنال های موجود در حافظه بازبایی و نمایش داده شد و سپس یکی از سیگنال ها بر اساس نام آن انتخاب و حذف گردید. پس از حذف، اطلاعات سیگنال های موجود دوباره نمایش داده شد. همانطور که مشاهده می شود، فهرست سیگنال های IR ذخیره شده به درستی نمایش داده شده و اطلاعات مربوط به سیگنال ها قابل مشاهده است. همچنین پس از حذف یک سیگنال مشخص، آن سیگنال از فهرست موجود حذف شده و در نمایش های بعدی نیز مشاهده نمی شود. در نتیجه، قابلیت های دریافت، ارسال و مدیریت سیگنال های IR در سامانه مطابق طراحی انجام شده به درستی عمل می کنند.

۸-۵- آزمون زیرسامانه Provisioning





شکل (۵-۱۵) صفحه تنظیمات Provisioning

در این آزمون، عملکرد زیرسامانه Provisioning از نظر ایجاد شبکه اختصاصی، نمایش خودکار پورتال پیکربندی، دریافت اطلاعات و ذخیره سازی تنظیمات مورد بررسی قرار گرفته است. ابتدا دستگاه در حالت Provisioning قرار گرفته و یک Access Point اختصاصی ایجاد شد و پس از اتصال کاربر به این شبکه، صفحه تنظیمات به صورت خودکار نمایش داده شد. سپس اطلاعات موردنیاز شامل مشخصات شبکه WiFi و پارامترهای اتصال به MQTT Broker وارد و ارسال گردید و عملکرد سامانه در دریافت و اعتبارسنجی این اطلاعات بررسی شد. در ادامه، تنظیمات با موفقیت در حافظه داخلی ESP32 ذخیره شده و دستگاه به صورت خودکار راه اندازی مجدد شد تا تنظیمات جدید در فرایند اتصال مورد استفاده قرار گیرند. نتایج آزمون نشان داد که زیرسامانه Provisioning مطابق طراحی انجام شده عمل کرده و امکان پیکربندی و تغییر تنظیمات ارتباطی دستگاه را بدون نیاز به تغییر یا بارگذاری مجدد Firmware فراهم می کند.

فصل ۶:

نتیجه گیری

در این فصل، نتایج حاصل از طراحی، پیاده سازی و ارزیابی سامانه هوشمند تشخیص دود و آتش و کنترل تجهیزات الکتریکی ارائه می شود. هدف اصلی پروژه، طراحی و توسعه یک سامانه مبتنی بر اینترنت اشیا بوده است که بتواند با استفاده از حسگرهای مختلف، برخی پارامترهای محیطی را به صورت مستمر پایش کرده و شرایط غیرعادی مانند افزایش میزان دود و گازهای قابل اشتعال یا وقوع حرکت را شناسایی کند و در صورت تشخیص وضعیت هشدار، اقدامات لازم را انجام دهد. علاوه بر قابلیت های مربوط به پایش و تشخیص شرایط محیطی، با استفاده از ارتباطات مادون قرمز (IR) و فرکانس رادیویی (RF)، امکان دریافت، ذخیره سازی و بازپخش فرمان های کنترل از راه دور نیز در سامانه فراهم شده است.

۶-۱- جمع بندی مراحل انجام پروژه

در ابتدا، نیازمندی های اصلی سامانه بررسی و بر اساس آن، معماری سخت افزاری و نرم افزاری مناسب مبتنی بر برد ESP32 طراحی شد. در بخش سخت افزار، حسگر DHT11 برای اندازه گیری دما و رطوبت، ماژول LDR برای اندازه گیری شدت نور محیط، حسگر MQ-2 برای تشخیص دود و گازهای قابل اشتعال و حسگر PIR برای تشخیص حرکت مورد استفاده قرار گرفتند. همچنین به منظور ایجاد قابلیت کنترل تجهیزات دارای کنترل از راه دور، از گیرنده ها و فرستنده های RF در دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز و یک گیرنده و فرستنده مادون قرمز استفاده شد و یک Active Buzzer نیز برای ایجاد هشدار صوتی در نظر گرفته شد.

در بخش نرم افزار، Firmware سامانه به صورت ماژولار طراحی و پیاده سازی شد و وظایف مختلف از جمله مدیریت حسگرها، ارتباط WiFi و MQTT، سیستم Provisioning، مدیریت حافظه، پردازش سیگنال های RF و IR و مدیریت سیستم هشدار در ماژول های مستقل قرار گرفتند. این ساختار باعث شده است که هر بخش از نرم افزار وظیفه مشخصی داشته باشد و در نتیجه، توسعه، آزمایش، اشکال زدایی و نگهداری Firmware با سهولت بیشتری انجام شود. همچنین برای حفظ اطلاعات پس از خاموش و روشن شدن دستگاه، از سیستم فایل داخلی LittleFS و ساختار JSON برای ذخیره تنظیمات و سیگنال های دریافت شده استفاده شد.

۶-۲- نتایج حاصل از پیاده سازی

نتایج حاصل از پیاده سازی و آزمون های انجام شده نشان داد که سامانه توانسته است قابلیت های اصلی تعریف شده در مرحله طراحی را در قالب یک سیستم یکپارچه پیاده سازی و اجرا کند. حسگر DHT11 توانست مقادیر دما و رطوبت محیط را دریافت کند و ماژول LDR نیز تغییرات شدت نور را از طریق ورودی آنالوگ ESP32 شناسایی کرد. حسگر MQ-2 نیز امکان بررسی میزان دود و گازهای قابل اشتعال و تشخیص عبور مقدار اندازه گیری شده از آستانه تعیین شده را فراهم کرد و در شرایط آزمایش شده، با افزایش مقدار خروجی حسگر، وضعیت هشدار فعال شد. همچنین حسگر PIR در صورت فعال بودن قابلیت تشخیص حرکت، توانست وقوع حرکت را شناسایی کرده و رویداد مربوطه را به سامانه اعلام کند. سیستم هشدار صوتی نیز مطابق طراحی انجام شده عمل کرد و در هنگام فعال شدن وضعیت هشدار، Buzzer به صورت متناوب فعال و غیرفعال شد و پس از قطع وضعیت هشدار، عملکرد آن متوقف گردید. در

بخش ارتباطی، ESP32 توانست به شبکه WiFi متصل شده و پس از آن ارتباط خود را با MQTT Broker برقرار کند و پیام های مربوط به داده های حسگرها و فرمان های کنترلی را ارسال و دریافت نماید. همچنین داشبورد تحت وب توانست اطلاعات دریافت شده از MQTT را به صورت زنده نمایش داده و امکان ارسال فرمان های کنترلی به دستگاه را فراهم کند.

یکی دیگر از نتایج مهم پروژه، پیاده سازی سیستم دریافت و بازپخش سیگنال های RF در دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز و سیستم دریافت و بازپخش سیگنال های IR است. در آزمون های انجام شده، سیگنال های RF و IR از کنترل های مربوطه دریافت، با نام مشخص ذخیره و در زمان موردنیاز مجدداً ارسال شدند و همچنین قابلیت فهرست کردن و حذف سیگنال های ذخیره شده نیز به درستی عمل کرد. استفاده از LittleFS نیز باعث شد اطلاعات مربوط به تنظیمات Provisioning، وضعیت تشخیص حرکت و سیگنال های ذخیره شده پس از راه اندازی مجدد دستگاه حفظ شوند.

در بخش Provisioning، دستگاه توانست یک Access Point اختصاصی ایجاد کرده و پس از اتصال کاربر، صفحه پیکربندی را به صورت خودکار نمایش دهد. اطلاعات وارد شده توسط کاربر پس از اعتبارسنجی در حافظه داخلی ذخیره شد و دستگاه با راه اندازی مجدد، تنظیمات جدید را برای اتصال به شبکه و MQTT Broker مورد استفاده قرار داد. در مجموع، نتایج آزمون های انجام شده نشان داد که اجزای اصلی سامانه و ارتباط میان بخش های سخت افزاری و نرم افزاری مطابق نیازمندی های تعریف شده عمل کرده اند.

۳-۶- محدودیت های پروژه

با وجود پیاده سازی و عملکرد صحیح قابلیت های اصلی، سامانه حاضر دارای محدودیت هایی است که باید در هنگام استفاده از آن مورد توجه قرار گیرد. یکی از مهم ترین محدودیت ها مربوط به حسگر MQ-2 است؛ زیرا در این پروژه مقدار خروجی آن به صورت مستقیم از ADC خوانده شده و با یک مقدار آستانه نرم افزاری مقایسه می شود، بنابراین نتیجه اندازه گیری می تواند تحت تأثیر عواملی مانند شرایط محیطی، زمان گرم شدن حسگر، کیفیت حسگر و نحوه قرارگیری آن قرار گیرد و استفاده از این سامانه به عنوان یک سیستم استاندارد اعلام حریق، نیازمند کالیبراسیون دقیق و انجام آزمون های تخصصی و استاندارد خواهد بود.

محدودیت دیگر مربوط به سیستم های RF و IR است، زیرا امکان دریافت و بازپخش سیگنال ها به نوع پروتکل، مشخصات سیگنال، فرکانس مورد استفاده و سازگاری تجهیزات با سخت افزار و کتابخانه های به کار رفته وابسته است؛ بنابراین نمی توان تضمین کرد که تمامی تجهیزات موجود در بازار با این سامانه قابل کنترل باشند. در بخش ارتباطی نیز استفاده از یک MQTT Broker عمومی در پیاده سازی فعلی، اگرچه برای توسعه و آزمایش سامانه مناسب بوده است، اما برای استفاده در محیط های واقعی و کاربردهای حساس توصیه نمی شود و لازم است از Broker اختصاصی، احراز هویت مناسب و ارتباط رمزنگاری شده استفاده شود.

۴-۶- پیشنهاد های توسعه در آینده

برای توسعه نسخه های آینده سامانه، می توان مجموعه ای از قابلیت های جدید را به سیستم اضافه کرد. یکی از مهم ترین موارد، افزایش امنیت ارتباطات است که می تواند با استفاده از MQTT over TLS، احراز

هویت کاربران و تجهیزات و مدیریت مناسب گواهی های دیجیتال انجام شود. همچنین به جای استفاده از Broker عمومی، می توان یک MQTT Broker اختصاصی را بر روی سرور یا زیرساخت ابری مستقر کرد تا کنترل بیشتری بر دسترسی کاربران، امنیت اطلاعات و مدیریت تجهیزات وجود داشته باشد.

از دیگر قابلیت های پیشنهادی، می توان به پیاده سازی بروزرسانی Firmware به صورت Over-The-Air یا OTA اشاره کرد که امکان بروزرسانی نرم افزار ESP32 را بدون نیاز به اتصال فیزیکی دستگاه به رایانه فراهم می کند. علاوه بر این، می توان روش های ارتباطی دیگری مانند GSM، LoRa یا سایر فناوری های ارتباطی را در کنار WiFi مورد استفاده قرار داد تا در صورت قطع شدن ارتباط WiFi، مسیر جایگزینی برای ارسال اطلاعات و هشدارها وجود داشته باشد.

در بخش پایش محیط نیز می توان با اضافه کردن حسگر های دقیق تر و کالیبره شده، قابلیت تشخیص شرایط خطر را افزایش داد و الگوریتم های پیشرفته تری برای تحلیل همزمان داده های چند حسگر به کار گرفت. همچنین توسعه داشبورد وب و اضافه کردن امکاناتی مانند ذخیره سازی تاریخچه داده ها، نمایش نمودار های بلند مدت، مدیریت چندین دستگاه و تعریف سطوح دسترسی مختلف می تواند قابلیت استفاده از سامانه را در مقیاس بزرگ تر افزایش دهد.

۵-۶- جمع بندی نهایی

در مجموع، نتایج حاصل از طراحی، پیاده سازی و آزمون سامانه نشان می دهد که با استفاده از یک برد کم هزینه ESP32 و ترکیب حسگر های محیطی، ارتباط WiFi و MQTT و قابلیت های RF و IR می توان یک هاب هوشمند چند منظوره برای پایش و کنترل محیط ایجاد کرد. سامانه طراحی شده علاوه بر دریافت و نمایش اطلاعات حسگرها و تشخیص برخی شرایط غیرعادی، امکان ایجاد هشدار صوتی، کنترل تجهیزات دارای کنترل از راه دور، ذخیره سازی اطلاعات در حافظه داخلی، پیکربندی بدون نیاز به تغییر Firmware و مدیریت دستگاه از طریق داشبورد وب را فراهم کرده است.

فصل ۷:

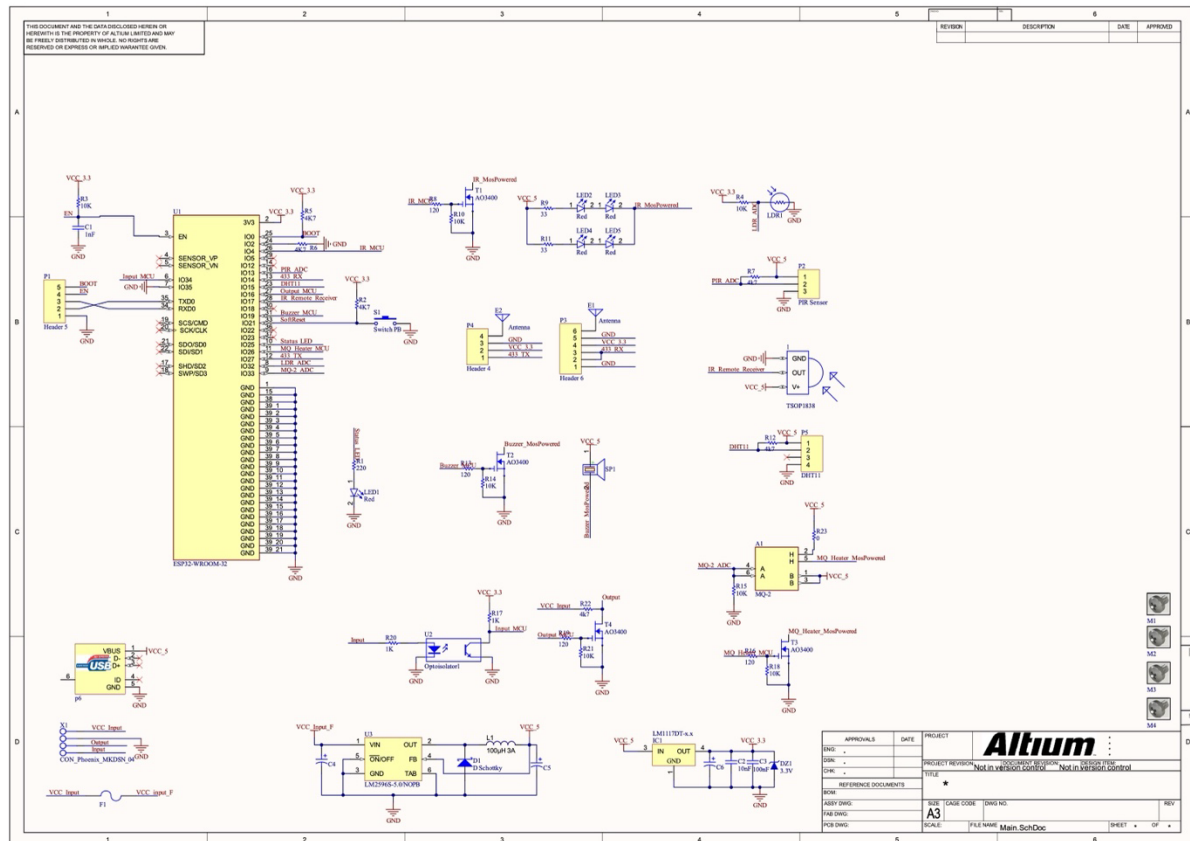
مراجع

- [1] National Institute of Standards and Technology (NIST), "IoT Device," 2024. [Online]. Available: https://csrc.nist.gov/glossary/term/iot_device. [Accessed August 2026].
- [2] J. Voas, "Networks of 'Things'," National Institute of Standards and Technology, NIST Special Publications 800-183, 2016.
- [3] Espressif Systems, "ESP32 series Datasheet," Espressif Systems.
- [4] A. Banks, E. Briggs, K. Borgendale and R. Gupta, "MQTT Version 5.0," OASIS Standards, 2019.
- [5] S. Ullrich, "rc-switch: Arduino library to operate 315/433 MHz remote controlled devices," [Online]. Available: <https://github.com/sui77/rc-switch>. [Accessed August 2026].
- [6] D. Conran et al., "IRremoteESP8266: Infrared remote library for ESP8266/ESP32," [Online]. Available: <https://github.com/crankyoldgit/IRremoteESP8266>. [Accessed August 2026].
- [7] Hanwei Electronics Group Corporation, "MQ-2 Gas Sensor Datasheet," Hanwei Electronics Group Corporation.
- [8] Adafruit Industries, "DHT Sensor Library Documentation," [Online]. Available: <https://adafruit.github.io/DHT-sensor-library/>. [Accessed August 2026].
- [9] Espressif Systems, "Arduino ESP32 Wi-Fi Documentation," Espressif Systems.
- [10] Arduino ESP32, "LittleFS File System Documentation," Espressif Systems.

فصل ۸:

پیوست ها

۸-۱- نقشه شماتیک مدار



۸-۲- کد منبع پروژه

کد منبع کامل Firmware مربوط به سامانه طراحی شده، به همراه فایل های پیکربندی و مستندات مرتبط، در مخزن GitHub پروژه قرار داده شده است. <https://github.com/samansayad93/homeintelligence>